Knowledge-Based Editor

Volume 2

Programmer's Apprentice Group

MIT AI Lab

January, 1985

Table of Contents: Knowledge Based Editor

Volume 1

OZ:<KBE.DOC>358.TEXT
OZ:<KBE.TEST>358.LISP

OZ:<KBE>KBE.INDEX

OZ:<KBE>LISPM.INIT
OZ:<KBE>SYSDEF.LISP

OZ:<KBE.SOURCE>BASIC.LISP
OZ:<KBE.SOURCE>NOISE.LISP
OZ:<KBE.SOURCE>SETS.LISP
OZ:<KBE.SOURCE>CONDITIONS.LISP
OZ:<KBE.SOURCE>ENGLISH.LISP
OZ:<KBE.SOURCE>SYSTEM-MAGIC.LISP
OZ:<KBE.SOURCE>FLAVOR-MAGIC.LISP
OZ:<KBE.SOURCE>MACRO.LISP
OZ:<KBE.SOURCE>KVARS.LISP
OZ:<KBE.SOURCE>KMAC.LISP
OZ:<KBE.SOURCE>INTERFACE-MACROS.LISP
OZ:<KBE.SOURCE>MBOX.LISP
OZ:<KBE.SOURCE>POST-OFFICE.LISP
OZ:<KBE.SOURCE>BEHAVIOR.LISP
OZ:<KBE.SOURCE>EVENTS.LISP
OZ:<KBE.SOURCE>ACTOR.LISP
OZ:<KBE.SOURCE>COMPLETE.LISP
OZ:<KBE.SOURCE>COMMENT-MACRO.LISP
OZ:<KBE.SOURCE>READMACRO.LISP
OZ:<KBE.SOURCE>OUTIL.LISP
OZ:<KBE.SOURCE>DTRACE.LISP
OZ:<KBE.SOURCE>DUTIL.LISP
OZ:<KBE.SOURCE>DMAC.LISP
OZ:<KBE.SOURCE>DVARS.LISP
OZ:<KBE.SOURCE>DB.LISP
OZ:<KBE.SOURCE>GC.LISP
OZ:<KBE.SOURCE>DDEBUG.LISP

Volume 2

OZ:<KBE.SOURCE>SURFACE.LISP
OZ:<KBE.SOURCE>EXPAND.LISP
OZ:<KBE.SOURCE>PRIMOP.LISP
OZ:<KBE.SOURCE>DFLOW.LISP
OZ:<KBE.SOURCE>UTIL.LISP
OZ:<KBE.SOURCE>SEGTRN.LISP
OZ:<KBE.SOURCE>ANALYZE.LISP
OZ:<KBE.SOURCE>CODER.LISP

Volume 3

OZ:<KBE.SOURCE>PSEUDO.LISP
OZ:<KBE.SOURCE>DICT.LISP
OZ:<KBE.SOURCE>GRAM.LISP
OZ:<KBE.SOURCE>REFER.LISP
OZ:<KBE.SOURCE>VERBS.LISP
OZ:<KBE.SOURCE>IMPLEMENT.LISP
OZ:<KBE.SOURCE>ADA-EVAL.LISP
OZ:<KBE.SOURCE>ADA-ANAL.LISP
OZ:<KBE.SOURCE>ADA-PARSE.LISP
OZ:<KBE.SOURCE>ADA-UNPARSE.LISP
OZ:<KBE.SOURCE>VIEW.LISP
OZ:<KBE.SOURCE>PLI.LISP
OZ:<KBE.SOURCE>DEFLIB.LISP
OZ:<KBE.SOURCE>SYSLIB.LISP
OZ:<KBE.SOURCE>KBELIB.LISP
OZ:<KBE.SOURCE>DEMO-LIB.LISP
OZ:<KBE.SOURCE>DEMO-ADA-LIB.LISP

Volume 4

OZ:<KBE.SOURCE>BRIDGE.LISP
OZ:<KBE.SOURCE>KAOS.LISP
OZ:<KBE.SOURCE>SOCIETY.LISP
OZ:<KBE.SOURCE>STANDARD-ACTOR.LISP
OZ:<KBE.SOURCE>REMOTE.LISP
OZ:<KBE.SOURCE>OVERLAPPING-REGIONS.LISP
OZ:<KBE.SOURCE>MOUSE-SENSITIVE-REGIONS.LISP
OZ:<KBE.SOURCE>OPTIONS.LISP
OZ:<KBE.SOURCE>SIMPLE.LISP
OZ:<KBE.SOURCE>SIMPLE-ACTOR.LISP
OZ:<KBE.SOURCE>MAGIC-ACTOR.LISP
OZ:<KBE.SOURCE>TV-MAGIC.LISP
OZ:<KBE.SOURCE>ZWEI-MAGIC.LISP
OZ:<KBE.SOURCE>POSSIBILITIES.LISP
OZ:<KBE.SOURCE>BELIEF.LISP
OZ:<KBE.SOURCE>EDIT.LISP
OZ:<KBE.SOURCE>EDIT-ACTOR.LISP
OZ:<KBE.SOURCE>CONSISTENCY.LISP
OZ:<KBE.SOURCE>EDIT-COMMANDS.LISP
OZ:<KBE.SOURCE>EDIT-MISC.LISP
OZ:<KBE.SOURCE>EDIT-SIMPLE.LISP
OZ:<KBE.SOURCE>EDIT-DEBUG.LISP
OZ:<KBE.SOURCE>KBE-MODE.LISP
OZ:<KBE.SOURCE>FOCUS.LISP
OZ:<KBE.SOURCE>TEST.LISP
OZ:<KBE.SOURCE>DOC.LISP
OZ:<KBE.SOURCE>PARSE.LISP
OZ:<KBE.SOURCE>EXPLAIN.LISP
OZ:<KBE.SOURCE>DOCUMENT.LISP
OZ:<KBE.SOURCE>PBM-HANDLERS.LISP
OZ:<KBE.SOURCE>INIT.LISP
OZ:<KBE.SOURCE>DEBUG.LISP

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; the prefix in this file is 6

(declare (special DEFINING-NAME LASTSEGCASE SUPERSEGCASE ALIST DF-NUM
                  TAGS SIMPLE-EXPRS RETVAR FREE-IN FREE-OUT LAST-HUNK
                  FIRST-HUNK INFO FLOWS-HELPS-ANALYZE DF-NUMS DF-NAME
                  DEFINING-CODE RE-FLOW-EVAL INLINE-LIST UNGROUP-LIST
                  *db-context-names* 6DFN-PORT-LIST K*PROG-STACK
                  MNEMONIC-NAME)
         (*expr analyze)
         (*fexpr deffacts))

;this is a lisp to surface plan converter.  self contained, it works
;on LISP progrmas.  In combination with the program FREAD it can be
;used to convert FORTRAN.  In combination with the programs of GSB it
;can be used to convert COBOL.
; in order to simplify things, these programs always assume that the
;input program is COMPLETELY SYNTACTICALLY CORRECT.  it never goes
;out of its way to check anything.  as a result if there are syntactic
;problems, you can get some pretty dumb garbage produced.
; Note that the programs assume no aliassing of inputs to fns with
;side-effects.  It is only really tested to work with looping programs,
;though there is code in here to deal with recursive programs.  Further
;its knowledge of side-effects is weak.  it gets the right dflow in an
;interesting class of situations, but not all the time.  it is easy
;to write a program it will get mixed up on.

;The top-level function is surface-analyze.
;It works in several basic passes.
;1. the fn expand-special-forms is used to preprocess the input so that every fexpr
;   and macro is removed from it.  The result uses a set of only
;   8 6fexprs: K*LAMBDA, K*PROG, K*GO, K*TAG, K*IFNULL, K*IF,
;   K*PSETQ, and K*QUOTE.
;   everything else in the file only has to deal with these things,
;   and ordinary exprs.
;2. the fn 6flow-eval operating in analogy to a lisp evaluator, runs over the
;   program creating all of the segments, and all of the control flow.
;   Annotations (@IOs) concerning data flow are made, but the actual creation
;   of this flow is left until later.  Note that at this stage, segments
;   are put in corresponding to setqs in order to facilitate the next stages.
;   these segs are removed later.  Information about dflow is
;   communicated to the next stage as follows:  each input and output
;   object is given a DF property which is a number.  Every port that
;   reads or writes the same variable gets the same number.
;   when the next stage runs over the program, it uses these numbers in
;   order to reconstruct the data flow relationships.
;3. the fn 6data-flow works in four stages:
; a. first it computes what variables are LIVE in each strait line section
;    of the program.
; b. next it computes what data flow has to be added due to side-effects.
;    Note that this step uses a weak but useful model of when side-effects
;    create additional dflow.
; c. it determines what dflows have to be joined at each join.
; d. lastly, it constructs the actual dflow assertions.
;4. the fn 6cleanup runs over the program performing a few cleanup chores
;   such as getting rid of the setq segs.
```

```
;;; This is the top-level entry-point to the surface analyzer.
;;; Apply it to a defun or define-cliche or an sexpr
;;; The segcase of the outer segment produced is returned.

(defun surface-analyze (code)
  (declare (special *input-language*))
  (let* ((DEFINING-CODE code)
         (RE-FLOW-EVAL nil))
    (cond ((and (listp code) (eq (car code) 'defun))
           (letS ()                    ;kill all old definitions
             (kill-seg-and-its-subsegs
               (~seg~segcase
                 (Elist (??& * @seg 'lambda-exp (cadr code))))))
           (let* ((DEFINING-NAME (cadr code)) ;only works for simple defuns
                  (MNEMONIC-NAME (cadr code))
                  (segcase (6flow-analyze '(lambda ., (cddr code)))))
             (assert-fact (&~name (-& segcase @seg) DEFINING-NAME))
             segcase))
          ((and (listp code) (eq (car code) 'define-cliche))
           (letS ()                    ;kill all old definitions
             (kill-seg-and-its-subsegs
               (~seg~segcase (Elist (??& * cliche-def (cadr code))))))
           (let* ((DEFINING-NAME nil)
                  (MNEMONIC-NAME (cadr code))
                  (name (cadr code))
                  (cdr-plist (caddr code))
                  (segcase
                    (6flow-analyze
                      '(progn ., (6not-just-variable (cdddr code))))))
             (>& segcase cliche-def name cdr-plist)
             segcase))
;ada stuff
          ((and (listp code)
                (eq (car code) 'def-ada-top)
                (memq (caar (cddddr code)) '(def-ada-fn def-ada-package)))
           (letS ()                    ;kill all old definitions
             (kill-seg-and-its-subsegs
               (~seg~segcase
                 (Elist (??& * @seg 'lambda-exp (cadr code))))))
           (let* ((DEFINING-NAME (cadr code))
                  (MNEMONIC-NAME (cadr code))
                  (segcase (6flow-analyze
                             '(,(caseq (caar (cddddr code))
                                  (def-ada-fn 'ada-top-fn)
                                  (def-ada-package 'ada-top-package))
                               ., (cdar (cddddr code))))))
             (assert-fact (&~name (-& segcase @seg) DEFINING-NAME))
             segcase))
          ((and (listp code)
                (eq (car code) 'def-ada-top)
                (eq (caar (cddddr code)) 'def-ada-cliche))
           (letS () ;kill all old definitions
             (kill-seg-and-its-subsegs
               (~seg~segcase (Elist (??& * cliche-def (cadr code))))))
           (let* ((DEFINING-NAME nil)
                  (MNEMONIC-NAME (cadr code))
                  (name (cadr code))
                  (cdr-plist (caddr (car (cddddr code))))
                  (segcase
                    (6flow-analyze (car (cddddr code)))))
             (>& segcase cliche-def name cdr-plist)
             segcase))
;should proably add some more stuff here for ada case.
;the stuff that is below is making up for the fact that
;the separation between lisp and ada isn't clean.
          (t (when (eq *input-language* 'ada)
               (if (numberp code) (setq code '(int-lit ,code)))
```

```
    (if (stringp code) (setq code '(str-lit ,code)))
    (if (and (listp code) (equal (car code) '(string-literal)))
        (set code '(str-lit ,(cadr code)))))
  (let* ((DEFINING-NAME nil)
         (MNEMONIC-NAME nil))
    (6kill-unneeded-top-seg
      (6flow-analyze
        (car (6not-just-variable (list code))))))))))
```

```
(defun 6not-just-variable (list)
  (cond ((and (= 1 (length list)) (symbolp (car list)))
         '((5noop ,(car list))))
        (T list)))

;note that it is important that we keep the top seg
;if the inner seg is a slot, so that the slot will
;stay distinct from other slots.  The problem would be if
;we implemented a slot with another naked slot, then
;we would have two slots in the same place which would mess
;things up.

(defun 6kill-unneeded-top-seg (segcase)
  (cond ((and (= 1 (length (??subseg (~seg segcase))))
              (null (??& segcase slot)))
         (let* ((subsegcase (~subseg (-& segcase subseg))))
           (kill-seg (~seg segcase))
           subsegcase))
        (T segcase)))

;This is the internal entry into the flow analyzer.  It returns the
;segcase of the plan produced.  It follows the stages outlined in the
;first page of this file.

(defun 6flow-analyze (code)
  (db-context-was <FLOW)
  (let ((DF-NUMS nil)
        (DF-NAME nil)
        (6DFN-PORT-LIST nil)
        (K*PROG-STACK nil)
        segcase)
    (setq code (expand-special-forms code))
    (unwind-protect
      (progn (setq segcase (6flow-eval code))
             (6data-flow segcase))
      (dolist (seg *db-segs*) (remprop seg 'df)))
    (6cleanup segcase)
    (db-context-is >FLOW)
    (push-db-context)
    segcase))
```

```
;;; This tells you if two segments have the same io behavior.
(defun 6sameio (seg1 seg2)
  (set= (6abstractio seg1) (6abstractio seg2)))

(defun 6abstractio (seg)
  (append (6aio2 (??input seg)) (6aio2 (??output seg))))

(defun 6aio2 (ports)
  (over ports -> port
        (??& port @io) -> @ios
         (~01 ~kind @ios) -> kind
    bag [(~keyword port)
         (caseq kind ((actual-arg lambda-arg) 'arg) (t kind))
         (caseq kind
           ((lambda-arg actual-arg) (~01 ~argno @ios))
           (side-effect (car (6aio2 (??& (&* (~seg~segcase port))
                                         input * (~id port)))))
           (free-var (~01 ~name @ios)))]))

;;; This is for backward compatibility with old programs.
;;; note it hacks COBOL right.

(defmacro define (name &body rest)
  (cond ((and (listp (car rest)) (eq (caar rest) 'cobol-fn))
         '(surface-analyze '(defun ,name nil ,@rest)))
        (T (grumble "DEFINE is obsolete.  Use SURFACE-ANALYZE.")
           '(surface-analyze '(defun ,name ,@rest)))))

;this allows you to define a group of forms together as one.  they
;must be ordered in the call so that each definition FOLLOWS every
;call of a fn except possibly a call inside itself.  (This implies
;that there can be self-recursion, but no mutual recursion.)  The first
;form is the TOP form, and its segcase is returned.  the plans for the
;other forms are substituted in at the points of call, and then forgotten.

(defmacro define-consolidated (fn-names &body defs)
  (let ((INLINE-LIST fn-names) UNGROUP-LIST return-segcase)
    (dolist (form (reverse defs))
      (setq return-segcase (eval form)))
    (assert-fact (&~code (-& return-segcase @seg) defs))
    (over (cdr INLINE-LIST) (kill-seg-and-its-subsegs (?lambda-exp _)))
    (over UNGROUP-LIST
      select (null (??& (&* _) recursive))
      (ungroup _))
    '',return-segcase))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; This is part of the KBE.  That is its excuse for having sixes
;;; in front of everything.  It can also stand alone as a sort of
;;; super macroexpander.

(declare (special DEFINING-CODE K*PROG-STACK))
```

```
;;; In order to simplify the task of making surface plans for programs
;;; the following defines constructs which embody all of the
;;; key features of sequential programming in LISP and FORTRAN.
;;; They have all of the features found in LAMBDA, PROGN, PROG, LET, COND,
;;; AND, OR, MAPCAR, DO, (and _SUBROUTINE, _FUNCTION, _IF) etc.
;;; THEY ARE THE ONLY FEXPRS THAT THE REST OF THE SYSTEM EVER
;;; HAS TO DEAL WITH.

;;; Everything in this file revoles around these fexprs (called 6fexprs).
;;; They are: K*LAMBDA, K*PROG, K*GO, K*TAG, K*IFNULL, K*IF,
;;; K*PSETQ, and K*QUOTE
;;; (STRING for cobol), (_LT, _EQ, _GT for fortran).

;;; (K*LAMBDA return? invars body) where
;;;     return? - is non nil if the value of the body is to be returned
;;;               as a return
;;;               value (as in lisp).
;;;     invars - are a list of input variables a la lambda (but note that
;;;              like fortran if you setq one of these variables,
;;;              then it becomes an OUTPUT as well as an input).
;;;     body - is a single form whose value is the return value (if any).
;;;            In general, the body will be a call on K*PROG
;;;   It is expected that the outermost form of a fn being defined will
;;;   be a K*LAMBDA, and that there will not be any internal K*LAMBDAs.
;;; (K*PROG boundvars statement*n ret-vars)   where
;;;     boundvars - are a list of var value pairs a la let (the values are
;;;                 evaluated before the boundvars are bound.)
;;;                 (Note that these are not inputs, and assigning to
;;;                 them doesn't effect anything on the outside.)
;;;     statement - is any lisp form.
;;;                 the value of the last form is returned.
;;;     ret-vars  - is a list of zero or more variables who's values are
;;;                 returned multiplely.  only zero and one are
;;;                 supported now.
;;;   Free variables can be set and read anywhere in a K*PROG.
;;;   The following special forms can appear in a K*PROG
;;; (K*TAG lab) is a label
;;; (K*GO lab) branches to lab unconditionally
;;; (K*IFNULL sexp lab) branches iff sexp is null.
;;; (K*IF sexp lab) branches iff sexp is not null.
;;;   the reason we have two of these
;;;   is so that we don't ever have to have a NOT.
;;; [_lt _eq _gt are additional predicates used by fortran]

;;; (K*QUOTE form) is normal lisp quote.
;;;   Note that to further simplify things,
;;;   expand-special-forms makes sure that every thing that gets
;;;   evaluated to itself (eg numbers) is wrapped in an explicit K*QUOTE.
;;; (STRING atom) is an additional quoter used by cobol.
;;; (K*PSETQ atom sexp atom sexp ...) is normal lisp PSETQ.

;;; (K*SLOT form slotname slottype)
;;; instances of kbe-/#-comment-macro all get turned into segments
;;; and further processing happens on them as if they were simple
;;; funcalls.  The form is a call on the function nil
;;; if there is no code.  If the code is a variable then it is a
;;; call on 6variable.  Otherwise it is the code.
```

```
;;; The following k*expanders convert every fexpr that can occure in an
;;; input program into code using only the 6FEXPRs.
;;; Each k*expander expects a single arg: the entire form to be redone.
;;; It returns an entire new form to evaluate.  The original code is
;;; not side-effected, but neither is it copied.

;;; The variable names which are generated by k*expanders, and therefore
;;; should not be used as guides for names to be created by the coder,
;;; begin with "_" to signal this fact.

;;; This fn is called with a form to translate that entire form into the
;;; 6fexprs.
;;; Note that it relies on the fact that every fexpr in the input
;;; must have a k*expander associated with it that tells how to translate it
;;; into terms of the 6fexprs.  As a result the only fexprs that
;;; expand-special-forms actually has to understand are the 6fexprs.
;;; Macros are expanded fully but substs are not.

;;; Note that everything is set up so that the original form is not
;;; side-effected, but so that there is maximal sharing between the
;;; result, and the original form.
;;;
;;; Note that macroexpansion is done only AFTER k*expanders are checked
;;; for, so that a k*expander can override a macro definition.

;;; also note that anything that conses up any of the K* things is
;;; reponsible for calling expand-special-forms on anything put
;;; inside the k* thing.

;;; It rebinds DEFINING-CODE so that the file can stand alone.

(defun expand-special-forms (form)
  (let ((DEFINING-CODE form))
    (cond ((self-evalling-form form) form)
          ((symbolp form) form)         ;Variable reference
          ((symbolp (car form))         ;Function application
           (cond ((get (car form) '6fexpr) form)      ;K* things
                 ((get (car form) 'k*expander)
                  (expand-special-forms
                    (funcall (get (car form) 'k*expander) form)))
                 ((not (eq form
                           (setq form (macroexpand-1-no-substs form))))
                  (expand-special-forms form))
                 (t (let ((x '(,(car form)
                               ,@(mapcar #'expand-special-forms
                                         (cdr form)))))
                      (cond ((equal x form) form)     ;Maximize sharing
                            (t x))))))
          ((6lambda-like-fn? (car form))      ;Expand inline lambdas
           (expand-special-forms (inline-lambda-k*expander form)))
          (t (barf "EXPAND-SPECIAL-FORMS: Cannot handle form.~% Form: ~:="
                   form)))))
```

```
(defun self-evalling-form (form)
  (or (memq form '(t nil))
      (numberp form)
      (stringp form)))

;;; Is a function a literal lambda or relative?
(defun 6lambda-like-fn? (fn)
  (and (listp fn)
       (memq (car fn) '(lambda named-lambda subst named-subst))))

;;; This is copied from lispm;qfctns.  The only difference is that
;;; it does not expand substs, which are functions so far as the
;;; PA is concerned.
(defun macroexpand-no-substs (macro-call &optional ignore)
    (do ((tm macro-call (macroexpand-1-no-substs tm))
         (otm nil tm))
        ((or (eq tm otm) (atom tm)) tm)))

(defun macroexpand-1-no-substs (macro-call &optional ignore)
    (prog (tm)
          (and (atom macro-call) (return macro-call))
          (cond ((not (atom (car macro-call)))
                 (return macro-call))
                ((not (symbolp (car macro-call))))
                ((setq tm (si:declared-definition (car macro-call)))
                 (return (cond ((atom tm) macro-call)
                               ((eq (car tm) 'macro)
                                (funcall (cdr tm) macro-call))
                               (t macro-call))))
                ((setq tm (si:open-code-p (car macro-call)))
                 (return (cons tm (cdr macro-call)))))
          (return macro-call)))

;this is called below in order to determine whether a piece of code
;is actually in the orginal form or not.

(defun 6good-code (form)
  (if (6is-in form DEFINING-CODE) (ncons form)))

;note this only checks for things that are cars of things.

(defun 6is-in (form list)
  (cond ((eq form list))
        ((listp list)
         (or (6is-in form (car list)) (6is-in form (cdr list))))))
```

```
;this is used by the analyzer instead of the true definition.
;the problem is that it cannot deal with the file-closes in loop fragments.
;the file-opens must stay so that cliches with
;file-name roles will have boxes for them to point to.

(defun (with-open-file k*expander) (form)
  (let (((nil (file path-name mode) . body) form))
    '(let ((,file (with-open-file-open ,path-name ,mode)))
       .,body)))

;;; These are the k*expanders for LISP
;;;this isn't acrually used anymore.  Defuns are parsed by flow-analyze.
(defun (:property defun k*expander) (form)
  (let* ((good (si:defun-compatibility (cdr form))))
    (si:process-defun-body (cadr good) (cddr good))))

;;; This is a macro, but we override the macroexpansion.
(defun (:property defsubst k*expander) (form)
  (warn "DEFSUBST will be turned into a DEFUN")
  (funcall (get 'defun 'k*expander) form))
```

```
;this handles the new style slot comment things
;note that compound names are just made into one big name
;and processed completely only when the analyzer is done.
;Note that THEs are optional in the comment.  Note that we
;cons up a list with function NIL if there is no code.

(defun (:property kbe-/#-comment-macro k*expander) (form)
  (let* (((code com) (cdr form))
         (names (Rlist (filterS #'(lambda (x)
                                    (not (string-equal x "the")))
                                (Elist com))))
         (body (cond ((null code) (list nil))
                     ((and (listp (car code)) (eq (caar code) '...))
                      (cons nil (cdar code)))
                     ((not (garden-variety-fn-apply (car code)))
                      (cons '6noop code))
                     (T (car code))))
         (slottype (cond ((string-equal (car names) "input")
                          (pop names)
                          'input)
                         ((string-equal (car names) "output")
                          (if (eq (car body) nil)
                              (barf "Output slot has no source: ~S" form))
                          (pop names)
                          'output)
                         (T 'segment)))
         (slotname (cond ((null names) (barf "Slot with no name: ~S" form))
                         ((intern (format nil "~{~A~^-~}" names)
                                  *kbe-package*)))))
    '(K*SLOT ,(expand-special-forms body) ,slotname ,slottype)))

;this checks that we have a simple function that won't blow up when
;we directly instantiate it later -- see (K*SLOT 6FEXPR)

(defun garden-variety-fn-apply (call)
  (and (listp call)
       (symbolp (car call))
       (null (get (car call) 'k*expander))))

;;; Sequential setq is turned into a series of k*psetq forms.
;;; Note that (setq) => nil and (setq foo bar baz) sets baz to nil.

(defun (:property setq k*expander) (body)
  (setq body (cdr body))
  (cond ((null body) nil)
        ((null (cddr body))
         '(k*psetq ,(car body) ,(expand-special-forms (cadr body))))
        (t '(progn ,@(do ((body-iter body (cddr body-iter))
                          var val k*psetqs)
                         ((null body-iter) (nreverse k*psetqs))
                       (setq var (car body-iter) val (cadr body-iter))
                       (push '(k*psetq ,var ,(expand-special-forms val))
                             k*psetqs))))))

(defun (quote k*expander) (form)
  '(K*QUOTE ,@(cdr form)))

;this is a pretty cludgy treatment of setf that
;works in the current demo.  It just makes setf
;act like a normal function.

(defun (setf k*expander) (form)
  (let* (((accessor value) (cdr form))
         access-list var)
    (prog ()
      L (when (symbolp accessor) (setq var accessor) (return nil))
        (if (or (not (listp accessor))
                (not (listp (cdr accessor)))
```

```
            (cddr accessor))
          (barf "Cannot handle complex SETF: ~A" form))
        (push (car accessor) access-list)
        (setq accessor (cadr accessor))
        (go L))
    (setq access-list (reverse access-list))
    '(k*setf ',access-list ,var ,value)))

;this makes it so that things print out again as a setf.

(pp:define-list-print-function k*setf (form s)
  (let* ((((nil access-list) var value) (cdr form))
         (accessor (reduceS #'(lambda (x y) (list y x))
                            var
                            (Elist (reverse access-list)))))
    (format s "~=" '(setf ,accessor ,value))))
```

```
;;; Cond.

(defconst 6used-temp nil
  "Tells whether the variable _cond-temp has been used in a cond.")

;;; Wrap a PROG around the body.  Bind _cond-ret and also
;;; _cond-temp if 6used-temp.

(defun (:property cond k*expander) (clauses)
  (setq clauses (cdr clauses))
  (let* ((6used-temp nil)
         (body (cond-k*expander1 clauses)))
    `(prog T (_cond-ret ,@(if 6used-temp '(_cond-temp)))
       ,@body
       (return-from T _cond-ret))))

;;; Recursively build prog body.  Each clause gets a tag, which is
;;; an _ gensym, or _cond-end after the last one.

(defun cond-k*expander1 (clauses)
  (cond ((null clauses) nil)
        (t (let* ((next (cond ((null (cdr clauses)) '_cond-end)
                              (t (gensym '_)))))
             `(,@(cond-k*expander-clause
                   (car clauses) next (null (cdr clauses)))
               ;; Goto end if fails.
               ,@(if (cdr clauses) '((go _cond-end)))
               ,next
               ,@(cond-k*expander1 (cdr clauses)))))))

;;; This produces the inline code for a clause.
;;; Much hair is required to determine what the return value is.

(defun cond-k*expander-clause (clause next last-clause?)
  (cond ((null clause) nil)
        ((null (cdr clause))
         (cond ((or last-clause? (eq (car clause) 't))
                '((setq _cond-ret ,(car clause))))
               ((atom (car clause))
                '((k*ifnull ,(car clause) ,next)
                  (setq _cond-ret ,(car clause))))
               (t ;;(not (atom (car clause)))
                ;; Note we never want to have dflow from a pred when we
                ;; don't absolutely have to.
                (setq 6used-temp t)
                '((setq _cond-temp ,(car clause))
                  (k*ifnull _cond-temp ,next)
                  (setq _cond-ret _cond-temp)))))
        (t `(,@(6and-pred (car clause) next)
             (setq _cond-ret
                   ,(cond ((null (cddr clause)) (cadr clause))
                          (t `(progn ,@(cdr clause)))))))))
```

```
;; the point of all this hair is to get rid of calls on
;; NOT so that the coder will have greater freedom
;; in the end.  this is possible because the presence of nots
;; can be encoded based on whether the basic pred k*if is used
;; rather than k*ifnull.  Also the following is simplified because
;; it is only called when the value of the expression is not going to
;; be used.  this simplifies everything.
;;  Note that NULL is not treated specially, IE every instance
;; of NULL used by the user will always stay around.

(defun 6and-pred (pred next)
  (cond ((eq pred 't) nil)
        ((atom pred) '((k*ifnull ,pred ,next)))
        ((memq (car pred) '(not)) (6or-pred (cadr pred) next))
        ((eq (car pred) 'and)
         (do ((things (cdr pred) (cdr things))
              (result nil))
             ((null things) result)
           (setq result (append result (6and-pred (car things) next)))))
        ((eq (car pred) 'or)
         (let ((this (gensym '_)))
           '(,@(do ((things (cdr pred) (cdr things))
                    (result nil))
                   ((null things) result)
                 (setq result
                       (append result (6or-pred (car things) this))))
             (go ,next) ,this)))
        (t '((k*ifnull ,(expand-special-forms pred) ,next)))))

(defun 6or-pred (pred this)
  (cond ((eq pred 't) '(go ,this))
        ((atom pred) '((k*if ,pred ,this)))
        ((memq (car pred) '(not)) (6and-pred (cadr pred) this))
        ((eq (car pred) 'or)
         (do ((things (cdr pred) (cdr things))
              (result nil))
             ((null things) result)
           (setq result (append result (6or-pred (car things) this)))))
        ((eq (car pred) 'and)
         (let ((next (gensym '_)))
           '(,@(do ((things (cdr pred) (cdr things))
                    (result nil))
                   ((null things) result)
                 (setq result
                       (append result (6and-pred (car things) next))))
             (go ,this) ,next)))
        (t '((k*if ,(expand-special-forms pred) ,this)))))
```

```
;;; note that the key difficulties here are dealing with names, and
;;; multiple return values.  the named KBE-PROG-NAMED is used to fill
;;; a logical gap and simplify things.

;;; each entry on K*PROG-STACK is of the form (name tag ret-vars)
;;; if there is any return, then the tag is set up.  if there are
;;; any return values, then the ret-vars are set up for the
;;; maximum number possible.

;;; note that in order to handle infinite loops we insert a pseudo exit
;;; here which stays in the plan up till the very end of the analysis
;;; phase.  This keeps the cflow complete through these stages.
;;; the pseudo exit is then deleted.

(defun (:property kbe-prog-named k*expander) (form)
  (let* ((name (second form))
         (var-vals (third form))
         (body (rest3 form))
         (inits
           (mapcar #'(lambda (x)
                       (cond ((symbolp x) (list x nil))
                             (T (list (car x)
                                      (expand-special-forms (cadr x))))))
                   var-vals))
         (K*PROG-STACK (cons (list name nil) K*PROG-STACK))
         (exprs (mapcar #'(lambda (x)
                            (cond ((symbolp x) '(k*tag ,x))
                                  (T (expand-special-forms x))))
                        body)))
    (cond ((null (cadar K*PROG-STACK))
           (if (not (and (listp (car (last body)))      ;kludge to detect
                         (eq (caar (last body)) 'go)))  ;infinite loops
               (barf "Odd no exit loop ~:=" form))
           (let ((tag (gensym '_)))
             (rplaca (cdar K*PROG-STACK) tag)
             (setq exprs
                   '(,@(nreverse (cdr (reverse exprs)))
                     (k*infinite-loop-pseudo-exit ,tag)
                     ,@(last exprs))))))
    '(k*prog (,@inits
              ,@(mapcar #'(lambda (x) (list x nil))
                        (cddar K*PROG-STACK)))
      ,@exprs
      (k*tag ,(cadar K*PROG-STACK))
      ,(reverse (cddar K*PROG-STACK)))))
```

```
;;; Anyone care to explain how the hell this works?
(defun (:property return-from k*expander) (form)
  (let* ((name (second form))
         (exprs (rest2 form))
         (entry (assq name K*PROG-STACK)))
    (cond ((null entry)
           (barf "Return not inside PROG or DO~% Form: ~S" form))
          ((cdr exprs)
           (barf "More than one return val not supported yet~% Form: ~S"
                 form)))
    (let* ((tag (or (cadr entry) (gensym '_)))
           (vars (cddr entry)))
      (prog1
        '(k*prog nil
          ,@(do ((expr-iter exprs (cdr expr-iter))
                 (var-iter (reverse vars) (cdr var-iter))
                 var
                 expanded-expr
                 (result nil))
                ((null expr-iter) result)
              (setq expanded-expr (expand-special-forms (car expr-iter))
                    var (car var-iter))
              (push (cond ((and (listp expanded-expr)
                                (eq (car expanded-expr) 'k*prog)
                                (null (car (last expanded-expr)))
                                (null (cdr exprs)))
                           expanded-expr)
                          (t (cond ((null var)
                                    (push (setq var (gensym '_)) vars)))
                             '(k*psetq ,var
                                       ,(expand-special-forms
                                          (car expr-iter)))))
                    result))
          (k*go ,tag)
          nil)
        (rplaca (cdr entry) tag)
        (rplacd (cdr entry) vars)))))

(defun (:property go k*expander) (form)
  (cond ((and (= (length form) 2) (symbolp (cadr form)))
         '(k*go ,(cadr form)))
        (t (barf "Bad GO statement~% Form: ~S" form))))


;the following makes simple cases of LOOP work.  Note that you can only
;use complex cases in complete programs, and the coder never makes them.
;we need this to interface correctly with the way we cludge PROG above.

(defun (:property loop k*expander) (form)
  (let ((body (cdr form)))
    (if (and (symbolp (car body)) (string-equal (car body) 'do))
        (pop body))
    (if (Rand-fast (listp (Elist body)))
        '(prog () L ,@ body (go L))
        (macroexpand-1 form))))
```

```
;;; Note how this prevents setqs to the inputs from being propogated to
;;; the outside, while side-effects to them are propagated within
;;; Handles only the &aux keyword for now.
;;; this is only intended to be called on top level forms.

(defmacro def-lambda-like-k*expander (name named?)
  '(defun (:property ,name k*expander) (form)
     (let* ((vars ,(cond (named? '(third form)) (t '(second form))))
            (body ,(cond (named? '(nthcdr 3 form)) (t '(nthcdr 2 form))))
            (local-vars (cdr (memq '&aux vars)))
            (invars (cond (local-vars
                           (reverse (cdr (memq '&aux (reverse vars)))))
                          (T vars))))
       '(k*lambda t ,invars
          ,(expand-special-forms
            '(prog T (,@(mapcar #'(lambda (var) (list var var))
                                invars)
                      ,@(mapcar #'(lambda (var) (list var nil))
                                local-vars))
                   ,@(nreverse (cdr (reverse body)))
                   (return-from T ,@(last body))))))))

(def-lambda-like-k*expander lambda nil)
(def-lambda-like-k*expander subst nil)
(def-lambda-like-k*expander named-lambda t)
(def-lambda-like-k*expander named-subst t)

;;; This handles all nested applications such as those created by LET.

(defun inline-lambda-k*expander (form)
  (let* ((vars (arglist (car form)))
         (body (cond ((memq (caar form) '(lambda subst)) (cddr (car form)))
                     (t (cdddr (car form)))))
         (vals (cdr form))
         (vars-vals
           (do ((vars-iter vars (cdr vars-iter))
                (vals-iter vals (cdr vals-iter))
                accum)
               ((null vars-iter) accum)
             (push (list (car vars-iter) (car vals-iter)) accum))))
    '(prog T ,vars-vals
           ,@(nreverse (cdr (reverse body)))
           (return-from T ,@(last body)))))
```

```
;the following handle all of the complex fexprs by expanding them
;out as macros.  Note that these expansions do not produce any
;of the K* things.  this is a nice modularity.

(defun (:property or k*expander) (forms)
  '(or-lexpr ., (cdr forms)))
(defun (:property and k*expander) (forms)
  '(and-lexpr ., (cdr forms)))

; we used to understand and and or in a fancy way, but
; we just treat them as lexprs now
;;; Note (or) => nil.
;(defun (:property or k*expander) (forms)
;  (setq forms (cdr forms))
;  (if forms
;      '(cond ,@(mapcar #'ncons (butlast forms))
;             (t ,@(last forms)))))

;;; Note (and) => t.
;(defun (:property and k*expander) (forms)
;  (setq forms (cdr forms))
;  (cond ((null forms) t)
;        (t (and-k*expander1 forms))))

;(defun and-k*expander1 (forms)
;  (cond ((null (cdr forms)) (car forms))
;        (t '(cond (,(car forms) ,(and-k*expander1 (cdr forms)))))))


(defun (:property progn k*expander) (form)
  '(prog T ()
     ,@(nreverse (cdr (reverse (cdr form))))
     (return-from T ,@(last (cdr form)))))

(defun (:property prog1 k*expander) (form)
  (cond ((< (length form) 2)
         (barf "Too few args to PROG1~% Form: ~:=" form)))
  '(prog T (_ret)
     (setq _ret ,(cadr form))
     ,@(cddr form)
     (return-from T _ret)))

(defun (prog2 k*expander) (form)
  (cond ((< (length form) 3)
         (barf "Too few args to PROG2~% Form: ~:=" form)))
  '(prog T (_ret)
     ,(cadr form)
     (setq _ret ,(caddr form))
     ,@(cdddr form)
     (return-from T _ret)))

(defun (:property return k*expander) (form)
  '(return-from nil ,@(cdr form)))

(defun (:property values k*expander) (form)
  '(return-from nil ,@(cdr form)))

(defun (values-list k*expander) (form)
 '(return-list ,@(cdr form)))

;;; only works for short lists at the moment.
(defun (return-list k*expander) (form)
  '(return-from nil
     ,@(do ((expr-iter (cdr form) (cdr expr-iter))
            (fn-iter '(car cadr caddr cadddr) (cdr fn-iter))
            (result nil))
           ((null expr-iter) result)
```

```
        (push '(,(car fn-iter) ,(car expr-iter)) result))))
```

```
(defun (prog k*expander) (list)
  (cond ((and (cadr list) (symbolp (cadr list)))
         '(kbe-prog-named ,@(cdr list)))
        (T '(kbe-prog-named nil ,@(cdr list)))))

(defun (prog* k*expander) (list)
  (cond ((and (cadr list) (symbolp (cadr list)))
         '(let* ,(caddr list)
            (kbe-prog-named ,(cadr list) nil ,@(cdddr list))))
        (T '(let* ,(cadr list)
              (kbe-prog-named nil nil ,@(cddr list))))))

;If you use any funny prog then all bets are off because we have no idea
;what vars are bound.

(defun (progv k*expander) (list)
  (barf "Cannot use PROGV.~% Data: ~S" list))

(defun (progw k*expander) (list)
  (barf "Cannot use PROGW.~% Data: ~S" list))

(defun (let k*expander) (list)
  (let* (vars forms)
    (do ((pairs (cadr list) (cdr pairs))) ((null pairs))
      (cond ((not (listp (car pairs)))
             (push (car pairs) vars) (push nil forms))
            (T (push (caar pairs) vars) (push (cadar pairs) forms))))
    '((lambda ,(nreverse vars) ,@(cddr list)) ,@(nreverse forms))))

(defun (let* k*expander) (list)
  (let* ((pairs (cadr list)) (body (cddr list)))
    (cond ((null (cdr pairs)) '(let ,pairs ,@body))
          (T '(let (,(car pairs)) (let* ,(cdr pairs) ,@body))))))

(defun (let-if k*expander) (list)
  (let* ((pred (cadr list)) vars forms)
    (do ((pairs (caddr list) (cdr pairs))) ((null pairs))
      (cond ((not (listp (car pairs)))
             (push (car pairs) vars) (push nil forms))
            (T (push (caar pairs) vars) (push (cadar pairs) forms))))
    (cond ((null vars) '(progn ,@(cdddr list)))
          (T '(let ,(mapcar #'list vars vars)
                (cond (,pred (psetq ,@(mapcan #'list vars forms))))
                ,@(cdddr list))))))

(defun (multiple-value-bind k*expander) (list)
  (let* ((vars (second list))
         (form (third list))
         (body (rest3 list))
         (var (gensym)))
    '(let ((,var ,form) ,@vars) (multiple-value ,vars ,var) ,@body)))

(defun (multiple-value k*expander) (list)
  (barf "MULTIPLE-VALUE not supported yet.~% Data: ~S" list))
```

```
(defun (do k*expander) (list)
  (cond ((and (cadr list) (symbolp (cadr list)))
         (s-old-style-do-macro nil (cdr list)))
        (T (s-do-macro nil (cdr list)))))

(defun (do-named k*expander) (list)
  (cond ((and (caddr list) (symbolp (caddr list)))
         (s-old-style-do-macro (ncons (cadr list)) (cddr list)))
        (T (s-do-macro (ncons (cadr list)) (cddr list)))))

(defun s-old-style-do-macro (name insides)
  (let* ((var (first insides))
         (init (second insides))
         (repeat (third insides))
         (end-test (fourth insides))
         (body (rest4 insides))
         (tag (gensym)))
    '(let ((,var ,init))
       (prog ,@name nil
         ,tag (cond (,end-test (return nil)))
              ,@body
              (setq ,var ,repeat)
              (go ,tag)))))

(defun s-do-macro (name insides)
  (let* ((triples (first insides))
         (end-list (second insides))
         (body (rest2 insides))
         (end-test (car end-list))
         (exit-forms (cond ((null (cdr end-list)) nil)
                           ((null (cddr end-list)) (cadr end-list))
                           (T '(progn ,@(cdr end-list)))))
         (bindings (mapcar #'(lambda (x)
                               (cond ((not (and (listp x) (cddr x))) x)
                                     (T '(,(car x) ,(cadr x)))))
                           triples))
         (updates (mapcan #'(lambda (x)
                              (cond ((and (listp x) (cddr x))
                                     '(,(car x) ,(caddr x)))))
                          triples))
         (tag (gensym)))
    '(let ,bindings
       (prog ,@name nil
         ,@(cond (end-list (ncons tag)))
              ,@(cond (end-test
                       '((cond (,end-test (return ,exit-forms))))))
              ,@body
              ,@(cond (updates '((psetq ,@updates))))
              ,@(cond (end-list '((go ,tag))))))))
```

```
;;; Mapping functions.

;;; These try and generate safe but good names for the loop variables so
;;; the coder will have a good hint.  We only have a good idea what the
;;; names should be if there is a lambda fn with args to look at.

(defun 6mapvars (fn number-args)
  (cond ((6lambda-like-fn? fn)
         (let* ((arglist (arglist fn)))
           (cond ((= (length arglist) number-args)
                  (mapcar #'(lambda (var)
                              (let ((plural (symbol-pluralize var))
                                    (listed (symbolconc var '-LIST)))
                                (cond ((6good-code plural) plural)
                                      ((6good-code listed) listed)
                                      (t (gensym '_)))))
                          arglist))
                 (t (barf "Wrong number of args to lambda")))))
        (t
         (do ((i 0 (1+ i)) list)
             ((= i number-args) list)
           (push (gensym '_) list)))))

(defun 6savevar (var)
  (let* ((saved (symbolconc var '-SAVED)))
    (cond ((6good-code saved) saved)
          ((6good-code 'saved) 'saved)
          ((6good-code 'old) 'old)
          (t (gensym '_)))))

(defun 6resultvar ()
  (cond ((6good-code 'result) 'result)
        ((6good-code 'list) 'list)
        ((6good-code 'value) 'value)
        (t (gensym '_))))
```

```
;;; This is a macro for defining k*expanders for map, maplist, etc.
;;; return is memq '(2nd-arg list nconc); apply is memq '(cdr car).
(defmacro def-map-k*expander (name return apply)
  '(defun (:property ,name k*expander) (call)
     (let* ((quoted-function (cadr call))
            (function
              (cond ((memq (car quoted-function) '(quote function))
                     (cadr quoted-function))
                    (t (barf
                         "~S can't handle unquoted function application~
                         ~% Data: ~:="
                         (car call) call))))
            (list-exprs (cddr call))
            (mapvars (6mapvars function (length list-exprs)))
            (returnvar ,(cond ((eq return '2nd-arg)
                               '(6savevar (car mapvars)))
                              (t '(6resultvar))))
            (inits (do ((mapvar-iter mapvars (cdr mapvar-iter))
                        (list-exprs-iter list-exprs (cdr list-exprs-iter))
                        inits)
                       ((null mapvar-iter) '(setq ,@inits))
                     (setq inits
                           (append inits
                                   (list (car mapvar-iter)
                                         (car list-exprs-iter))))))
            (end-test (do ((mapvar-iter mapvars (cdr mapvar-iter))
                           ends)
                          ((null mapvar-iter)
                           '(cond ((or ,@ends) (return ,returnvar))))
                        (push '(null ,(car mapvar-iter)) ends)))
            (repeats (do ((mapvar-iter mapvars (cdr mapvar-iter))
                          repeats)
                         ((null mapvar-iter) '(setq ,@repeats))
                       (setq repeats
                             (append repeats
                                     (list (car mapvar-iter)
                                           '(cdr ,(car mapvar-iter))))))))
            (inner-application
              (selectq ',apply
                (car '(,function
                       ,@(do ((mapvar-iter mapvars (cdr mapvar-iter))
                              cars)
                             ((null mapvar-iter) (nreverse cars))
                           (push '(car ,(car mapvar-iter)) cars))))
                (cdr '(,function ,@mapvars))))
            (application
              (selectq ',return
                (2nd-arg inner-application)
                (list '(setq ,returnvar
                             (append ,returnvar
                                     (ncons ,inner-application))))
                (nconc '(setq ,returnvar
                              (nconc ,returnvar
                                     (ncons ,inner-application))))))
            (return-init
              (selectq ',return
                (2nd-arg '((setq ,returnvar ,(car mapvars))))
                ((list nconc) nil))))
       '(prog (,@mapvars ,returnvar)
              ,inits
              ,@return-init
          _l ,end-test
             ,application
             ,repeats
             (go _l)))))

(def-map-k*expander map 2nd-arg cdr)
```

```
(def-map-k*expander mapc 2nd-arg car)
(def-map-k*expander maplist list cdr)
(def-map-k*expander mapcar list car)
(def-map-k*expander mapcon nconc cdr)
(def-map-k*expander mapcan nconc car)
```

```
;this is stuff for expanding cobol programs.

(defun (cobol-fn k*expander) (form)
  '(kbe-prog-named nil ,(Rlist  [(Elist (cadr form)) ['quote 'undefined]])
    .,(cddr form)))

(defun (pif k*expander) (form)
  '(k*if ., (cdr form)))

(defun (stop k*expander) (ignore) '(return))

(defun (stop-run k*expander) (ignore) '(return))

(defun (goback k*expander) (ignore) '(return))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;this operates like an evaluator, making all of the surface plan
;except for data flow.  In leu of data flow, DF numbers are
;associated with each port.  These are later used
;as unambiguous variable names governing the data flow.

;[note it assumes that the piece of code is not a split, this
;fits with the fact that in lisp there is no way to write a naked split.]

;there are several special variables which it uses to keep track of things
;while it works.
;DEFINING-NAME -- the name of the fn currently being defined this is used
;  to detect recursive calls.
;RE-FLOW-EVAL -- this is used as a hack when you are flow-evaluating a
;   recursive fn for the second time, in order to keep your hands
;   on the definition from the prior time.
;INLINE-LIST -- this is used by define-consolidated in order to tell
;   6instantiate-funcall what fns to include inline.
;UNGROUP-LIST -- used when consolidating things to keep track of what to
;   ungroup before returning the result.
;DEFINING-CODE -- this holds the actual code
;  which is the original definition.  it is used to decide what is good to
;  put in a code field, and what is not.
;LASTSEGCASE -- this holds the segcase which immediately precedes (in clfow)
;  the next seg to be evaluated.
;SUPERSEGCASE -- the segcase of the top seg being defined
;ALIST -- this is a normal alist that links each var name with a df-num
;  It grows and shrinks as variables are bound and unbound.
;DF-NUM -- this counts up continuously recording what the largest df-num
;  used so far is.  (note that these numbers are actually bit strings with
;  only one 1 in them that shifts as it counts up).
;DF-NUMS -- a list of all the df-nums created (this is actually bound
;  above in 6flow-analyze, because 6data-flow uses it).
;DF-NAME -- this is an alist, which shows what variable is carrying the
;  dflow indicated by the dfnum.  it is often just a guess, but better
;  than nothing.  it is also used by 6data-flow.
;TAGS -- this is an alist linking each tag with the name of the join
;  corresponding to it. (note that it grows and shrinks like a stack
;  as you enter and leave calls on K*PROG where tags are defined and redefined.)
;  Also note that any join that is only branched to once, and all inaccessable
;  code will be removed from the program in this pass.
;RETVAR -- this is used purely in order to get more mnemonic
;  names for output objects.  it is set by k*psetq 6fexpr in order to
;  get information to 6instantiate-funcall and k*quote 6fexpr.

;must be nil when 6flow-analyze called by itself.
(setq DEFINING-CODE nil DEFINING-NAME nil)

(def-alistq ALIST)
(def-alistq TAGS)
(def-alistq DF-NAME)
```

```
;these fns implement operations with dfnums
;the first generates them

(defun 6df-num ()
  (let* ((new-bits (lsh (car DF-NUM) 1)))
    (push (reverse DF-NUM) DF-NUMS)
    (rplaca DF-NUM new-bits)
    (if (zerop new-bits) (push 1 DF-NUM))
    (top DF-NUMS)))

;this links a port and a dfnum, returning the dfnum

(defun 6+df (port dfn)
  (if (null dfn) (barf "6+DF: Null dfn~% Port: ~:=" port))
  (putprop (~seg~segcase port)
           [[(~obj port) ! dfn] ! (get (~seg~segcase port) 'df)]
           'df)
  (push (cons dfn port) 6DFN-PORT-LIST)
  dfn)

;this gets a port from a dfn

(defun 6dfn->port (dfn)
  (cdr (assoc dfn 6DFN-PORT-LIST)))

;this retrieves a dfnum for a port

(defun 6?df (port)
  (cdr (assq (~obj port) (get (~seg~segcase port) 'df))))

;this records a guess at a variable name.  It returns the df-num

(defun 6df-name (dfn name)
  (setf (DF-NAME dfn) name)
  dfn)

;this is a very typical combination

(defun 6df (port name)
  (6+df port (6df-name (6df-num) name)))



;these are just to help debugging.  the first takes in a bit string,
;and returns a list of the bit positions on in it.

(defun 6d (list)
  (over DF-NUMS -> dfn
   over i <- (1- (length df-nums)) by (1- i) until nil
   select (6set^? dfn list) bag i))

(defun 6dfns ()
  (sortcar
    (over *db-segs* -> s
      nconc (over (get s 'df) -> [obj ! dfn]
              list [['/[ s obj] ! (or (6d dfn) ['none])]]))
    #'(lambda (x y)
       (cond ((eq (car x) (car y)) (alphalessp (cadr x) (cadr y)))
             (t (alphalessp (car x) (car y)))))))
```

```
(defun 6flow-eval (code)
  (let* ((superseg (genseg (or MNEMONIC-NAME 'seg)))
         (SUPERSEGCASE (& superseg))
         (LASTSEGCASE SUPERSEGCASE)
         (DF-NUM [1])
         (ALIST [nil])
         TAGS RETVAR returnval)
    (+& SUPERSEGCASE segtype 'simple)
    (+& SUPERSEGCASE @seg 'opencode nil nil)
    (+& (+& SUPERSEGCASE output nil (genobj superseg 'ret) nil)
        @io 'returnval nil nil)
    (setq returnval (6eval code))
    (+& SUPERSEGCASE cflow LASTSEGCASE SUPERSEGCASE)
    (let ((ret (car (??& (&* SUPERSEGCASE output) @io 'returnval))))
      (if (and returnval ret) (6+df (~deep ret) returnval)))
    (over (??subseg (~seg SUPERSEGCASE)) -> subseg
          (??& SUPERSEGCASE cflow * (&* subseg)) -> cflow-in
      (cond ((null cflow-in)
             (barf "6FLOW-EVAL found inaccessible code.~% Seg: ~S"
                   subseg))
            ((eq (?segtype subseg) 'join)
             (cond ((null (cdr cflow-in))
                    (make-fact-be-out (car cflow-in))
                    (assert-fact (&~from (-& SUPERSEGCASE cflow (&* subseg))
                                         (~from (car cflow-in))))
                    (kill-seg subseg))
                   (t
                    (let ((next
                           (~seg (~to (?& SUPERSEGCASE cflow (&* subseg))))))
                      (if (eq (?segtype next) 'join)
                          (fuse-joins subseg next)))))))))
    SUPERSEGCASE))
```

```
;before we go on, a word about the way free variables are handled.
;there is no easy way to look at the code, and tell what free
;varaibles are used.  (A particular problem is that some fn it calls
;may use a free variable that is never mentioned in the program
;itself).  Therefore, 6eval starts out assuming there aren't any, and
;then whenever it encouters one, it adds a corresponding input to the
;superseg, and makes an entry on the alist for it.  NOTE that no
;distinction is made based on whether the free var is an input or an
;output or both at this time, this is all straightend out in
;6create-super-outputs.  (the NIL that the alist starts out as is there
;so that there will always be some alist to add the new var to.)  the
;fact that a var is free is detected simply by the fact that it is not
;on the alist.  (all this work happens in the fn 6symeval.)

(defun 6symeval (v)
  (or (ALIST v)
      (let ((in (+& SUPERSEGCASE input nil
                    (genobj (~seg SUPERSEGCASE) v) nil)))
        (+& in @io 'free-var v nil)
        (nconc ALIST [[v ! (6df in v)]])
        (ALIST v))))

;this is the part that actually looks like an evaluator.  the
;real guts of it are in the 6fexpr defuns which tell you exactly
;what it really does with each 6fexpr.

(defun 6eval (form)
  (let (fexpr)
    (cond ((self-evalling-form form)
           (funcall (get 'k*quote '6fexpr) '',form))
          ((symbolp form) (6symeval form))
          ((setq fexpr (get (car form) '6fexpr))
           (funcall fexpr form))
          (t (6apply (let (RETVAR) (mapcar #'6eval (cdr form)))
                     (6instantiate-funcall form))))))

;the funny order of args to 6apply makes it simpler to get the cflow right.
;note that nothing at all is done about side-effect outputs here.

(defun 6apply (argvals seg)
  (cond (LASTSEGCASE
         (+& SUPERSEGCASE subseg (& seg))
         (+& SUPERSEGCASE cflow LASTSEGCASE (& seg))
         (setq LASTSEGCASE (& seg))
         (dolist (@io (??& (&* (&* seg) port) @io 'actual-arg))
           (6+df (~deep @io) (nth (1- (~argno @io)) argvals)))
         (dolist (@io (??& (&* (&* seg) port) @io 'free-var))
           (6+df (~deep @io) (6symeval (~name @io))))
         (dolist (@io (??& (&* (&* seg) port) @io 'side-effect))
           (6+df (~deep @io)
                 (6?df
                   (?& (~segcase (~deep @io)) input * (~id (~deep @io))))))
         (let* ((ret (car (??& (&* (&* seg) output) @io 'returnval))))
           (cond (ret
                  (if RETVAR (+& (~deep ret) @createvar RETVAR))
                  (6df (~deep ret) RETVAR)))))
        (t (kill-seg-and-its-subsegs seg)))) ;kill inexcutable code immediately!
```

```
;this creates a segment for a constant datum.  it takes the whole
;quoted expression as an argument.

(defun (k*quote 6fexpr) (item)
  (let* ((segcase (& (genseg
                       (cond ((or (eq (car item) 'cobol-string)
                                  (eq (car item) 'string) ;to be deleted
                                  (symbolp (cadr item))
                                  (numberp (cadr item)))
                              (intern-KBE
                                (format nil "<~a>" (cadr item))))
                             (t 'c)))))
         (obj (genobj (~seg segcase) (or RETVAR 'val))))
    ;; kludge to keep K*QUOTE from getting out of this module.
    ;; There should be a more general way to do this. eg, STRING
    ;; should be K*STRING and we should have to turn it back to STRING.
    ;; Should be data-driven somehow ... --kmp
    (let* ((item-copy (copytree item)))
      (if (eq (car item-copy) 'K*QUOTE) (rplaca item-copy 'QUOTE))
      (+& segcase @seg 'constant item-copy (6good-code item)))
    (+& segcase segtype 'simple)
    (+& segcase plantype 'constant)
    (+& (+& segcase output nil obj nil)
        @io 'returnval nil nil)
    (6apply nil (~seg segcase))))

;this is used by COBOL
(putprop 'cobol-string (get 'k*quote '6fexpr) '6fexpr)
(putprop 'string (get 'k*quote '6fexpr) '6fexpr) ;to be deleted

;note that segments are created for k*psetqs here, and then removed later.
;note that the expansion of (cond (foo (go bar))) leads to a meaningless
;k*psetq that is ignored here.

(defun (k*psetq 6fexpr) (code)
  (6build-k*psetq
    (over [var val ! rest] <- (cdr code) by rest until (null var)
          (let ((RETVAR var)) (6eval val)) -> dfn
      select dfn
      bag [var ! dfn])))

(defun 6build-k*psetq (pairs)
  (if pairs
      (let* ((seg (genseg 'k*psetq))
             (segcase (& seg)))
        (+& segcase @seg 'funcall 'k*psetq nil)
        (+& segcase segtype 'simple)
        (over pairs -> [var ! dfn]
         over i <- 1 by (1+ 1) until nil
              (+& segcase input nil (genobj seg var) nil) -> in
              (+& segcase output 'id (genobj seg var) (~obj in)) -> out
          (6+df in dfn)
          (6+df out (6symeval var)))
        (6apply (mapcar #'cdr pairs) (~seg segcase))
        (cdar (last pairs)))))) ;the returnval

;this is for slot stuff.  Note we take advantage of the
;fact that we always have a segment in order to just
;assert the slot info

(defvar *SUGGESTED-NAME-FROM-SLOT* nil
  "used to pass a suggested name to 6instantiate-funcall")

(defun (:property k*slot 6fexpr) (call)
  (let* (((form slotname slottype) (cdr call))
         (*SUGGESTED-NAME-FROM-SLOT* slotname)
         (args (Rlist (6eval (filterS
                               #'(lambda (x) (not (is-&in-out x)))
```

```
                                (Elist (cdr form))))))
        (seg (6instantiate-funcall form)))
    (+& SUPERSEGCASE slot (& seg) slotname slottype)
    (6apply args seg)))

(defun is-&in-out (x)
  (and (symbolp x)
       (string-equal x "&in-out")))
```

```
;this makes a seg for a funcall on an expr or lexpr.
;In addition, it creates complete annotation on the ports that shows
;what free-vars are used and created.  this is used by later stages in
;order to figure the dflow out.

;this locates the basic prototype description of the
;form's fn.  (the lexpr stuff is an add on cludge, but
;works.  The basic idea is that it fakes up the lexpr
;segment assertions so that they look like expr seg
;assertions for the right number of arguments, then does
;what it does for an expr.)

(defun 6instantiate-funcall (form)
  (let* (((fn . args) form) tmp
         (proto-seg
          (cond ((eq fn nil)
                 (prog1 (6fast-proto *SUGGESTED-NAME-FROM-SLOT* args)
                   (setq args (Rlist (filterS #'(lambda (x)
                                                  (not (is-&in-out x)))
                                              (Elist args))))))
                ((and (boundp 'INLINE-LIST) (memq fn INLINE-LIST))
                 (copyseg (?lambda-exp fn) fn))
                ((and (eq fn DEFINING-NAME) RE-FLOW-EVAL)
                 (6copy-top-seg RE-FLOW-EVAL fn))
                ((setq tmp (?lambda-exp fn))
                 (6copy-top-seg tmp fn))
                ((setq tmp (get fn '6proto-seg))
                 (funcall tmp))
                (t
                 (when (not (or (fboundp fn) (memq fn SIMPLE-EXPRS)))
                   (warn "~S isn't a known function." fn))
                 (6fast-proto fn args)))))
    (--& (&* proto-seg) @seg)
    (--& (&* proto-seg) plantype)
    (cond ((and DEFINING-NAME (eq fn DEFINING-NAME))
           (+& SUPERSEGCASE recursive (& proto-seg))
           (+& (& proto-seg) @seg 'recursion nil nil))
          ((not (and (boundp 'INLINE-LIST) (memq fn INLINE-LIST)))
           (+& (& proto-seg) plantype 'new-view)
           (+& (& proto-seg) @seg 'funcall fn (6good-code form))))
    (over (??& (&* (&* proto-seg) input) @io 'lambda-arg) -> lexpr@io
      select (minusp (~argno lexpr@io))
      (make-fact-be-out lexpr@io)
      (make-fact-be-out (~deep.lexpr@io))
      (over i <- (- (~argno lexpr@io)) by (1+ i) until nil
       over (nthcdr (1- (- (~argno lexpr@io))) args)
       (+& (+& (& proto-seg) input nil
               (genobj proto-seg (~obj (~deep lexpr@io))) nil)
           @io 'lambda-arg nil i)))
    (over (??& (&* (&* proto-seg) output) @io 'lambda-arg) -> lexpr@io
      select (minusp (~argno lexpr@io))
      (make-fact-be-out lexpr@io)
      (make-fact-be-out (~deep lexpr@io))
      (over i <- (- (~argno lexpr@io)) by (1+ i) until nil
       over (nthcdr (1- (- (~argno lexpr@io))) args)
       (+& (+& (& proto-seg) output nil
               (genobj proto-seg (~obj (~deep lexpr@io))) nil)
           @io 'lambda-arg nil i)))
    (over (--& (&* (&* proto-seg) port) @io 'lambda-arg) -> @io
          (nth (1- (~argno @io)) args) -> arg
          (and (symbolp arg) (not (eq arg T)) arg) -> var-arg
      (+& (~deep @io) @io 'actual-arg var-arg (~argno @io)))
    proto-seg))
```

```
;this copyies the deffacts from a seg to get the info needed for
;a funcall new-view.  won't work for joins or TC's

(defun 6copy-top-seg (seg root)
  (let* ((newseg (genseg root)) changes)
    (over (??& (&* seg) input) -> in
          (+& (& newseg (~case (~segcase in))) input nil
              (genobj newseg (~obj in)) nil) -> new
      (push (cons (~obj in) (~obj new)) changes)
      (over (??& in @io)
        (+& new @io (~kind _) (~name _) (~argno _))))
    (over (??& (&* seg) output) -> out
          (+& (& newseg (~case (~segcase out))) output (~kind out)
              (genobj newseg (~obj out))
              (cdr (assq changes (~id out)))) -> new
      (push (cons (~obj out) (~obj new)) changes)
      (over (??& out @io)
        (+& new @io (~kind _) (~name _) (~argno _))))
    (over (??& (&* seg) segtype)
      (+& (& newseg (~case (~segcase _))) segtype (~type _)))
    (over (??& (&* seg) assert)
      (+& (& newseg (~case (~segcase _))) assert
          (sublis changes (~wff _))))
    (over (??& (&* seg) when)
      (+& (& newseg (~case (~segcase _))) when
          (sublis changes (~wff _))))
    newseg))

(defvar 6fast-proto-names)

(defun 6fast-proto (fn arg-list)
  (let* ((args arg-list)
         (ins)
         (outs (ncons (or RETVAR 'z)))
         (6fast-proto-names outs))
    (prog (arg var is-in-out)
      L (cond ((null args) (return nil)))
        (setq arg (pop args)
              is-in-out nil)
        (cond ((is-&in-out arg)
               (setq is-in-out T
                     arg (pop args))))
        (setq var (6fast-proto-name arg arg-list))
        (push var ins)
        (if is-in-out
            (push [(6fast-proto-name nil arg-list) 'side-effect var] outs))
        (go L))
    (6fast-defspecs '(,fn ,(nreverse outs) .,(nreverse ins)))))

(defun 6fast-proto-name (arg args)
  (cond ((and arg (symbolp arg) (not (eq arg t))
              (not (memq arg 6fast-proto-names)))
         (push arg 6fast-proto-names)
         arg)
        (T (lets ((let (Elist '(a b c d e f g h i j k l m n o p q r s t))))
             (cond ((and (not (memq let 6fast-proto-names))
                         (not (memq let args)))
                    (push let 6fast-proto-names)
                    (done let)))))))
```

```
;this makes it fast to cons up a dummy definition for something.  it
;is mainly used by the basic data base set up fns defined at the end
;of this file.  it basically just takes in a fn name, and a list of
;outputs, and inputs.  however, there are two special characteristics
;that an output can have.  this is shown by passing a list instead of
;an atom for that output.  the options are:
; 1. (z PART in) showing that the output is a part of the input
; 2. (z SIDE-EFFECT in) showing that the output comes from
;    side-effecting the input [note if there is more than one output
;    then all but one must be side-effect ones].
;In addition to the above either (but not both) the list of inputs or
;outputs can have an atomic cdr.  This is converted into a
;specification for lexpr style optional args.  These cannot have either
;of the extra options attached to them
; note this doesn't use genobj, so you got to get your obj names right.

(defun 6fast-defspecs (specs)
  (let* (((name outs . ins) specs)
         (seg (genseg name))
         (num 0) intrn)
    (+& (& seg) segtype 'simple)
    (+& (& seg) plantype 'builtin)
    (+& (& seg) @seg 'lambda-exp name nil)
    (over list <- ins until nil
      (setq num (1+ num))
      (cond ((null list) (return))
            ((atom list)
             (let ((in (genobj seg list)))
               (push (cons list in) intrn)
               (+& (+& (& seg) input nil in nil)
                   @io 'lambda-arg list (- num)))
             (return)))
      (let ((in (genobj seg (car list))))
        (push (cons (car list) in) intrn)
      (+& (+& (& seg) input nil in nil)
          @io 'lambda-arg (car list) num)))
    (over list <- outs
      (cond ((null list) (return))
            ((atom list)
             (+& (+& (& seg) output nil (genobj seg list) nil)
                 @io 'lambda-arg list (- num))
             (return)))
      (cond ((atom (car list))
             (+& (+& (& seg) output nil (genobj seg (car list)) nil)
                 @io 'returnval nil nil))
            ((eq (cadr (car list)) 'part)
             (+& (+& (& seg) output 'part (genobj seg (caar list))
                     (cdr (assq (caddar list) intrn)))
                 @io 'returnval nil nil))
            ((eq (cadr (car list)) 'side-effect)
             (+& (+& (& seg) output 'id (genobj seg (caar list))
                     (cdr (assq (caddar list) intrn)))
                 @io 'side-effect nil nil))))
    seg))
```

```
;these are the descriptions of the basic LISP functions
;the flow analyzer knows about lisp system fns by asking (FBOUNDP
;SYSTEM name) this list has all of the other fns which it treats as
;just bing simple fns (eg expr or lexpr with a return value and no
;side-effects, no free-var ios, nothing special to say at all).  (this
;is to make this common case very simple and space effecient.)  The
;funny fns E0, E1, ... serve as the assertion templates for all of
;these simple fns.

(setq SIMPLE-EXPRS
      '(make-array maknum *dummy-expr*
    ;for cobol you would need these too.
;       eofp invalid-keyp alphanumericp numericp alphabeticp concatenate
;       current-date time-of-day day date
    ;for fortran you would need these too.
;       _+i _-i _*i _//i _**i _u-i _+r _-r _*r _//r _**r _u-r _**ri
;       _+d _-d _*d _//d _**d _u-d _**di _itor _itod _rtod _dtor _rtoi
;       _dtoi _exp _dexp _alog _dlog _sin _dsin _cos _dcos _sqrt _dsqrt
;       _iabs _abs _dabs _float _dfloat _sngl _dble _make-array
       ))

;these define all of the funny prototypes that we need because they
;have special junk in them.

(defun (k*setf 6proto-seg) ()
  (6fast-defspecs '(k*setf ((out side-effect in))
                           access-list in value)))

(defun (5noop 6proto-seg) ()
  (6fast-defspecs '(5noop ((out part in)) in)))
(defun (car 6proto-seg) ()
  (6fast-defspecs '(car ((obj part list)) list)))
(defun (cdr 6proto-seg) ()
  (6fast-defspecs '(cdr ((obj part list)) list)))
(defun (cddr 6proto-seg) ()
  (6fast-defspecs '(cddr ((obj part list)) list)))
(defun (caar 6proto-seg) ()
  (6fast-defspecs '(caar ((obj part list)) list)))
(defun (cadr 6proto-seg) ()
  (6fast-defspecs '(cadr ((obj part list)) list)))
(defun (cdar 6proto-seg) ()
  (6fast-defspecs '(cdar ((obj part list)) list)))
(defun (assoc 6proto-seg) ()
  (6fast-defspecs '(assoc ((res part list)) obj list)))
(defun (rplacd 6proto-seg) ()
  (6fast-defspecs '(rplacd (z (cell side-effect cons)) cons obj)))
(defun (rplaca 6proto-seg) ()
  (6fast-defspecs '(rplaca (z (cell side-effect cons)) cons obj)))
(defun (arraystore 6proto-seg) ()
  (6fast-defspecs '(arraystore (z (arrayout side-effect array))
```

```
                                array index item)))
(defun (format 6proto-seg) ()
  (6fast-defspecs '(format (z (file-out side-effect file))
                           file string . arg)))

(defun (arrayfetch 6proto-seg) ()
  (6fast-defspecs '(arrayfetch ((item part array)) array index)))
(defun (aset 6proto-seg) ()
  (6fast-defspecs '(aset (z (arrayout side-effect array))
                         obj array . idx)))
(defun (aref 6proto-seg) ()
  (6fast-defspecs '(aref ((item part array)) array . idx)))
(defun (non-null 6proto-seg) ()
  (~seg
    (plan-def non-null nil
     (deffacts non-null
       (plantype (builtin)) (segtype (split)) (@seg (lambda-exp non-null))
       (input (nil obj nil))
       (@io (obj lambda-arg obj 1))
      non-null (when (obj))       ;note interaction with 5get-lisp-pred
      null     (when ((not obj))))))))
(defun (k*infinite-loop-pseudo-exit 6proto-seg) ()
  (~seg
    (plan-def k*infinite-loop-pseudo-exit nil
     (deffacts k*infinite-loop-pseudo-exit
       (plantype (builtin)) (segtype (split))
       (@seg (lambda-exp k*infinite-loop-pseudo-exit))
      never    (when (T))
      always   (when ((not T)))))))
```

```
;The following similar stuff would be needed for FORTRAN.
;note that some of the LISP functions are needed for FORTRAN
;e.g. arraystore and arrayfetch
(comment

(defun (_lt 6proto-seg) ()
  (~seg
    (plan-def lt nil
     (deffacts _lt
        (plantype (builtin))
        (segtype (split))
        (@seg (lambda-exp _lt))
        (input (nil obj nil))
        (@io (obj lambda-arg obj 1))
       yes (when ((minusp obj)))
       no  (when ((not (minusp obj))))))))
(defun (_eq 6proto-seg) ()
 (~seg
    (plan-def eq nil
     (deffacts _eq
        (plantype (builtin))
        (segtype (split))
        (@seg (lambda-exp _eq))
        (input (nil obj nil))
        (@io (obj lambda-arg obj 1))
       yes (when ((zerop obj)))
       no  (when ((not (zerop obj))))))))
(defun (_gt 6proto-seg) ()
  (~seg
    (plan-def gt nil
     (deffacts _gt
        (plantype (builtin))
        (segtype (split))
        (@seg (lambda-exp _gt))
        (input (nil obj nil))
        (@io (obj lambda-arg obj 1))
       yes (when ((plusp obj)))
       no  (when ((not (plusp obj))))))))
 ) ;end comment

;this is the similar basic definitions corresponding to the stuff that
;comes out of Glenn Burks cobol parser.  NOTE THAT ALL OF HIS STUFF
;WHICH CORRESPONDS TO ACTUAL LISP FNS IS DEFINED ALONG WITH THE LISP STUFF.
;AND THEREFORE THE LISP STUFF MUST BE THERE TOO.

(defun (substring 6proto-seg) ()
 (6fast-defspecs '(substring ((z part x)) x i j)))
(defun (Cread 6proto-seg) ()
  (6fast-defspecs '(Cread ((side side-effect file) . item) file)))
(defun (Cwrite 6proto-seg) ()
  (6fast-defspecs '(Cwrite ((outfile side-effect infile)) infile . obj)))
(defun (Crewrite 6proto-seg) ()
  (6fast-defspecs '(Crewrite ((outfile side-effect infile)) infile . obj)))
(defun (Copeni 6proto-seg) ()
  (6fast-defspecs '(Copeni ((file side-effect infile)) infile . flag)))
(defun (Copeno 6proto-seg) ()
  (6fast-defspecs '(Copeno ((file side-effect infile)) infile . flag)))
(defun (Copenio 6proto-seg) ()
  (6fast-defspecs '(Copenio ((file side-effect infile)) infile)))
(defun (Cclose 6proto-seg) ()
  (6fast-defspecs '(Cclose ((outfile side-effect infile)) infile . flag)))
(defun (skip 6proto-seg) ()
  (6fast-defspecs '(skip to-channel ((outfile side-effect infile))
                         num infile)))
(defun (nterpri 6proto-seg) ()
  (6fast-defspecs '(nterpri ((outfile side-effect infile)) infile num)))
```

```
;the only funny thing that this does is that it over-rides
;assertions allready made by 6flow-eval in order to cause
;the seg being created to be a lambda-exp Note that it is
;not expected that a k*lambda will ever by anything but
;the outer most form of something.

(defun (k*lambda 6fexpr) (form)
 (let* (((return? args body) (cdr form)))
  (-& SUPERSEGCASE @seg)
  (+& SUPERSEGCASE @seg 'lambda-exp
      DEFINING-NAME (6good-code DEFINING-CODE))
  (if (null return?)  ;eg for fortran and cobol
      (let ((ret-out (~deep (-& (&* SUPERSEGCASE output) @io 'returnval))))
        (make-fact-be-out ret-out)))
  (over args -> arg
   over i <- 1 by (1+ i)
        (+& SUPERSEGCASE input nil
            (genobj (~seg SUPERSEGCASE) arg) nil) -> in
    (+& in @io 'lambda-arg arg i)
    (push [arg ! (6df in arg)] ALIST))
  (6eval body)))

;this does the key work involving bound variables, and sequencing of
;statements.  Note that the binding to new values creates a
;k*psetq segment.

(defun (k*prog 6fexpr) (form)
  (let (((inits . body) (cdr form)))
    (let ((ALIST ALIST)
          (TAGS TAGS) ;bound so can pop out again.
          (pairs (over inits -> [var expr]
                       bag [var ! (let ((RETVAR var)) (6eval expr))])))
      (over pairs -> [var ! _]
            (push [var ! (6df-name (6df-num) var)] ALIST))
      (6build-k*psetq pairs)
      (over body -> line
            select (and (not (atom line)) (eq (car line) 'k*tag))
            (let* ((joincase (& (genseg 'join))))
              (+& joincase segtype 'join)
              (+& joincase plantype 'join)
              (+& joincase @seg 'join (cadr line) nil)
              (+& SUPERSEGCASE subseg joincase)
              (push [(cadr line) ! joincase] TAGS)))
      (over b <- body -> sexpr
            (cond ((not (null (cdr b)))
                   (let (RETVAR) (6eval sexpr)))
                  ((cdr sexpr)
                   (barf "Multiple returns not supported yet."))
                  (sexpr
                   (return (6eval (car sexpr))))        ;one return val
                  (t
                   (return nil)))))))
```

```
(defun (k*go 6fexpr) (form)
  (if LASTSEGCASE (+& SUPERSEGCASE cflow LASTSEGCASE
                      (& (~seg (TAGS (cadr form)))
                         (gencase (~seg (TAGS (cadr form)))))))
  (setq LASTSEGCASE nil) ;signal default inaccesability of following code
  nil) ;no output

(defun (k*tag 6fexpr) (form)
 (let* ((lab (cadr form)))
   (if LASTSEGCASE (+& SUPERSEGCASE cflow LASTSEGCASE
                       (& (~seg (TAGS lab)) (gencase (~seg (TAGS lab))))))
   (setq LASTSEGCASE (tags lab))
   nil)) ;no output

(defun (k*ifnull 6fexpr) (form)
  (predicate-6fexpr 'non-null [(cadr form)] 'null 'non-null (caddr form)))

(defun (k*if 6fexpr) (form)
  (predicate-6fexpr 'non-null [(cadr form)] 'non-null 'null (caddr form)))

(defun (k*infinite-loop-pseudo-exit 6fexpr) (form)
  (predicate-6fexpr (car form) nil 'never 'always (cadr form)))

(defun predicate-6fexpr (pred exprs branch-case no-branch-case lab)
  (let* ((RETVAR nil)
         (vals (mapcar #'6eval exprs))
         (seg (6instantiate-funcall [pred ! exprs])))
    (6apply vals seg)
    (+& SUPERSEGCASE cflow (& seg branch-case)
        (& (~seg (TAGS lab)) (gencase (~seg (TAGS lab)))))
    (setq LASTSEGCASE (& seg no-branch-case)))
  nil) ;no output

(defun (_lt 6fexpr) (form)  ;used by fortran
  (predicate-6fexpr (car form) [(cadr form)] 'yes 'no (caddr form)))
(defun (_eq 6fexpr) (form)  ;used by fortran
  (predicate-6fexpr (car form) [(cadr form)] 'yes 'no (caddr form)))
(defun (_gt 6fexpr) (form)  ;used by fortran
  (predicate-6fexpr (car form) [(cadr form)] 'yes 'no (caddr form)))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;this set of fns figures the data flow out starting from the df-nums
;created above.  It does a lot of basic graph algorithmic stuff.  It
;looks at the surface plan created so far as a graph of straight line
;sections.  Each straight line section begins at the superseg, a join,
;or a split, and ends on the superseg, a join, or a split.  Note
;however, that this segcase is not actually in the segment, but rather
;just indicates the source of the control flow which points to the
;actual first segment.  (for the superseg, things are a little
;different.  The outputs of the superseg are treated as data-flow
;sinks at the end of the last section.  The inputs of the superseg are
;treated as dflow sources at the start of the first section.  The
;computation of written, live, part and join treats the inputs of the
;superseg as also being sinks at the end, so that side-effects to the
;inputs will be detected.  The actual consruction of dflow may create
;additional outputs for them.)  There is an information structure (a
;big hunk) associated with each section which contains:

;START - The segcase of the place the section begins after.  The variable
;   FIRST-HUNK points to the hunk that starts on the supersegment.
;END - The segcase that is the last segcase in control flow
;   that is in the section.  Note that this is the segcase that is the
;   ~TO of the cflow going to this seg.  The variable LAST-HUNK points to
;   the hunk that ends on the supersegment.
;FLAG - a flag which is used to accelerate transitive closure computations.
;PREV - a list of all of the hunks that immediately preceed this one.
;NEXT - a list of all of the hunks that immediately follow this one.

;WRITTEN - a set of the dfnums written in this section.
;LIVE - a set of the dfnums that are LIVE after the END of this section (a bit
;   string) (ie read before being written on some path from there to the
;   end of the superseg)

;PART - alist that shows for every live dfnum, a bit set of the other
;   dfnums it is part of. Note that this is kept completely transatively
;   closed at all times.  It is current before the START of the section.

;JOIN - a list of every dfnum that must be joined here.  this is only
;   created for sections that have more than one predecessor.
;SOURCE - is used in the computation of join.  it is an alist, which holds for
;   each dfnum of interest, the hunk where it is written nearest to
;   this spot.
;UNKNOWN - this is allso used in the computation of join, it is a list of all
;   of the dfnums where it is still unkown whether or not they should
;   be joined.
```

```
;note that on the lispm this is a named structure.

(defstruct (dflow-hunk (:type :named-list))
  start end flag prev next written live part join source unknown)

(defmacro 6make-dflow-hunk (start)
  '(make-dflow-hunk start ,start end nil flag nil prev nil next nil
                    written nil live nil part nil join nil
                    source nil unknown nil))

(pp:define-list-print-function dflow-hunk (hunk s)
  (format s "dfflowH-~=" (start hunk)))

;this is just for debugging, it prints out the hunks intellagibly

(defun 6h (hunk) ;hunk or list of hunks
  (cond ((listp hunk) (pp:pp (over hunk bag (6h _))))
        (t [[(start hunk) (end hunk) (flag hunk)]
            [(prev hunk) (next hunk)]
            [(6d (written hunk)) (6d (live hunk)) (6a (part hunk))]
            [(over (join hunk) bag (car (6d _)))
             (over (source hunk) bag [(car (6d (car _))) ! (cdr _)])
             (over (unknown hunk) bag (car (6d (car _))))]])))

(defun 6a (alist) ;dfnum -> dfset
  (over alist bag [(car (6d (car _))) ! (6d (cdr _))]))
```

```
;fields start&end,prev&next are abstracted from the surface plan
;coming from the cflow in the surface plan coming from flow-eval.
;In addition, the special variable FLOWS-HELPS-ANALYZE is set to a
;list of the superseg, and every seg that is the start in a
;section.  This is used by analyze as a hint for where to look in
;order to group up a lot of exprs before it does more complex
;groupings.

;the field written is abstracted from the DF properties of ports in
;the surface plan. Live is computed from next, written, and a procedure
;FIND-LIVE that computes what is live at the begining of a section
;given what is live at the end of the section by
;finding a fixed point of the following set of equations:
; for all sections i, i.live = U over all j in i.next of (FIND-LIVE j j.live)

;field part is then computed by looking at the part and id assertions on
;ports in order to be able to add additional dflow for side-effects.
;The model behind this is rather weak, but it does work in a lot of
;useful cases.  the whole thing runs off of part and id assertions in
;ports.  Its model is that if you side-effect A and A is part of B
;then you have side-effected B as well.  There are two basic problems
;with this.  first, the assertions on the segments only make it
;possible to determine some of the part relationships, so things get
;missed.  Second, there are other ways that side-effects can
;propogate, for example through some computational dependence amoung
;array indexing.  (Note it is also assuming there is never any
;aliasing between arguments.)

;once part has been computed, a pass is made over the program adding additional
;outputs to side-effect segments.  The basic action is that if A is
;side-effected by a segment, and A is part of B, and B is live at that
;moment, then B is made an additional input and output of the segment.
;NOTE that field written is updated at this time becuase it is used in
;the computation of join.  red is not updated because it is not used later.

;field join is the most time consuming thing to compute of all.
;first a set of candidates for possible joining is computed based
;on the fact that nothing needs to be joined unless it is both,
;live at the join, and written in at least two different sections of code.
;then these are further examend, and joined unless they come from the same
;source on each approach to the join.  this is described in much more detail
;below.

;after join is computed, the required ports are added to the joins

;then the actual data flow is constructed.  This includes adding side-effect
;outputs corresponding to inputs to the superseg if necessary.
```

```
;this is the main fn that calls the rest one after another.

(defun 6data-flow (SUPERSEGCASE)
  (let (INFO LAST-HUNK FIRST-HUNK)
    (6construct-df-info SUPERSEGCASE nil)
    (setq FLOWS-HELPS-ANALYZE
          [(~seg SUPERSEGCASE) ! (over INFO bag (start _))])
    (6find-write-estimate-live)
    (6trans-live)
    (6find-parts)
    (6side-effects)
    (6find-joining)
    (over INFO (6do-joining _))
    (6create-dflow)))

;this takes a starting point, and a prior-hunk, and returns the hunk for
;the indicated section.  as a side-effect, it sets up previous and next
;pointers in the hunks.

(defun 6construct-df-info (segcase prior-hunk)
  (let* ((hunk (over INFO (if (equal segcase (start _)) (return _)))))
    (when (not hunk)
      (setq hunk (6make-dflow-hunk segcase))
      (push hunk INFO)
      (if (equal segcase SUPERSEGCASE) (setq FIRST-HUNK hunk))
      (repeat ()
        (setq segcase (~to (?& SUPERSEGCASE cflow segcase)))
        (when (equal segcase SUPERSEGCASE)
          (setq LAST-HUNK hunk)
          (return))
        (caseq (?segtype (~seg segcase))
          (join (6construct-df-info (& (~seg segcase)) hunk) (return))
          (split (over (??& SUPERSEGCASE cflow (&* (~seg segcase)))
                       (6construct-df-info (~from _) hunk)) (return))))
      (setf (end hunk) segcase))
    (cond (prior-hunk
           (push prior-hunk (prev hunk))
           (push hunk (next prior-hunk))))))
```

```
;these little fns implement operations on sets of bits in a bit string
;(eg sets of dfnums) they are used here because they are very compact
;and fast.  Each bit string is a list of fixnums, with flag6 bits per
;fixnum.  note that it is not assumed that two bit strings will be the
;same length (zeros are used to extend them on the right).  The dfnums
;are themselves sets of this type with just one element in them.

;this is used to tell if a dfnum is in a set (or more generally if there
;is any over lap between two sets).

(defun 6set^? (x y)
  (over x -> xe
   over y -> ye
   fastor (not (zerop (logand xe ye)))))

;this forms the intersection of two sets, possibly destroying the FIRST set in
;the process.  The answer is returned (note it is not guarrenteed to be
;eq to the first set).  Is is guarrenteed that there will be no sharing
;between the second set and the answer.

(defun 6set^ (x y)
  (cond ((or (null x) (null y)) nil)
        (t (repeat ((s1 x) (send y))
             (setf (car s1) (logand (car s1) (car send)))
             (cond ((or (null (cdr send)) (null (cdr s1)))
                    (setf (cdr s1) nil)
                    (return x)))
             (pop s1)
             (pop send)))))

;this forms the union of two sets, possibly destroying the FIRST set in
;the process.  The answer is returned (note it is not guarrenteed to be
;eq to the first set).  Is is guarrenteed that there will be no sharing
;between the second set and the answer.

(defun 6setU (x y)
  (cond ((null x) (copylist y))
        ((null y) x)
        (t (repeat ((s1 x) (send y))
             (setf (car s1) (logior (car s1) (car send)))
             (cond ((null (cdr send)) (return))
                   ((null (cdr s1))
                    (setf (cdr s1) (copylist (cdr send)))
                    (return)))
             (pop s1)
             (pop send))
           x)))

;this removes every element in the second set from the first set,
;as with 6setU it usually side-effects the first set.

(defun 6set- (x y)
  (cond ((or (null x) (null y)) x)
        (t (repeat ((s1 x) (send y))
             (if (or (null s1) (null send))
                 (return x))
             (setf (car s1) (boole 4 (car s1) (car send)))
             (pop s1)
             (pop send)))))

;these are dfns so that you can use these in OVER easy.

(eval-when (eval load compile)
  (defprop 6setU (6setU over-acc over-body) u1overstep)
  (defprop 6setU over-acc u1overreturn)
  (defprop 6setU nil u1overinit)
  (defprop qsetU (qsetU over-acc over-body) u1overstep)
  (defprop qsetU over-acc u1overreturn)
```

```
(defprop qsetU nil u1overinit))
```

```
;this takes in a live set that is assumed to be accurate just after
;tsegcase, and moves back to just after fsegcase updating (without any
;side-effects) the live set.  (Note there must be no branching over this
;span.)  the set of the variables written is also returned (this takes
;no extra computation).

;note that the inputs to the superseg are treated as if they needed to be
;supplied at the end of the last section.  (This is vital so that if these
;inputs are side-effected, the program can detect the fact, and make additional
;outputs corresponding to them!)  They are also treated as sources at the
;beginning of the first section.  Also note that the inputs to a setq
;are only live if the corresponding outputs are.

(defun 6find-live1 (fsegcase tsegcase live)
  (setq live (copylist live))
  (repeat ((segcase tsegcase) this-w this-r written)
    (setq this-w (over (??& (&* (~seg segcase)) output)
                       6setU (6?df _))
          this-r (cond ((??& segcase @seg 'funcall 'k*psetq)
                        (over (??& segcase input) -> in
                              (?& segcase output 'id * (~obj in)) -> out
                         select (6set^? (6?df out) live)
                         6setU (6?df in)))
                       (t
                        (over (??& (&* (~seg segcase)) input)
                         6setU (6?df _)))))
    (cond ((equal segcase SUPERSEGCASE)
           (setq live (6setU live this-w)))
          (t
           (setq live (6set- live this-w))
           (setq written (6setU written this-w))))
    (setq live (6setU live this-r))
    (setq segcase (~from (?& SUPERSEGCASE cflow * segcase)))
    (when (equal segcase fsegcase)
      (when (equal segcase SUPERSEGCASE)
        (setq this-w (over (??& (&* (~seg segcase)) input)
                           6setU (6?df _)))
        (setq live (6set- live this-w))
        (setq written (6setU written this-w)))
      (return [live ! written]))))
```

```
;this determines what is written, and makes a good fast initial estimate of
;what is live after each section.  note that it carfully looks at the sections
;in an advatagious order (bottom up breadth first).  note this expects all of
;the flags to start as NIL and leaves them all as T.

(defun 6find-write-estimate-live ()
  (repeat ((todo [LAST-HUNK]) hunk live written)
    (setq hunk (when-pop-fails todo (return)))
    (when (null (flag hunk))
      (setf (flag hunk) t)
      (desetq (live . written)
              (6find-live1 (start hunk) (end hunk) (live hunk)))
      (setf (written hunk) written)
      (over (prev hunk)
            (setf (live _) (6setU (live _) live)))
      (setq todo (append todo (prev hunk))))))

;this computes what is live based on what is read in each section note
;that things are accelerated two ways, first things are done in an
;advantagious order to make information propogate fast, ie bottom up
;breadth first, and second, flags are used to reduce redundant
;computations. it assumes that all of the flags start as T, and leaves them
;all set to nil.

(defun 6trans-live ()
  (repeat ((to-do INFO) hunk live)
    (setq hunk (when-pop-fails to-do (return)))
    (when (flag hunk)
      (setf (flag hunk) nil)
      (setq live (over (next hunk)
                       6setU (car (6find-live1 (start _) (end _) (live _)))))
      (when (not (equal live (live hunk)))
        (setf (live hunk) live)
        (over (prev hunk)
              (push _ to-do)
              (setf (flag _) t))))))
```

```
;this finds out what is part of what, and constructs field part.  instead
;of really doing a full computation of part by relaxation, in anology to
;the other computations, it currently estimates part in a single pass
;over the program.  the format of field part is an alist, with dfnums
;keying bit string sets of all the other dfnums it is part of.  In
;order to make things a bit faster, each dfnum is always considered to
;be part of itself if it is part of anything. (this saves a lot of
;Unions.)  Note that this is maintained completely transitively closed
;at all times.  At a join, a dfnum is considered part of another dfnum
;if it is part of it on any of the entering paths. (the program uses
;what is live at one of the preceeding sections as a good estimate of
;what is live at the start of this section. in can only be wrong in
;that less is actually live.)  (it assumes that there are no part
;relationships between inputs to start with and so just ignores the
;inputs to the superseg al together.)  it assumes that all of the
;flags start as NIL and restores them to that at the end.

(def-alistq parts)

(defun 6find-parts ()
  (repeat ((to-do [FIRST-HUNK]))
    ;find section with all predecessors done if any.
    (repeat (waiting)
      (cond ((null to-do)
             (setq to-do (nconc to-do waiting))
             (return))
            ((flag (top to-do))
             (pop to-do))
            ((over (prev (top to-do)) fastand (flag _))
             (setq to-do (nconc to-do waiting))
             (return))
            (t
             (push (pop to-do) waiting))))
    (if (null to-do) (return))
    (let* ((hunk (pop to-do))
           (live (if (prev hunk) (live (car (prev hunk)))))
           (the-parts (over (prev hunk)
                            bag (6find-parts1 (start _) (end _) (part _)))))
      (setf (part hunk) (over DF-NUMS -> dfn
                              (if (6set^? dfn live)
                                  (over the-parts -> parts
                                   6setU (parts dfn))) -> new-part
                        select new-part
                        bag [dfn ! new-part]))
      (setf (flag hunk) t)
      (setq to-do (append (next hunk) to-do))))
  (over INFO (setf (flag _) nil)))

;this takes in a parts specification which is assumed to be accurate
;for the point just after fsegcase, and moves forward to and including
;tsegcase, updating (without side-effecting) the parts information.  (there
;must be no branching between these two points.)

(defun 6find-parts1 (fsegcase tsegcase parts)
  (setq parts (over parts revlist (append _ nil))) ;note must keep dfns eq
  (repeat ((segcase fsegcase) obj-dfn id-dfn)
    (setq segcase (~to (?& SUPERSEGCASE cflow segcase)))
    (dolist (out (??& (&* (~seg segcase)) output))
      (when (memq (~kind out) '(id part))
        (setq obj-dfn (6?df out)
              id-dfn  (6?df (?& (&* (~seg segcase)) input * (~id out))))
        (setf (parts obj-dfn)
              (6setU (or (parts obj-dfn) (append obj-dfn nil))
                     (or (parts id-dfn) (append id-dfn nil))))
        (if (eq (~kind out) 'id)
            (setf (parts id-dfn) (parts obj-dfn)))))
    (if (equal segcase tsegcase) (return parts))))
```

```
;this actually adds in the extra ports called for by the side-effects.
;unfortunately, I cannot think of any way to get mnemonic object names here.

(defun 6side-effects ()
 (let ((subsegs (??subseg (~seg SUPERSEGCASE))))
  (over subsegs -> subseg
        (??& (&* (&* subseg) output) @io 'side-effect) -> se@ios
    (if se@ios
        (let* ((hunk
                (repeat ((segcase (& subseg)))
                   (if (or (equal segcase SUPERSEGCASE)
                           (not (eq (?segtype (~seg segcase)) 'simple)))
                       (return (over INFO (if (equal segcase (start _))
                                              (return _)))))
                   (setq segcase
                         (~from (?& SUPERSEGCASE cflow * segcase)))))
               (live (car (6find-live1 (& subseg) (end hunk) (live hunk))))
               (parts (6find-parts1 (start hunk) (& subseg) (part hunk))))
          (over se@ios -> se@io
                (over (??& (&* subseg) output)
                  6setU (6?df _)) -> out-dfnums
                (parts (6?df (~deep se@io))) -> part-dfnums
            (over DF-NUMS -> dfnum
              select (and (6set^? dfnum part-dfnums)
                          (6set^? dfnum live)
                          (not (6set^? dfnum out-dfnums)))
              (cond ((and (??& * recursive (&* subseg))
                          (not (6set^? (6?df (~deep se@io)) live)))
                     ;;this is a kludge so recursive side-effecting things work
                     (6+df (~deep se@io) dfnum))
                    (t
                     (let* ((name (or (DF-NAME dfnum)
                                      (barf "6SIDE-EFFECT: ~
                                                 Side-effecting a non-~
                                                 variable.~
                                             ~% Subseg: ~S~
                                             ~% DFnum:  ~S"
                                             subseg dfnum)))
                            (new-in (+& (& subseg) input nil
                                        (genobj subseg name) nil))
                            (new-out (+& (& subseg) output 'id
                                         (genobj subseg name)
                                         (~obj new-in))))
                       (6+df new-in dfnum)
                       (6+df new-out dfnum)
                       (setf (written hunk)
                             (6setU (written hunk) dfnum)) ;for join.
                       (+& new-in @io 'free-var name nil)
                       (+& new-out @io 'side-effect name nil)))))))))))

;note that the above may create two extra inputs that come from the
;exact same place if a setq or the like causes them to have different
;dfnums.  note that this will only happen if they are both live,
;so therfore since they are being used in two different places
;it isn't so gross, and also since there are two different dfnums you
;need to have sources for later, I cannot think of anyway to fix it.
```

```
;here we find what dfnums have to be joined at each join.  The basic idea is
;as follows:
;0. join checking only has to occure at those sections that are actually headed
;   by joins (the fn 6set-up-linkages sets things up so that you never have
;   to look at the other sections).
;1. only dfnums that are written in at least two different sections ever
;   need to be joined.  This is a considerable savings since many dfnums
;   are actually only written once (for example all those corresponding to
;   dflow implemented by nested expressions).
;2. if a dfnum comes from exactly the same place on every path into a join,
;   then it definitely does not have to be joined.
;3. if a dfnum comes from different places on two paths to the join, then it
;   must join.

;the last two criteria are easy to state but difficult to compute
;because of their very non-local nature.  A process of relaxation is used
;in order to propogate information over the graph in order to make
;a decision.  the heart of this, is that the system keeps track
;of a nonotonically increasing state of knowledge about what the
;source of every dfnum is.  An important thing to note is that
;there is never any connection between two different dfnums
;(the existance of the explicit setq boxes is what makes this possible)
;and so the computation of whether a dfnum should join never effects
;any other dfnum.  There are prev different states of knowledge,
;(listed in order from most to least informative.)

;1. DEFINITE = [S] the dfnum is actually written in the indicated
;   section, either by some basic fn, or because it is joined at the head of
;   this section.
;2. DEFERRED = [NIL ! source] the dfnum's source can be determined by
;   interpreting the cdr as one of these four.
;3. PROVISIONAL = [S ! source] this means that the dfnum will either
;   join at section S, or if it doesn't have to join, then the source will
;   be as specified in the cdr.
;4. MAYBE JOIN = [S NIL] it is possible that the dfnum may have to be joined
;   at the indicated section.
;Note that the defferred and provisional entries can be arbitrarily nested,
;which leads to long lists of hunks.  all such lists can be interpreted as
;follows (this is an alternate way of looking at the above).  if the last
;element if the list is non NIL, then it is a possible definite source.
;all other non-nil entries in the list are places where the dfnum may
;have to join.  If there are none of these possible joins, then the source
;is definite.

;The first thing the program does is determine which dfnums are
;written more than once (the others are never even considered for
;joining).  it then constructs alists for all these dfnums which go in
;SOURCE of each hunk. [the main program constructs an alist where
;every entry is MAYBE-JOIN, for each join-section. (and for the first
;section which cannot be a join, and which must be done so that there
;will be sources to start with.)  6set-up-linkages then computes the
;alist for the non join sections following each join section.  this
;process introduces DEFINITE entries in the alists, for all those
;dfnums that are actually written in the sections.  It is critical
;that all of the MAYBE-JOIN entries are shared, so that if a
;modification is made to them at the join section, it will immediately
;propogate to all of the other alists.  In addition to this, the main
;program puts in UNKNOWN of each join section, a list of those alist
;entries that correspond to dfnums that are LIVE at the join. (dead
;ones never have to be joined) throughout the computation, UNKNOWN
;holds the entries corresponding to dfnums that may or may not have to
;be joined, while JOIN holds a list of those dfnums that are known
;to have to be joined.]
```

```
;after the initial setup, computation goes as follows.  the program looks
;at each join section, and for each dfnum whose fate is still unknown,
;it tries to determine whether enough information exists about the sources
;of the dfnum, that it can be determined that it either must join, or must not.
;the compuation continues, propagating information through the graph
;until the fate of every dfnum has been decided.  At each cycle there are
;three possible outcomes for each dfnum.  They are all based on looking
;at the source for the dfnum on each path entering the join section.
;Before reading the following consider this lemma: [given a dfnum, and a
;section there is at most one cons cell with that section as its car that
;appears in any source list for that dfnum (it may appear in many source lists)
;as a result, if the same section appears in two source lists, then the lists
;must be EQ at that point and therefore have identical tails.]
;1. consider the first element of each source list which is not NIL.
;   discard all those elements that are eq to the section being processed,
;   and which are not definite sources.  If the remainder of the
;   elements are all eq, then the dfnum definitely does not have to join.
;   (the indefinite circular references can be ignored,
;   because they will automaticaly change to agree with any choice you make.)
;2. consider every final element of a list which is not NIL, if it is not
;   true that they are all eq, then the dfnum must be joined.
;   Note that in this comparison, the sources with nil at the end are
;   ignored, because they don't as yet correspond to any actual constraint
;   on what the ultimate source will be.
;3. otherwise you still don't know whether to join or not, but if there
;   is any source that does not have nil at the end, then it can be used
;   to make a provisional judgment about what the dfnum leaving the join
;   can be.  it is used to modify the entry for the dfnum in question,
;   so that this information will propogate to other joins.
;one feature of the process above is that the choice for each join is
;made seperately.  there are situations (for example 2 entry loops) where
;a decision can only be made if the decision is made at more than one
;join at a time, and therefore the simple process above would not terminate.
;  To deal with this we consider the following.  suppose a pas has been made
;over the whole data structure, and no change has been made (ie nothing has
;been joined by test 1, and no provisional judgment has been changed with
;test 3).  In this case, a steady state has been reached, and further
;iterations will not change anything.  further we know that test 2 is
;true at every point.  the essence of this test is that there would
;be no conflict if nothing at all was joined anywhere.  As a result,
;iteration stops here.  we know that everything that must join has been joined.
```

```
(declare (special 6CHANGE))

(defun 6find-joining ()
  (let* ((join-info (over INFO select (cdr (prev _)) bag _))
         written-twice written-once w2-dfnums)
    (dolist (hunk INFO)
      (setq written-twice (6setU written-twice
                                 (6set^ (append written-once nil)
                                        (written hunk))))
      (setq written-once (6setU written-once (written hunk))))
    (setq w2-dfnums (over DF-NUMS select (6set^? _ written-twice) bag _))
    (6set-up-linkages first-hunk
                      (over w2-dfnums -> dfn
                       bag [dfn first-hunk]))
    (over join-info -> hunk
          (live (car (prev hunk))) -> live
          (over w2-dfnums -> dfn bag [dfn hunk nil]) -> alist
      (setf (unknown hunk) (over alist -> [! entry dfn !_]
                                 select (6set^? dfn live) bag entry))
      (6set-up-linkages hunk alist))
    (repeat (6CHANGE) ;loop until everything is decided.
      (setq 6CHANGE nil)
      (dolist (hunk join-info)
        (6process-jhunk hunk))
      (if (null 6CHANGE) (return)))))

(defun 6set-up-linkages (hunk alist)
  (setq alist
        (over alist -> [dfn ! old]
         bag [dfn ! (cond ((6set^? (written hunk) dfn) [hunk]) (t old))]))
  (setf (source hunk) alist)
  (over (next hunk)
    select (null (cdr (prev _)))
    (6set-up-linkages _ alist)))

;this sets 6change iff some change happens.

(defun 6process-jhunk (hunk)
  (let* (unknown join)
    (over (unknown hunk) -> [! item dfnum ! entry]
          nil -> source nil -> source-list
          nil -> section nil -> provisional
          (over (prev hunk) -> h
                (source h) -> alist
            bag (alist dfnum)) -> knowledge
      (cond ((over knowledge -> list
                fastand (over l <- list -> item
                         select item
                         (return (cond ((and (eq item hunk) (cdr l)))
                                       (section (eq item section))
                                       (t
                                        (setq section     item
                                              source-list list))))))
             (setf (car entry) nil)
             (setf (cdr entry) source-list)
             (setq 6CHANGE t))
            ((over knowledge -> list
                   (car (last list)) -> item
                   (cond ((null item))
                         (source (if (not (eq item source)) (return t)))
                         (t (setq source      item
                                  provisional list))))
             (setf (cdr entry) nil)
             (push dfnum join)
             (setq 6CHANGE t))
            (t
             (push item unknown)
             (if (equal (cdr entry) '(nil))
```

```
                (cond (provisional
                       (setf (cdr entry) provisional)
                       (setq 6CHANGE t)))))))
    (setf (unknown hunk) unknown)
    (setf (join hunk)
          (nconc join (join hunk)))))
```

```
;this just creates the ports called for on the joins.
;this is done during the data flow creation, so that you can
;get more reasonable mnemomic names.

(defun 6do-joining (hunk)
  (let* ((joinsegcase (start hunk)))
    (over (join hunk) -> dfnum
          (or (DF-NAME dfnum) '_obj) -> name
          (+& joinsegcase output nil
              (genobj (~seg joinsegcase) name) nil) -> out
      (6+df out dfnum)
      (+& out @io 'free-var name nil)
      (over (prev hunk) -> h
            (+& (end h) input 'id
                (genobj (~seg (end h)) name) (~obj out)) -> in
        (6+df in dfnum)
        (+& in @io 'free-var name nil)))))

;this actually creates the dflow assertions.  If takes particular
;advantage of the fact that once the joining has been done, each sink
;port has dflow from just a single place.  Therefore, as soon as you
;know A source for the dflow, you know THE source for the dflow.
;(As a side action, when it reaches the end of the program, it looks to
;see if any of the inputs to the program have been side-effected, and creates
;appropriate ports and dflow if they have been.  [It also removes the input
;if it turns out that it was only used as an output (this is only relavent
;to FORTRAN).]) {I thought of having this part get rid of the DF properties
;(it has to visit them all anyway), but there is no easy way to do that
;without getting rid of some that you want to keep around till the very end.}

(defun 6create-dflow ()
  (let* ((alist (over (??& SUPERSEGCASE input) -> in
                      bag [(6?df in) ! in])))
    (6create-dflow1 alist FIRST-HUNK)))

;this makes the dflow in a single section, and passes the alist along to
;the following sections. it assumes that the alist is correct for the start.

(defun 6create-dflow1 (alist hunk)
  (cond ((null (flag hunk))
         (setf (flag hunk) t)
         (repeat ((segcase (start hunk)))
           (setq segcase (~to (?& SUPERSEGCASE cflow segcase)))
           (when (equal segcase SUPERSEGCASE)
             (6make-super-outputs alist)
             (return))
           (dolist (in (??& segcase input))
             (+& SUPERSEGCASE dflow (alist (6?df in)) in))
           (dolist (out (??& (&* (~seg segcase)) output))
             (setf (alist (6?df out)) out))
           (when (equal segcase (end hunk))
             (over (next hunk) (6create-dflow1 (copyalist alist) _))
             (return))))))
```

```
;this makes dflow to the outputs of the superseg, and adds side-effect
;outputs if needed.  Once all of the outputs are finally known, it
;looks at PART of the last section in order to determine the part
;relationships between the outputs and the inputs (it's a bit bugy the
;way this is done).

(defun 6make-super-outputs (alist)
  (let* ((out-nums (over (??& SUPERSEGCASE output) -> out
                         6setU (6?df out)))
         (parts (6find-parts1 (start last-hunk) (end last-hunk)
                              (part last-hunk))))
    (over (??& SUPERSEGCASE input) -> in
          (6?df in) -> dfn
      select (and (not (6set^? dfn out-nums))
                  (not (eq in (alist dfn))))
      (let* ((@io (?& in @io)) out)
        (cond ((??& SUPERSEGCASE dflow in)
               (setq out (+& SUPERSEGCASE output 'id
                             (genobj (~seg SUPERSEGCASE) (~obj in))
                             (~obj in)))
               (+& out @io 'side-effect
                   (if (eq (~kind @io) 'free-var) (~name @io)) nil))
              (t (make-fact-be-out @io)
                 (setq out ;eg in fortran
                       (add-fact (&~keyword (make-fact-be-out in)
                                            'output)))
                 (add-fact (&~deep (make-fact-be-out @io) out))))
        (6+df out dfn)))
    (over (??& SUPERSEGCASE output) -> out
          (6?df out) -> dfn
          (parts dfn) -> p
          (alist dfn) -> from
      (cond ((null from)
             (--& out @io)
             (--& out @createvar)
             (make-fact-be-out out))
            (t (over (??& SUPERSEGCASE input) -> in
                     (6?df in) -> d
                (when (6set^? d p)
                  (if (null (~kind out))
                      (let ((@io (-& out @io)))
                        (make-fact-be-out out)
                        (setq out (>& SUPERSEGCASE output
                                      'part (~obj out) (~obj in)))
                        (assert-fact (&~deep @io out))))
                  (return)))
               (+& SUPERSEGCASE dflow from out))))))
```

```
;this runs over the plan cleaning up the final details.
;at the moment, this consists of
;1. removing the k*psetq segments.
;2. removing every segment that doesn't have any dflow to anywhere,
;   and which doesn't have a part assertion on it.
;3. consolidate any joins that are adjacent (1,2 may have unconvered some)

(defvar 6ignored-fns
  '(ada-use~pseudo ada-dcl~pseudo ada-constant-dcl~pseudo
    ada-init-dcl~pseudo ada-type-dcl~pseudo ada-subtype-dcl~pseudo
    ada-new-package~pseudo ada-new-fn~pseudo def-ada-fn~pseudo
    def-ada-generic~pseudo def-ada-package~pseudo
    def-ada-package-body~pseudo ada-file-def~pseudo
    ada-fn~create ada-fn~open ada-fn~reset ada-fn~read
    ada-fn~write ada-fn~set_index ada-fn~get))

(defun 6cleanup (SUPERSEGCASE)
  (dolist (subseg (??subseg (~seg SUPERSEGCASE)))
    (when (??& (&* subseg) @seg 'funcall 'k*psetq)
      (over (??& (&* subseg) output) -> out
            (?& (&* subseg) input * (~id out)) -> in
            (~from (-& SUPERSEGCASE dflow * in)) -> source
        (over (--& SUPERSEGCASE dflow out)
              (+& SUPERSEGCASE dflow source (~to _))))
      (assert-fact (&~from (-& SUPERSEGCASE cflow (&* subseg))
                           (~from
                            (-& SUPERSEGCASE cflow * (&* subseg)))))
      (kill-seg subseg)))
  (dolist (subseg (??subseg (~seg SUPERSEGCASE)))
    (if (and (eq (?segtype subseg) 'simple)
             (null (??& SUPERSEGCASE dflow (&* (&* subseg) output)))
             (null (??& SUPERSEGCASE slot (&* subseg)))
             (null (??& (&* (&* subseg) output) @io 'side-effect))
             (not (memq (~01 ~name (??& (&* subseg) @seg 'funcall))
                        6ignored-fns)))
        (let ((constant (~01 ~name (??& (&* subseg) @seg 'constant))))
          (cond ((null constant)
                 (grumble "6CLEANUP: No used outputs for fn.~% subseg: ~S"
                          subseg))
                (t
                 (if (not (member constant '('nil 'undefined)))
                     (grumble "6CLEANUP: Constant wasn't used.~% Data: ~S"
                              subseg))
                 (assert-fact
                  (&~from (-& SUPERSEGCASE cflow (&* subseg))
                          (~from (-& SUPERSEGCASE cflow * (&* subseg)))))
                 (kill-seg-and-its-subsegs subseg))))))
  (dolist (subseg (??subseg (~seg SUPERSEGCASE)))
    (when (eq (?segtype subseg) 'join)
      (let ((next (~seg (~to (?& SUPERSEGCASE cflow (&* subseg))))))
        (if (and (??& (&* next) segtype 'join)
                 (over (??& SUPERSEGCASE dflow (&* (&* subseg) port))
                  fastand (eq (~seg (~segcase (~to _))) next)))
            (fuse-joins subseg next))))))

;this sets things up that are needed in order for these fns to run
;probably shouldn't be in this file.

(defun 6set-up-flows ()
  (db-init)
  (push-db-context)
  (db-context-was >I))

(6set-up-flows)
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp REPEAT Indent:1;
;;; Lisp REPEAT* Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

(declare (*expr analyze code))

;;; (PROCESS spec)
;;;  Used to read in a program, make an analyzed plan for it,
;;;  and then dump out the plan in a file which can be read back
;;;  in, and which is easy for a person to read.
;;;
;;;  If the spec is a string, it specifies the input filename.
;;;  If the spec is a symbol, the PGMS file will be assumed and that
;;;     tag in PGMS will be done only. (cf *PREAD)
;;;
;;;  The data base dump will appear in a file with second filename PLAN.
;;;  There may be more than one fn in the input file.  If there are
;;;  then there will be multiple entries in the output file.
;;;
;;;  Each fn definition in the input must be a call on the fn DEFINE
;;;  eg,
;;;      (define hash (symbol) (abs (remainder (maknum symbol) 621.)))
;;;
;;;  Each entry in the PLAN output is a call on the fn PLAN-DEFINIITION.
;;;  All you have to do is evaluate the forms in order to get the plan
;;;  back into the data base (eg by loading the file) each call on
;;;  PLAN-DEFINITION returns the segcase of the top level seg being
;;;  defined.
;;;
;;;  The output in the PLAN file is described in comments in the source
;;;  to the DB module.

(defun process (arg)
  (cond ((symbolp arg)
         (process-aux arg
                      nil
                      (file-default-fn2 (get-pname arg) 'PLAN ':newest)))
        ((stringp arg)
         (process-aux nil
                      (file-default-fn2 arg nil ':newest)
                      (file-default-fn2 arg 'PLAN ':newest)))
        (t
         (ferror "Arg to PROCESS must be a symbol or a string: ~S" arg))))

(defun process-aux (name input-filename output-filename)
  (cond ((and input-filename (not (probef input-filename)))
         (ferror "Input file doesn't exist: ~A" input-filename))
        ((probef output-filename)
         (ferror "Output file exists: ~A" output-filename)))
  (with-open-file (outstream output-filename ':OUT)
    (cond (name ;if we got a name, get it from the PGMS file...
           (process-1 (*pread name) outstream nil))
          (t    ;else munch up everything in the file we were given...
           (with-open-file (instream input-filename ':IN)
             (do ((pgm (read instream nil) (read instream nil))
                  (flag nil t))
                 ((null pgm))
               (process-1 pgm outstream flag))))))
  (db-audit)
  (beep))

(defun process-1 (pgm outstream page?)
```

```
(let ((segcase (analyze (eval pgm))))
  (if page? (tyo #\page outstream))
  (disp-print (~name (?& segcase @seg)) outstream)))
```

```
;;; This page contains definitions of some non-standard lisp functions
;;; which is referenced by code produced by the Knowledge-Based Editor.

(defun *equal (x y) (equal x y))

(defun arrayfetch (array index) ;only vectors for now
  (arraycall t array index))

(defun arraystore (array index item) ;only vectors for now
  (store (arraycall t array index) item))
```

```
;;; (BDB context)
;;;  Binds a new context under a given context.

(defun bdb (context)
  (set-db-context context)
  (push-db-context))

;;; (RDB context)
;;;  Restores the data-base back to a given context.

(defun rdb (context)
  (if (null context) (barf "No context -- RDB"))
  (restore-db-context context))
```

```
;;; This section deals with getting example programs from files in a
;;; certain tagged format. The file is a sequence of records of the form:
;;;
;;;     ;^Akey
;;;     expression
;;;
;;; The last record is ;^A|eof|
;;;
;;; The ^A must be the second character on the line. The first should
;;; probably be a semicolon. The tag is the next lisp s-expression on
;;; that line and should be a symbol.
;;;
;;; It is the intent that the expression should be a DEFINE or DEFINEF
;;; form or something like that.
;;;
;;; Convenience macros for user interaction are provided at the bottom
;;; of this file.

(defvar *pgms-filename* "KBE: MISC; PGMS LISP >" "Sample program storage")

(defvar last-*pread nil "Used by *PREAD as a history mechanism.")

;;; Reads a tagged thing from the PGMS file.

;;; (*PREAD key file)
;;;
;;;  Reads a form from a given file. The file must be marked with a
;;;  the tag, key, as described above.

(defun *pread (thing)
  (setq last-*pread
        (or (*pread-1 thing *pgms-filename*)
            (grumble "Test program not found: ~S" thing)))
  last-*pread)

(defun *pread-1 (key file) ;should probably bind PACKAGE --kmp
  (with-open-file (instream file ':in)
    (do ((line (readline instream nil) (readline instream nil)))
        ((null line) nil) ;Return NIL if we fall off the end
      (cond ((and (> (string-length line) 2)
                  (eq (aref line 1) #/^A)
                  (eq (read-from-string line nil 2) key))
             (return (read instream)))))))) ;Return tagged form if found

;;; For reading/analyzing/coding with little-error checking.

(defun *snarf (item)
  (let ((*db-check-facts* nil))
    (code (analyze (eval (*pread item))))))
```

```
;simple little fns to aid in running tests and printing out info.

(defun *disp (form) (disp-print form standard-output))

(defun *disp-dump (name &optional (filename (get-pname name)))
  (let ((output-filename (file-default-fn2 filename 'PLAN ':newest)))
    (cond ((probef output-filename)
           (ferror "Output file exists: ~A" output-filename)))
    (with-open-file (outstream output-filename ':OUT)
      (disp-print name outstream))
    'done))

;this prints a plan out in a form that is both easily readable, and
;which can be read back in in order to restore the data base.  several
;pieces of commentary are also printed out.  this is done in a way that
;prevents them from doing anything when they are read back in.

(defun disp-print (name f)
  (let ((segcase (~01 ~segcase (??& * @seg 'lambda-exp name))))
    (terpri f)
    (multiple-value-bind (def pas)
        (construct-plan-def (~seg segcase))
      (cond ((??& segcase plantype)
             (format f "~%'(~:=)~%" pas)))
      (pp:pp def f))))
```

```
;note you cannot load the result below in, because you will
;get triple definitions of everything.

(declare (special <anal <code))

(defun *print-test (name)
  (let ((saved-context *db-context*)
        (output-filename (file-default-fn2 (get-pname name)
                                           'TEST ':newest)))
    (cond ((probef output-filename)
           (ferror "Output file exists: ~A" output-filename)))
    (with-open-file (outstream output-filename ':OUT)
      (unwind-protect
        (progn
          (disp-print name   outstream)
          (tyo       #\form outstream)
          (set-db-context <code)
          (disp-print name   outstream)
          (tyo       #\form outstream)
          (set-db-context <anal)
          (disp-print name   outstream))
        (set-db-context saved-context)))
    (beep)
    'done))
```

```
;;; These macros are just for convenience of type-in. They should not
;;; be called by code.

;;; (PREAD [item])
;;;
;;;  Reads item from the PGMS file.
;;;
;;;  With no argument, just returns the result of the last PREAD,
;;;  *PREAD or PEVAL.

(defmacro pread (&rest stuff)
  (cond (stuff '(*pread ',(car stuff)))
        (t     'last-*pread)))

;;; (PEVAL item)
;;;
;;;  Reads and evaluates an item from the PGMS file.

(defmacro peval (item)
  '(eval (*pread ',item)))

;;; (SNARF item)
;;;
;;;  This is a fast way to get a program read in, analyzed, and coded.
;;;  It assumes you know there will be no errors and turns off some of
;;;  DB's errror checking.

(defmacro snarf (item) '(*snarf ',item))

;;; (DISP item)
;;;
;;;  Prints on the terminal the plan (a la PP) and a summary (a la PAS).

(defmacro disp (form) '(*disp ',form))

;;; (DISP-DUMP item)
;;;
;;;  Prints out the plan for the function, name, to a file with
;;;  first-filename of the function and second-filename PLAN.
;;;
;;;  To specify a particular filename rather than getting the default,
;;;  use the ordinary function *DISP-DUMP, described above.

(defmacro disp-dump (name) '(*disp-dump ',name))

;;; (PRINT-TEST name)
;;;
;;;  Does a DISP-PRINT of name in the current context, the <CODE context,
;;;  and the <ANAL context. The output goes to a file with first filename
;;;  of the name of the function and second filename TEST.

(defmacro print-test (x) '(*print-test ',x))

;;; (RUN-TEST test)
;;;
;;;  Reads, evaluates, analyzes, codes, and prints out analysis
;;;  a function named test to a file.

(defmacro run-test (test)
  '(progn (code (analyze (peval ,test)))
          (print-test ,test)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

(declare (special segfacts segtypes))

(define-trace-option *SEGTRN-Debug*
                     "Show trace of SEGTRN internals"
  nil :boolean)

;what follows is a set of utilities for performing simple
;transformations on plans.  basically they try very hard to do the
;right things with input, output, segtype, subseg, dflow, and cflow
;assertions. they try to be reasonable (though not creative) with
;expect, assert, plantype and role assertions.  in general they do not
;try to do anything with @seg, @io ,@cflow, @dflow, comment, celink,
;speclink, or support assertions. in fact they sort of assume that
;there are none of these assertions, if there are, it shouldn't mess
;anything up, but some of them might get deleted when a seg they
;reference is deleted (some might not which is even worse).  a comment
;with each program tries to detail the effect that that program has on
;each type of asssertion.

;all these functions are intended to work correctly if there is
;sharing of subsegs (ie. a seg with more than one superseg). it is not
;clear what would happen if a seg was the subseg of the same seg
;twice. (i am not sure that that would be reasonable (note that this
;is distinct from a seg filling two roles)).

;these routines tacitly assume that dflow, cflow, subseg, and role
;assertions never cross over a segment boundry.  if this is true, then
;the routines never cause any of these multi-level assertions to be
;created.
;    we realize that at least multi-level dflow, cflow, and subseg
;assertions can exist (they clearly must get made by other than these
;routines) the routines should work ok when they are present because
;it will not detect that they are there.
```

```
;the purpose of this function is to quickly hack up a minimal set of
;specs for a segment. it takes as argument a list whose first element
;is the function name and the rest of the elements of which are type
;predicates (for Fortran, _ip _rp or _dp) the first type predicate
;applies to the output, the rest apply to the inputs in order.
;quicky-defspecs then generates a segname (which it returns) and
;segtype and @seg assertions it then makes an input, @io, and expect
;(using the type code) for each input, and an output, @io, and assert
;(using the type code) for the output. in addition it produces an
;assert that the output is equal to the result of applying the
;function to the inputs.

;THE CREATION OF EXPECTS AND ASSERTS HAS BEEN TEMPORARILY SUSPENDED
;IN ORDER TO SAVE TIME AND SPACE, SINCE WE DON'T DO ANYTHING WITH THEM NOW.
;THE CODE IS JUST COMMENTED OUT.
(defun quicky-defspecs (specs)
  (let* (((name out . ins) specs)
         (seg (genseg name))
         (segcase  (& seg))
         (outobj (genobj seg 'z))
         (inobjs (over ins -> in
                   over '(x y) by (or (cdr _) '(arg)) until nil -> ob
                    list (genobj seg ob))))
    (+& segcase @seg 'lambda-exp name nil)
    (+& segcase segtype 'simple)
    (+& (+& segcase output nil outobj nil)
        @io 'returnval nil nil)
    (over inobjs -> inobj
     over ins until nil -> in
     over i <- 1 by (1+ i) until nil
      (+& (+& segcase input nil inobj nil)
          @io 'lambda-arg nil i))
    (~seg segcase)))

;this subroutine deletes a seg, everything refering to it, and then
;recurses deleting everything about its subsegs and roles

(defun kill-seg-and-its-subsegs (seg)
  (dolist (subseg (??subseg seg)) (kill-seg-and-its-subsegs subseg))
  (kill-seg seg))
```

```
;this function groups a set of segs into 1 seg.  it takes a list of
;segs about which it assumes 3 things (1) they are all subsegs of the
;same seg [which is refered to as SSEG (super-seg) in the program] (2)
;there is at most one cflow path into the set of segs [there may be
;none]. (3) all data flow out of the set of segs can go through the
;the nil case [this restriction may not be reasonable but it makes
;things a lot easier].  if SSEG contains no segs except for the segs
;to be grouped then no work is done and SSEG is returned.  Otherwise a
;new seg USEG (upper-seg) is created [the arg prefix is used to make a
;mnemonic name for USEG].
;       the intent of this is that it work just like you had drawn a
;box around the grouped segs on a piece of paper. after drawing the
;box, some SUBSEG, DFLOW, CFLOW, TINPUT, TOUTPUT and ROLE links are
;liable to cross the new boundry.  these are fixed up by breaking them
;in the middle and forcing them to pass explicitly through the
;boundry. with DFLOWS this causes INPUT and OUTPUT ports to be created
;(the routine tries to generate the minimum number of ports needed).
;with CFLOW this may cause there to be cases on the new seg (in this
;case a segtype of 'PRED rather than 'EXPR is given to the new seg).
;with SUBSEGs there is no problem. with ROLEs, any roles the grouped
;segs used to have in the superseg are deleted, but no new roles are
;given to them or to the new seg.
;       it is intended that the grouping will not cause there to
;exist any assertions which skip over a segment boundry.  however, if
;some assertions already skip over boundries, they will probably not
;be detected, and will continue to be correct.
;       after grouping you must explicitly create the PLANTYPE and
;ROLEs if any.
;       note that this isn't going to work if you try to group up an
;uppermost seg.

(defun group (a b) (group2 a b 'conditional))

(defun group2 (dsegs prefix conditional?)
  (let* ((sseg (?supseg (car dsegs))) useg)
    (cond ((and (eq conditional? 'conditional)
                (qset= dsegs (??subseg sseg)))
           (setq useg sseg))
          (t
           (setq useg (genseg prefix))
           (>& (& sseg) subseg (& useg))
           (dolist (dseg dsegs)
             (--& * role (&* dseg))
             (-& * subseg (&* dseg))
             (>& (& useg) subseg (& dseg)))
           (g2move-cflows sseg useg dsegs)
           (g2move-dflows sseg useg dsegs)
           ;(g2move-tports useg dsegs) ;see note on fn.
           ))
    (format-trace *SEGTRN-Debug* "~S ^X ~:=" useg '(group ,dsegs ,prefix))
    useg))
```

```
(defun g2move-cflows (sseg useg dsegs)
  (let* (ingoing inside outgoing)
    (dolist (cf (??& (&* sseg) cflow))
      (cond ((memq (~seg (~from cf)) dsegs)
             (cond ((memq (~seg (~to cf)) dsegs)
                    (push cf inside))
                   (t (push cf outgoing))))
            (t (if (memq (~seg (~to cf)) dsegs)
                   (push cf ingoing)))))
    (over inside
      (make-fact-be-out _)
      (>& (& useg) cflow (~from _) (~to _)))
    (if (cdr ingoing)
        (barf "Too many ingoing cflows in group.~% Data: ~S"
              [useg dsegs]))
    (if ingoing (cflow-splice-in (car ingoing) (& useg)))
    (cond ((cdr outgoing)
           (>& (& useg) segtype 'split)
           (over outgoing (cflow-splice-in _ (& useg (gencase useg)))))
          (t
           (>& (& useg) segtype 'simple)
           (if outgoing (cflow-splice-in (car outgoing) (& useg)))))))

(defun g2move-dflows (sseg useg dsegs)
  (let* (ingoing inside outgoing)
    (dolist (df (??& (&* sseg) dflow))
      (cond ((memq (~seg (~segcase (~from df))) dsegs)
             (cond ((memq (~seg (~segcase (~to df))) dsegs)
                    (push df inside))
                   (t (push df outgoing))))
            (t (if (memq (~seg (~segcase (~to df))) dsegs)
                   (push df ingoing)))))
    (over inside
      (make-fact-be-out _)
      (>& (& useg) dflow (~from _) (~to _)))
    (over (over ingoing classify [(~from _) ! _]) -> [from ! idflows]
          nil -> iport
      (cond ((io-fact? from)
             (setq iport
                   (>& (& useg) input nil (genobj useg (~obj from)) nil))
             (>& (& sseg) dflow from iport)
             (over idflows
               (make-fact-be-out _)
               (>& (& useg) dflow iport (~to _))))
            (t
             (over idflows
               (>& (& sseg) dflow from
                   (>& (& useg) tinput (~segcase (~to _))
                       (genobj useg (~obj (~to _))) (~to _)))))))
    (over (over outgoing classifyq [(~from _) ! _]) -> [from ! odflows]
          nil -> oport
      (cond ((io-fact? from)
             (setq oport
                   (>& (& useg) output nil (genobj useg (~obj from)) nil))
             (>& (& useg) dflow from oport))
            (t
             (setq oport (>& (& useg) toutput (~segcase from)
                             (genobj useg (~obj from)) from))))
      (over odflows
        (make-fact-be-out _)
        (>& (& sseg) dflow oport (~to _))))))
```

```
;I <Dick> don't understand what this is doing!
;I have spliced it out for now.  It seems to make some things blow
;up and is clearly partly redundant with g2move-dflows

(defun g2move-tports (useg dsegs)
  (over (over dsegs nconc (??& * tinput * * (&* (&* _) input))) -> ti
        (~lower-io ti) -> li
        (+& (& useg) input nil (genobj useg (~obj li)) nil) -> newli
    (+& (& useg) dflow newli li)
    (^dflow-to ti (+& (~segcase ti) tinput (& useg) (~obj ti) newli))
    (kill-port ti))
  (over (over dsegs nconc (??& * toutput * * (&* (&* _) port))) -> to
        (~lower-io to) -> lio
        (~lower-env to) -> le
        (cond ((memq (~seg le) dsegs)
               (cond ((~case le) (~to (?& (&* useg) cflow le (&* useg))))
                     (t (& useg))))
              (t le))
          -> newle
        (cond ((input-fact? lio)
               (~from (?& (&* useg) dflow * lio)))
              (t lio))
          -> newlio
    (if (output-fact? newlio)
        (setq newlio
              (cond ((~01 ~to (??& * dflow newlio (&* (&* useg) output))))
                    (t (~to (+& (& useg) dflow lio
                                (+& (& useg) output nil
                                    (genobj useg (~obj lio)) nil))))))))
    (^dflow-from to (>& (~segcase to) toutput newle
                        (genobj (~seg~segcase to) (~obj lio)) newlio))
    (kill-port to))
  (over (over dsegs nconc (??& * toutput (&* _))) -> to
        (~lower-env to) -> le
    (^dflow-from to
      (assert-fact (&~lower-env to
                                (cond ((~case le)
                                       (~to (?& (&* useg)
                                                cflow
                                                le
                                                (&* useg))))
                                      (t (& useg))))))
    (kill-port to)))
```

```
;this routine is the inverse of group. ungroup takes as argument, a
;useg and makes all its subsegs (dsegs) subsegs of its superseg (sseg)
;it deletes useg and every assertion mentioning it and fixes up all
;the subseg dflow and cflow assertions.
;  One problem area is involved with cflow.  If sseg and useg either
;both have, or neither have cflow then things are easy.  However, if
;only one has, then there is a problem.  The basic solution is to give
;the one without cflow some cflow which is compatable with its dflow.
;The one exception to this comes when the seg with cflow has atmost
;one subseg, and this seg is simple (as apposed to a split).  In this
;case, the cflow in the seg is deleted, and there is no cflow in the
;result.
;    A more complex problem comes up with plantype and role
;information.  There are two cases which can be delt with reasonably.
;Outside of these cases, any such information refering to useg is
;simply deleted.  One case is when useg fills some role in sseg, and
;useg has exactly one subseg.  In this case, this subseg assumes the
;role.  the other case is when useg is the only subseg of sseg.  In
;this case sseg assumes the complete plantype information from useg.

(defun ungroup (useg)
  (format-trace *SEGTRN-Debug* "(UNGROUP ~S) ~S ~S"
                useg (?supseg useg) (??subsg useg))
  (cond ((?supseg useg) (ungroup-middle useg))
        ((cdr (??subseg useg))
         (barf "cannot ungroup top seg (~A) with >1 subseg" useg))
        (t (ungroup-middle (car (??subseg useg)))))
  nil)

;this subroutine takes a seg with no cflow, and puts in clfow
;compatible with the df in the seg.  It is assumed that all of the
;subsegs are simple (ie top seg is an EXPR or something)

;becuase this is taking away choices that would otherwise be
;available to the coder, it should try to make the order look
;good with regard to nesting of expressions and the like.
;Unfortunately, it does not do that.  maybe someday -- dick

(def-alistq dffrom)    ;fsets showing where dflow comes from to a seg

(defun make-compatable-cflow (seg)
  (let* ((segpat (&* seg))
         (segcase (& seg))
         (subsegs (??subseg seg))
         (dffrom (over subsegs -> sub
                   list [sub ! (over (??& segpat dflow *
                                         (&* (&* sub) port))
                                     makefset (~seg~segcase (~from _)))]))
         (lastseg seg)
         (segsdone [seg]))  ;this is an fset
    (repeat ()
      (if (null subsegs) (return))
      (dolist (sub subsegs)
        (when (fset< (dffrom sub) segsdone)
          (>& segcase cflow (& lastseg) (& sub))
          (setq subsegs  (delq sub subsegs)
                segsdone (fsetU segsdone [sub])
                lastseg  sub))))
    (>& segcase cflow (& lastseg) (& seg))))
```

```
(defvar *force-slots-up* nil)

(defun ungroup-middle (useg)
  (let* ((sseg (?supseg useg))
         (dsegs (??subseg useg))
         (brothers? (not (null (cdr (??subseg sseg)))))
         (init (?role useg 'init)))
;check that the cflow is compatable between useg and sseg, if not
;fix it up.
    (cond ((??& (&* sseg) cflow (&* useg))
           (cond ((null (??& (&* useg) cflow))
                  (cond ((and (null (cdr (??subseg sseg)))
                              (eq (?segtype useg) 'simple))
                         (--& (&* sseg) cflow))
                        (t (make-compatable-cflow useg))))
                 ((and init (null (??& (&* useg) cflow * (&* init))))
                  (cflow-splice-in (?& (&* useg) cflow (&* useg)) (& init))
                  (>& (& useg) cflow (& useg) (& init)))))
          (t
           (if (??& (&* useg) cflow)
               (cond ((or (null dsegs)
                          (and (null (cdr dsegs))
                               (eq (?segtype (car dsegs)) 'simple)))
                      (--& (&* useg) cflow))
                     (t
                      (make-compatable-cflow sseg))))))
;fix up subseg relations
    (-& (&* sseg) subseg (&* useg))
    (over dsegs
      (-& (&* useg) subseg (&* _))
      (+subseg sseg _))
;fix up plantype and role information if possible
    (cond ((and (null (cdr dsegs))
                (null (??& (&* useg) plantype 'tempcomp)))
           (--& (&* useg) plantype)
           (--& (&* useg) role)
           (--& (&* useg) cliche)
           (--& (&* useg) slot)
           (over (--& * role (&* useg))
                 (assert-fact (&~roleseg _ (& (car dsegs)))))
           (ugmove-slots-down useg (car dsegs)))
          ((not brothers?)
           (--& (&* sseg) plantype)
           (--& (&* sseg) role)
           (over (--& (&* useg) plantype)
                 (assert-fact (&~segcase _ (& sseg))))
           (over (--& (&* useg) role)
                 (assert-fact (&~segcase _ (& sseg))))
           (let ((lower-cliche (~01 (--& (&* useg) cliche))))
             (when (and lower-cliche (null (??& (&* sseg) cliche)))
               (assert-fact (&~segcase lower-cliche (& sseg)))
               (over (--& (&* useg) slot)
                     (assert-fact (&~segcase _ (& sseg))))))
           (ugmove-slots-up sseg useg))
          (t
           (--& (&* useg) plantype)
           (--& (&* useg) role)
           (--& (&* sseg) role (&* useg))
           (--& (&* useg) cliche)
           (cond (*force-slots-up*
                  (over (--& (&* useg) slot)
                        (assert-fact (&~segcase _ (& sseg)))))
                 (T (--& (&* useg) slot)))))
    (ugmove-tports useg)
    (ugmove-cflow sseg useg)
    (ugmove-dflow sseg useg)
    (kill-seg useg)))
```

```
(defun ugmove-slots-up (sseg useg)
  (over (--& * slot (&* useg))
    (if (null (??& * slot (&* sseg)))
        (assert-fact (&~slotseg _ (& sseg))))))

;note cannot be a tc
(defun ugmove-slots-down (useg dseg)
  (over (--& * slot (&* useg))
    (if (null (??& * slot (&* dseg)))
        (assert-fact (&~slotseg _ (& dseg))))))

;this should not destroy any dflow in useg
;and must run before any dflow is destroyed.
(defun ugmove-tports (useg)
  (over (??& (&* useg) tinput) (^dflow-to _ (~lower-io _)))
  (over (??& (&* useg) toutput) (^dflow-from _ (~lower-io _)))
  (dolist (ti (??& * tinput * * (&* (&* useg) input)))
    (dolist (df (??& (&* useg) dflow (~lower-io ti)))
      (=dflow-to ti (>& (~segcase ti) tinput (~segcase (~to df))
                        (genobj (~seg~segcase ti) (~obj ti)) (~to df))))
    (kill-port ti))
  (over (??& * toutput * * (&* (&* useg) port))
        -> to
        (cond ((input-fact? (~lower-io to))
               (~to (a?& (&* useg) dflow (~lower-io to))))
              (t
               (~from (?& (&* useg) dflow * (~lower-io to)))))
        -> newlio
        (cond ((eq (~seg (~lower-env to)) useg)
               (~segcase newlio))
              (t
               (~lower-env to)))
        -> newle
    (^dflow-from to (>& (~segcase to) toutput newle
                        (genobj (~seg~segcase to) (~obj to)) newlio))
    (kill-port to)))

;fix up cflow
(defun ugmove-cflow (sseg useg)
  (let* ((in (~01 (--& (&* sseg) cflow * (&* useg))))
         (lower-to (and in (~01 ~to (--& (&* useg) cflow (~to in))))))
    (if (and in lower-to) (+cflow (~from in) lower-to)))
  (over (--& (&* useg) cflow * (&* useg)) -> inside
        (~01 ~to (--& (&* sseg) cflow (~to inside))) -> outer-to
    (if outer-to (+cflow (~from inside) outer-to)))
  (over (--& (&* useg) cflow)
    (assert-fact (&~segcase _ (& sseg)))))

(defun ugmove-dflow (sseg useg)
  (over (??input useg) -> port
        (~01 (--& (&* sseg) dflow * port)) -> dflowF
    (cond (dflowF (^dflow-from port (~from dflowF)))))
  (over (??output useg) -> port
        (~01 (--& (&* useg) dflow * port)) -> dflowF
    (cond (dflowF (^dflow-from port (~from dflowF)))))
  (over (--& (&* useg) dflow)
    (assert-fact (&~segcase _ (& sseg)))))
```

```
;this routine moves a seg (dseg) down into another seg useg (they must
;both be subsegs of the same seg sseg).  no segs are created or
;destroyed unless sseg is left with only useg as a subseg in which
;case sseg is imploded onto useg.  subseg, cflow and dflow assertions
;are fixed up.  when dseg is put in useg it is not put in the cflow if
;any.  the only thing that happens with cflow is that dseg is spliced
;out of any cflow it is in in sseg (this is not going to work well if
;sseg is a pred) the whole idea of moving in is really anathama to
;cflow.  similarly roles are deleted but none are asserted.  another
;potential difficulty comes from the fact that if there is the wrong
;kind of dflow (ie dseg gets data from a seg which gets data from
;useg) then when dseg gets put into useg there will be circular dflow
;induced constraints on cflow.  the person who calls this routine must
;be sure that the dflow will be alright.  it is also probably not a
;good idea to move a segment into a shared segment.

(defun move-seg-down (useg dseg)
  (let* ((dsegcase (& dseg))
         (dsegpat (&* dseg))
         (usegcase (& useg))
         (usegpat (&* useg))
         (sseg (-supseg dseg))
         (ssegcase (& sseg))
         (ssegpat (&* sseg)))
    (format-trace *SEGTRN-Debug* "(MOVE-SEG-DOWN ~S ~S)" useg dseg)
    (--& * role dsegpat)
    (>& usegcase subseg dsegcase)
;
    (over (??& ssegpat dflow (&* usegpat output) (&* dsegpat input))
          -> dflow
          (~from dflow) -> source
          (?& usegpat dflow * source) -> sdflow
          (~from sdflow) -> ssource
       (make-fact-be-out dflow)
       (if (null (??& ssegpat dflow source))
           (kill-port source))
       (>& usegcase dflow ssource (~to dflow)))
    (over (--& ssegpat dflow * (&* dsegpat input)) -> dflow
          (~from dflow) -> source
          (~01 ~to (??& ssegpat dflow source (&* usegpat input))) -> input
      (when (null input)
        (setq input
              (>& usegcase input nil (genobj useg (~obj source)) nil))
        (>& ssegcase dflow source input))
      (>& usegcase dflow input (~to dflow)))
    (over (--& ssegpat dflow (&* dsegpat output) (&* usegpat input))
          -> dflow
          (~from dflow) -> source
          (~to dflow) -> dest
      (^dflow-from dest source)
      (kill-port dest))
    (over (--& ssegpat dflow (&* dsegpat output)) -> dflow
          (~from dflow) -> source
          (~01 ~to (??& usegpat dflow source (&* usegpat output))) -> exit
      (when (null exit)
        (setq exit
              (>& usegcase output nil (genobj useg (~obj source)) nil))
        (>& usegcase dflow source exit))
      (assert-fact (&~from dflow exit)))
;
    (let* ((cflowin (~01 (--& ssegpat cflow * dsegpat)))
           (cflowout (~01 (--& ssegpat cflow dsegpat))))
      (if (and cflowin cflowout)
          (assert-fact (&~from cflowout (~from cflowin)))))
;
    (if (null (cdr (??subseg sseg)))
        (cond ((null (?supseg sseg)) (ungroup useg))
```

```
            (t (ungroup sseg)))))
    nil)
```

```
;this routine is the inverse of move-seg-down. it moves a seg DSEG
;outof its superior USEG into that segs superior SSEG.  this is
;exactly similar to moving down and is subject to all the same
;warnings. if the number of subsegs of USEG drops to 1 or 0 useg is
;expunged by calling ungroup.

(defun move-seg-up (dseg)
  (let* ((useg (-supseg dseg))
         (sseg (?supseg useg)))
    (format-trace *SEGTRN-Debug* "(MOVE-SEG-UP ~S) ~S" dseg useg)
    (--& * role (&* dseg))
    (>& (& sseg) subseg (& dseg))
;
    (over (??& (&* useg) dflow
               (&* (&* useg) input) (&* (&* dseg) input)) -> dflow
          (~from dflow) -> source
          (?& (&* sseg) dflow * source) -> sdflow
      (make-fact-be-out dflow)
      (if (null (??& (&* useg) dflow source))
          (kill-port source))
      (>& (& sseg) dflow (~from sdflow) (~to dflow)))
    (over (--& (&* useg) dflow * (&* (&* dseg) input)) -> dflow
          (~from dflow) -> source
          (~01 ~to (??& (&* useg) dflow source (&* (&* useg) output)))
          -> exit
      (when (null exit)
        (setq exit
              (>& (& useg) output nil (genobj useg (~obj source)) nil))
        (>& (& useg) dflow source exit))
      (>& (& sseg) dflow exit (~to dflow)))
    (over (--& (&* useg) dflow
               (&* (&* dseg) output) (&* (&* useg) output)) -> dflow
          (~from dflow) -> source
          (~to dflow) -> dest
      (^dflow-from dest source)
      (kill-port dest))
    (over (--& (&* useg) dflow (&* (&* dseg) output)) -> dflow
          (~from dflow) -> source
          (~01 ~to (??& (&* sseg) dflow source (&* (&* useg) input)))
          -> input
      (when (null input)
        (setq input
              (>& (& useg) input nil (genobj useg (~obj source)) nil))
        (>& (& sseg) dflow source input))
      (>& (& useg) dflow input (~to dflow)))
;
    (let* ((cflowin (~01 (--& (&* useg) cflow * (&* dseg))))
           (cflowout (~01 (--& (&* useg) cflow (&* dseg)))))
      (if (and cflowin cflowout)
          (assert-fact (&~from cflowout (~from cflowin)))))
;
    (if (null (cdr (??subseg useg))) (ungroup useg))))
```

```
;this routine takes a join apart into two joins one of which feeds
;into the other.  the combination has the same number of net cases
;that the single one had.  the lower one has one extra case comming
;from the upper one.  (BREAK-UP-JOIN TOP-CASES BOT-CASES JOIN) splits
;JOIN up so that the top new join has the cases corresponding to
;TOP-CASES, and no more. the bottom join (which is the old JOIN minus
;a few cases) has cases corresponding to BOT-CASES.  A cons of the new
;top and bottom joins is returned.  if either TOP-CASES or BOT-CASES
;is so small a set that the break up operation wouldn't make sense (ie
;(length TOP-CASES)<2 or (length BOT-CASES)<1) then [JOIN ! JOIN] is
;returned.  in this case the original JOIN already has no extranious
;cases.  any ROLEs are just ignored. all DFLOW, CFLOW, and SUBSEGs are
;fixed up.

(def-alistq objtran)
(def-alistq dflows-by-id-obj)

(defun break-up-join (top-cases bot-cases join)
  (let ((tseg join))
    (when (not (or (null (cdr top-cases)) (null bot-cases)))
      (setq tseg (genseg 'join))
      (let* ((tsegcase (& tseg))
             (supseg (?supseg join))
             (supsegcase (& supseg))
             (supsegpat (&* supseg))
             (t-jcase (& join (gencase join)))
             (objtran nil))

 ;create new join
        (+& supsegcase subseg tsegcase)
        (+& tsegcase @seg 'join nil (~code (?& (&* join) @seg)))
        (+& tsegcase plantype 'join)
        (+& tsegcase segtype 'join)
        (+& supsegcase cflow tsegcase t-jcase)

;set up objtrn and the inter-join dflow
        (over (??& (&* join) output) -> output (~obj output) -> obj
              (car (??& output @io)) -> @io
              (+& tsegcase output nil (genobj tseg obj) nil) -> new-from
              (+& t-jcase input 'id (genobj join obj) obj) -> new-to
          (setf (objtran obj) (~obj new-from))
          (when @io
            (assert-fact (&~deep @io new-from))
            (assert-fact (&~deep @io new-to)))
          (+& supsegcase dflow new-from new-to))

;move dflow up to top join.
        (over (??& supsegpat cflow * (&* join)) -> cflow
         select (member (~case (~to cflow)) top-cases)
          (let* ((jscase (~to cflow))
                 (tscase (& tseg (gencase tseg))))
            (make-fact-be-out cflow)
            (assert-fact (&~to cflow tscase))
            (over (--& supsegpat dflow * (&* jscase port)) -> dflow
                  (~to dflow) -> to
                  (~01 (--& to @io)) -> @io
                  (>& tscase input 'id (genobj tseg (~obj to))
                      (objtran (~id to))) -> new-to
              (when @io
                (assert-fact (&~deep @io new-to)))
              (+dflow (~from dflow) new-to)
              (kill-port to))))))
```

```
;get rid of any dflow that doesn't need to be in top join.
       (over (??& supsegpat dflow (&* (&* tseg) output)) -> dflow
             (makeqset
              (~nset ~from
                     (??& supsegpat dflow *
                          (&* (&* tseg) input * *
                              (~obj (~from dflow)))))) -> froms
         select (null (cdr froms))
           (+dflow (car froms) (~to dflow))
           (kill-port (~from dflow))) ;note this kills all id ports too

;get rid of any dflow that doesn't have to be in the lower join.
       (over (??& (&* join) output)
               -> output
             (makeqset
              (~nset ~from
                     (??& supsegpat dflow *
                          (&* (&* join) input * * (~obj output)))))
               -> froms
             (over froms
                   (when (eq (~seg~segcase _) tseg)
                     (setq froms (delq _ froms))
                     (return _)))
              -> tj-from
         select (and tj-from (equal froms [output]))
           (^dflow-from output tj-from)
           (kill-port output)))) ;this kills id ports too

   (format-trace *SEGTRN-Debug* "~S ^X ~S"
                 '(,tseg . ,join)
                 '(break-up-join ,top-cases ,bot-cases ,join))
   [tseg ! join]))
```

```
;this is the inverse of BREAK-UP-JOIN it fuses two joins together into one.
;the resulting join (which is the old j with the cases from upj
;added in) is returned.  there must be a direct cflow from upj to j.

(defun fuse-joins (upj j)
  (let* ((upjpat (&* upj))
         (jcase (& j))
         (jpat (&* j))
         (supseg (?supseg upj))
         (supsegpat (&* supseg))
         (upj-jcase (~to (-& supsegpat cflow upjpat jpat)))
         objtran)

  ;set up for the ports that are in both joins
    (over (??& supsegpat dflow (&* upjpat port) (&* jpat port))
      (setf (objtran (~obj (~from _))) (~id (~to _)))
      (kill-port (~to _)))

;set up dflow for the ports that are in the upper join and not
;in the lower.
    (over (??& upjpat output) -> out
      select (null (objtran (~obj out)))
      (let* ((@io (~01 (??& out @io)))
             (new-from (>& jcase output nil (genobj j (~obj out)) nil)))
        (when @io
          (assert-fact (&~deep @io new-from)))
        (setf (objtran (~obj out)) (~obj new-from))
        (^dflow-from out new-from)
        (over (??& supsegpat cflow * jpat) -> cf
              (+& (~to cf) input 'id
                  (genobj j (~obj new-from)) (~obj new-from)) -> new-to
          (when @io
            (assert-fact (&~deep @io new-to)))
          (+dflow new-from new-to))))

  ;set up ports that are in the lower join and not in the higher one.
    (over (??& supsegpat dflow * (&* upj-jcase port)) -> dflow
          (~01 (??& (~to dflow) @io)) -> @io
          (genobj upj (~obj (~to dflow))) -> new-obj
      (setf (objtran new-obj) (~id (~to dflow)))
      (over (??& supsegpat cflow * upjpat) -> cf
            (+& (~to cf) input 'id (genobj upj new-obj) new-obj) -> new-to
        (when @io
          (assert-fact (&~deep @io new-to)))
        (assert-fact (&~to dflow new-to)))
      (kill-port (~to dflow)))

  ;move all of the dflow down from the upper to the lower.
    (over (--& supsegpat cflow * upjpat) -> cflow
          (~to cflow) -> j1case
          (& j (gencase j)) -> j2case
      (assert-fact (&~to cflow j2case))
      (over (--& supsegpat dflow * (&* j1case port)) -> dflow
            (~to dflow) -> to
            (~01 (??& to @io)) -> @io
            (+& j2case input 'id
                (genobj j (~obj to)) (objtran (~id to))) -> new-to
        (when @io
          (assert-fact (&~deep @io new-to)))
        (+dflow (~from dflow) new-to)))

    (kill-seg upj))
  (format-trace *SEGTRN-Debug* "~S ^X ~S"
                j '(fuse-joins ,upj ,j))
  j)
```

```
;the following are some smaller scall db transformations that seem
;to be usefull.  (^stands for move)


;this takes in a port and a seg, and asserts a copy of the port,
;which the seg changed.  It calls genobj
;so that that gets set up too.  it returns the new port.
;note that it assumes that there is not already a port of
;the same name on the new seg, and yells if this ain't true.
;Something better is going to have to happen with this I think!!
;it only works for non-temporal ports!

(defun =port-seg (port newseg)
  (if (not (eq (~obj port) (genobj newseg (~obj port))))
      (barf "=PORT-SEG: Copied port clashes.~
             ~% Port: ~S~
             ~% Newseg: ~S"
              port newseg))
  (assert-fact (&~seg~segcase port newseg)))

;this takes a new and old port, and makes every dflow from
;the old port go from the new one instead.

(defun ^dflow-from (old-port new-port)
  (over (--& * dflow old-port)
    (+dflow new-port (~to _))))

;this is the same with regard to the to.

(defun ^dflow-to (old-port new-port)
  (over (--& * dflow * old-port)
    (+dflow (~from _) new-port)))

;these are similar, but non-destructive.

(defun =dflow-from (old-port new-port)
  (over (??& * dflow old-port)
    (+dflow new-port (~to _))))

;this is the same with regard to the to.

(defun =dflow-to (old-port new-port)
  (over (??& * dflow old-port)
    (+dflow (~from _) new-port)))


;this splices a seg into a cflow, note that it works whether this
;is a new seg boundry around the destination or source, or
;whether it is just a new seg fitting in the cflow

(defun cflow-splice-in (cf segcase)
  (make-fact-be-out cf)
  (+cflow (~from cf) segcase)
  (+cflow segcase (~to cf)))

;similar but splices out a seg. (only works for SIMPLE segs)

(defun cflow-splice-out (seg)
  (+cflow (~from (-& * cflow * (& seg))) (~to (-& * cflow (& seg)))))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; All internal fns in this file have the prefix "3" to
;;; avoid name conflicts.

(declare (special segfacts psegs nsegs 3affected
                  segtypes FLOWS-HELPS-ANALYZE))

(define-trace-option *ANALYZE-Debug*
                     "Show trace of ANALYZE internals"
  nil :boolean)

;;; There was a comment here that said:
;;;
;;;   All the @ data base statments are ignored totally.
;;;
;;; What does that mean? What was the scope of the statement? Is it
;;; really still true? --kmp  yes its true --DICK

(defmacro 3n1 (seg) '(first  (3nsegs ,seg)))
(defmacro 3n2 (seg) '(second (3nsegs ,seg)))
(defmacro 3p1 (seg) '(first  (3psegs ,seg)))
(defmacro 3p2 (seg) '(second (3psegs ,seg)))

;_____________________________________________________________________

(defmacro 3joinp (seg)
  '(eq (3segtype ,seg) 'join))

(defun 3pred2p (seg)
  (and (eq (3segtype seg) 'split) (null (cddr (3nsegs seg)))))

(defmacro 3simplep (seg)
  '(eq (3segtype ,seg) 'simple))

;_____________________________________________________________________

(defun 3segtype (seg)
  (if seg (memoizeq segtypes seg
            (cond ((??& * recursive (&* seg)) 'rec)
                  (t (?segtype seg))))))

;these find and memoize the basic cflows, they find only out-in cflow!!
;and they refuse to find recs (this is how recs don't get recognized as
;part of anything but an ssr.)

(defun 3psegs (seg)
  (memoizeq psegs seg
    (letS* ((cfl (Elist (??& * cflow * (&* seg))))
            (supseg (~seg (~segcase cfl)))
            (fseg (~seg (~from cfl))))
      (Rlist (Fselect fseg (and (not (eq supseg seg))
                                (not (eq supseg fseg))
                                (not (eq (3segtype fseg) 'rec))))))))

(defun 3nsegs (seg)
  (memoizeq nsegs seg
    (letS* ((cfl (Elist (??& * cflow (&* seg))))
            (supseg (~seg (~segcase cfl)))
            (tseg (~seg (~to cfl))))
```

```
(Rlist (fselect tseg (and (not (eq supseg seg))
                          (not (eq supseg tseg))
                          (not (eq (3segtype tseg) 'rec))))))))
```

```
; (we will discuss 3optnext note that 3optprev is symetrical) the
;expressions FIRST and NEXT must both evaluate to a segment. they are
;each evaluated at most once, but their order of evaluation is not
;guarenteed.  if NEXT is not simple, it is returned and VAR is set to
;INIT. if NEXT is simple then it successor is returned, and VAR is set
;to NEXT.  note two things.  VAR is always the predecessor of the
;segment which is returned (no matter what); and VAR holds the
;optionally appearing simple segment if it appears.  these are
;included in order to make it easier to deal with optional segments.

(defmacro 3optnext (var first next)
  '(progn
     (setq ,var ,next)
     (cond ((3simplep ,var)
            (3n1 ,var))
           (t (prog2 nil ,var (setq ,var ,first))))))

(defmacro 3optprev (var first prev)
  '(progn
     (setq ,var ,prev)
     (cond ((3simplep ,var)
            (3p1 ,var))
           (t (prog2 nil ,var (setq ,var ,first))))))

;_____________________________________________________________________


(defun 3get-top-j (join psegs)
  (let* (top-cases bot-cases)
    (letS* ((cfl (Elist (??& * cflow * (&* join)))))
      (cond ((memq (~seg (~from cfl)) psegs)
             (push (~case (~to cfl)) top-cases))
            (t (push (~case (~to cfl)) bot-cases))))
    (car (break-up-join top-cases bot-cases join))))

(defun 3get-bot-j (join psegs)
  (let* (top-cases bot-cases)
    (letS* ((cfl (Elist (??& * cflow * (&* join)))))
      (cond ((memq (~seg (~from cfl)) psegs)
             (push (~case (~to cfl)) bot-cases))
            (t (push (~case (~to cfl)) top-cases))))
    (cdr (break-up-join top-cases bot-cases join))))

;this gives you a list of all the ssrs in a plan, most of the actions below
;happen on an ssr by ssr basis, for speed, and so they can be called on
;only one bit if needed.

(declare (special 3SSRS))

(defun 3ssrs (seg)
  (let (3SSRS) (3ssrs1 seg) 3SSRS))

(defun 3ssrs1 (seg)
  (if (eq (?plantype seg) 'ssr) (push seg 3SSRS))
  (mapc #'3ssrs1 (??subseg seg)))
```

```
;this is the main entry point, note the special variable
;FLOWS-HELPS-ANALYZE.

;Special variables are used to control specific aspects of the
;analysis process.

(defvar NO-GROUP-IN-TC    nil
  "prevents the intermediate grouping of ssrs in a TC")
(defvar NO-DFLOW-MOVEMENT nil
  "prevents movement of segments based on data flow")

(defun analyze-for-recognition (segcase)
  (let ((NO-GROUP-IN-TC T) (NO-DFLOW-MOVEMENT T))
    (analyze segcase)))

(defun analyze (supsegcase)
  (cond ((??subseg (~seg supsegcase))
         (db-context-was <ANAL)
         (let ((supseg (~seg supsegcase))
               segtypes psegs nsegs segfacts)
           ;;
           (letS* ((seg (Elist (??subseg supseg))))
             (cond ((and (null (?plantype seg)) (??subseg seg))
                    (format-trace *ANALYZE-Debug*
                                  "~%Recurring to analyze ~A"
                                  seg)
                    (analyze (& seg)))))
           ;;
           (format-trace *ANALYZE-Debug* "~2%Initial bottom-up grouping")
           ;;
           (if (and (boundp 'FLOWS-HELPS-ANALYZE)
                    (eq (car FLOWS-HELPS-ANALYZE) supseg))
               (letS* ((segcase (Elist (cdr FLOWS-HELPS-ANALYZE)))
                       (cflow (if ;if this is really there
                                  (memq (~seg segcase) *db-segs*)
                                  (car (??& supsegcase cflow segcase))))
                       (plan-temp (if cflow (3exprp (~seg (~to cflow))))))
                 (cond (plan-temp (funcall (car plan-temp)
                                           (cdr plan-temp))))))
           (3group-up supseg)
           ;;
           (format-trace *ANALYZE-Debug*
             "~%Simplification of structure based on dflow")
           (if (not NO-DFLOW-MOVEMENT) (3dflow-simp supseg))
           ;;
           (format-trace *ANALYZE-Debug* "~%Temporal analysis of SSRs")
           (db-context-is <SSRANAL) (push-db-context) ;for debugging
           (3ssr-simp supseg)
           ;;
           (format-trace *ANALYZE-Debug*
                         "~%Top down simplification of EXPRs")
           (3expr-simp supseg)
           ;;
           (3fix-slot-assertions supseg)
           ;;
           (3fix-infinite-loops supseg)
           ;;
           (db-context-is >ANAL)
           (push-db-context))))
    supsegcase)
```

```
;this fixes up infinite loops by removing the dummy exits that
;are inserted early on in order for things to work (a kludge)
;we are assuming here that if an ssr has an instance of a
;special k*infinte loop-speudo-exit in it then
;it can be discarded for sure.

(defun 3fix-infinite-loops (supseg)
  (dolist (ssr (3ssrs supseg))
    (let ((body (~01 ~seg ~roleseg (??& (&* ssr) role * 'body))))
      (when (lets ((seg (Elist (??subseg body))))
              (Ror-fast
                (??& (&* seg) @seg 'funcall 'k*infinite-loop-pseudo-exit)))
        (kill-seg-and-its-subsegs ssr)))))

;there is a hack here to deal with ssr things.  if one or more
;slot names have the form <name2-of-name1>, then the ssr(s) containing
;them are grouped together as a tc and given the slot name <name1>.
;The individual slots themselves are given the names <name2>
;in the subthing.

(defun 3fix-slot-assertions (supseg)
  (let* ((slot-groups
          (letS* ((slot-fact (Elist (??& (&* supseg) slot)))
                  ((name1 name2) (3slotname-parse (~slotname slot-fact))))
            (cond (name1 (make-fact-be-out slot-fact)))
            (Reqalist (fselect name1 name1)
                      (list (~slotseg slot-fact)
                            name2
                            (~slottype slot-fact))))))
    (letS* (((name1 . prs) (Elist slot-groups))
            (ssrs (letS* () (Reqset (3up-to-ssr (car (Elist prs))))))
            (mainseg (car ssrs)))
      (when (cdr ssrs)
        (if (cdr (letS* () (Reqset (?supseg (Elist ssrs)))))
            (barf "Compound slot names not all in same TC. ~% SSRs: ~S"
                  ssrs))
        (let* ((upper (?supseg (car ssrs)))
               (roles (??& (&* upper) role))
               rolenames)
          (setq mainseg (group2 ssrs name1 nil))
          (+& (& mainseg) plantype 'tempcomp)
          (dolist (role roles)
            (cond ((memq (~seg (~roleseg role)) ssrs)
                   (push (~rolename role) rolenames)
                   (+& (& mainseg) role
                       (~roleseg role) (~rolename role)))))
          (+& (& upper) role (& mainseg)
              (cond ((memq 'term rolenames) 'term)
                    ((memq 'acc rolenames) 'acc)
                    ((memq 'gen rolenames) 'gen)
                    (t 'trn)))))
      (+& (& mainseg) cliche 'pseudo-subcliche)
      (+& (& supseg) slot (& mainseg) name1 'segment)
      (dolist (pr prs)
        (let (((sc name2 type) pr))
          (+& (& mainseg) slot sc name2 type))))))
```

```
(defun 3up-to-ssr (obj)
  (prog (seg)
        (setq seg (cond ((fact? obj) (~seg (~segcase obj)))
                        (t (~seg obj))))
      L (setq seg (?supseg seg))
        (cond ((null seg) (barf "Complex slot name not in SSR."))
              ((eq (?plantype seg) 'ssr) (return seg)))
        (go L)))

;;; (3slotname-parse sym)
;;;
;;;  If sym has the string "-OF-" as a part of its pname,
;;;     returns a list of two things:
;;;      1. the part of the symbol following the -OF-
;;;      2. the part of the symbol preceding the -OF-
;;;     where both symbols are interned on the KBE package.
;;;
;;;  Otherwise it returns NIL

(defun 3slotname-parse (symbol)
  (let ((pname (get-pname symbol)))
    (let ((idx (string-search "-OF-" pname)))
      (cond ((not idx) nil)
            (t
             (list (intern-KBE (substring pname (+ idx 4)    ))
                   (intern-KBE (substring pname 0         idx))))))))
```

```
;this does basic bottom up grouping of a segment.
;it is modified so that it can be called with a seg that already has a
;plantype, in order to do a little extra grouping.  temporal-compose
;calls it that way.

(defun 3group-up (supseg)
  (repeat (segs plantype done (priortype (?plantype supseg)))
    (setq segs     (??subseg supseg)
          segtypes nil
          nsegs    nil
          psegs    nil)
    (setq plantype (or (if (and (null priortype) (null (cdr segs)))
                           ['3exprize [(car segs) 'action]])
                       (Ror-fast (3exprp (Elist segs)))
                       (Ror-fast (3simplepredp (Elist segs)))
                       (Ror-fast (3complexpredp (Elist segs)))
                       (Ror-fast (3bigloopp (Elist segs)))
                       (3ssrp supseg segs))
          done (= (length segs) (length (cdr plantype))))
    (if (and (null plantype) (null priortype))
        (barf "3GROUP-UP: Did not completely analyze.~% supseg: ~S"
              supseg))
    (if (or (null plantype) (and done priortype)) (return))
    (funcall (car plantype) (cdr plantype))
    (format-trace *ANALYZE-Debug* "~%")
    (if done (return))))
```

```
;this takes a segment and checks to see whether a compound predicate,
;simple xor or loop can be constructed starting with the given seg.
;It returns either nil if no match is possible or a cons of the
;constructor fn and an alist of the segments which will form the body
;of the new segment and thier roles.  what constructor fn is
;appropriate depends on what plan type is applicable.

(defun 3simplepredp (p)
  (if (3pred2p p)
      (let (a b a1 a2 j p2)
        (cond ((and (3simplep (setq a (3p1 p)))
                    (let ((dflow (??& * dflow (&* (&* a) output))))
                      (and dflow (Rand-fast (eq (~seg~segcase
                                                  (~to (Elist dflow))) p)))))
               ['3predize-i ! [[a 'init][p 'pred]]])
              ((and (3pred2p (setq p2 (3p1 p)))
                    (setq j (cond ((eq (3n1 p2) p) (3n2 p2))
                                  (t (3n1 p2))))
                    (3joinp j) ;is a join
                    (or (eq j (3n1 p)) (eq j (3n2 p)))
                    (set= (letS* ((dflowF
                                   (Elist
                                    (??& * dflow *
                                         (&* (~to (?& * cflow (&* p) (&* j)))
                                             input)))))
                                 (Rlist (cons (~from dflowF) (~id (~to dflowF)))))
                          (letS* ((dflowF
                                   (Elist
                                     (??& * dflow *
                                          (&* (~to (?& * cflow (&* p2) (&* j)))
                                              input)))))
                                 (Rlist (cons (~from dflowF) (~id (~to dflowF))))))))
               ['3predize ! [[p 'pred] [p2 'pred]
                                       [(3get-top-j j [p p2]) 'join]]])
              ((and (3joinp (setq j (3optnext a1 p (3n1 p))))
                    (eq j (3optnext a2 p (3n2 p))))
               ['3xorize ! [[p 'pred]
                            !(if (not (eq a1 p)) [[a1 'action]])
                            !(if (not (eq a2 p)) [[a2 'action]])
                            [(3get-top-j j [a1 a2]) 'join]]])
              ((and (3joinp (setq j (3optprev b p (3p1 p))))
                    (or (eq j (3optnext a p (3n1 p)))
                        (eq j (3optnext a p (3n2 p)))))
               ['3loop-ssrize ! [[p 'l-pred]
                                 !(if (not (eq a p)) [[a 'l-body]])
                                 !(if (not (eq b p)) [[b 'l-body]])
                                 [(3get-bot-j j [a]) 'l-join]]])))))
;_____________________________________________________________________

;this checks whether a segment can be the start of an expr

(defun 3exprp (seg)
  (if (and (3simplep seg)
           (or (null (3psegs seg))
               (not (3simplep (3p1 seg))))
           (3n1 seg)
           (3simplep (3n1 seg)))
      ['3exprize [seg 'action]
       ! (Rlist (list (enumerateS #'(lambda (ns)
                                      (or (null ns) (not (3simplep ns))))
                                  #'3n1
                                  (3n1 seg))
                      'action))]))
```

```
;this works on commutable preds and two exit loops.

(defun 3complexpredp (p)
  (if (3pred2p p)
      (let (a b c j p2)
        (cond ((and (3pred2p (setq p2 (3p1 p)))
                    (3joinp (cond ((eq (3n1 p2) p)
                                   (setq j (3optnext b p2 (3n2 p2))))
                                  (t
                                   (setq j (3optnext b p2 (3n1 p2))))))
                    (or (eq j (3optnext a p (3n1 p)))
                        (eq j (3optnext a p (3n2 p))))
                    (3commutable-preds p p2 (cond ((eq a p) j)
                                                  (t a))))
               ['3commute-preds ! [[p 'pred]
                                   [p2 'pred]
                                   !(if (not (eq a p)) [[a 'action]])
                                   [(3get-top-j j [a b]) 'join]
                                   !(if (not (eq b p2)) [[b 'action]])]])
              ((and (3pred2p (setq p2 (3optprev b p (3p1 p))))
                    (3joinp (setq j (3optprev c p2 (3p1 p2))))
                    (or (eq j (3optnext a p (3n1 p)))
                        (eq j (3optnext a p (3n2 p)))))
               ['3loop-ssrize ! [[p 'l-pred]
                                 [p2 'l-pred]
                                 !(if (not (eq a p)) [[a 'l-body]])
                                 !(if (not (eq b p)) [[b 'l-body]])
                                 !(if (not (eq c p2)) [[c 'l-body]])
                                 [(3get-bot-j j [a]) 'l-join]]])))))

(defun 3bigloopp (j)
  (let (a b p frontq endq)
    (cond ((and (3joinp j)
                (3pred2p (setq p (3optnext a j (3n1 j)))))
           (setq frontq [p]
                 endq   frontq)
           (if (repeat ()
                 (if (null frontq) (return nil))
                 (setq p (3optnext a (car frontq) (3n1 (car frontq))))
                 (if (eq p j) (return t))
                 (if (3pred2p p) (setq endq (cdr (rplacd endq [p]))))
                 (setq p (3optnext a (car frontq) (3n2 (car frontq))))
                 (if (eq p j) (return t))
                 (if (3pred2p p) (setq endq (cdr (rplacd endq [p]))))
                 (setq frontq (cdr frontq)))
               ['3loop-ssrize ! [[(3get-bot-j j [a]) 'l-join]
                                 !(if (not (eq a (top frontq))) [[a 'l-body]])
                                 [(top frontq) 'l-pred]
                                 !(letS* ((p2 (enumerateS
                                                #'(lambda (p) (eq p j))
                                                #'(lambda (p)
                                                    (3optprev b nil (3p1 p)))
                                                (3optprev b nil
                                                          (3p1 (top frontq))))))
                                    (nconc (Rnconc [!(if b [[b 'l-body]])
                                                    [p2 'l-pred]])
                                           (if b [[b 'l-body]])))]])))))
```

```
;this recognizes that a seg is an ssr because it already has a rec
;[eg it wasn't originally coded as a loop, but was really recursive.]
;the only effort which then has to be done is to make sure the right roles
;get set up.  [note that there is a bit of effort in the other parts of
;this file in order to prevent recs from getting recognized as part of
;other things.]

(defun 3ssrp (supseg segs)
  (let* ((recs (~nset ~seg~subseg (??& (&* supseg) recursive))))
    (if (and recs (qset< recs segs))
        ['3recursive-ssrize
           ![!(Rbag (list (Elist recs) 'rec))
             !(Rbag (list (filterS #'(lambda (s) (not (memq s recs)))
                                   (Elist segs))
                          'other))]])))

(defun 3recursive-ssrize (segs)
  (barf "analysis of recursive programs not currently supported")
;this does not set up the correspondance between the inputs of the
;body and the rec right!
  (when (eq (cadadr segs) 'rec)
    (barf "3RECURSIVE-SSRIZE: Cannot handle multiple recursion.~
          ~% Segs: ~S"
          segs))
  (let* ((rec (caar segs))
         (body (group2 (mapcar #'car segs) 'tcbody 'always))
         (ssr (?supseg body))
         (dj (genseg 'dj)))
;make the rec be linked to the body instead of the ssr.
    (assert-fact (&~segcase (-& * recursive (&* rec)) (& body)))
;set up the dfjoin
    (+& (& body) subseg (& dj))
    (+& (& dj) segtype 'join)
    (+& (& dj) @seg 'join nil nil)
    (dolist (df (--& (&* body) dflow * (&* (&* body) output)))
      (let* ((from (~from df))
             (sourcedf (--& (&* body) dflow *
                            (&* (&* (~seg~segcase from)) ;the original join
                                input 'id * (~obj from))))
             (obj (~obj from))
             (nout (+& (& dj) output nil (genobj dj obj) nil)))
        (--& from @io) (--& from @createvar)
        (make-fact-be-out from)
        (+& (& body) dflow nout (~to df))
        (dolist (sdf sourcedf)
          (--& (~to sdf) @io) (--& (~to sdf) @createvar)
          (make-fact-be-out (~to sdf))
          (+& (& body) dflow (~from sdf)
              (+& (& dj (cond ((eq (~seg~segcase (~from sdf)) rec) 'rec)
                              (t 'body)))
                  input 'id (genobj dj obj) (~obj nout))))))
;set up plantype and roles
    (+& (& ssr) plantype 'ssr)
    (+& (& ssr) role (& body) 'body)
    (+& (& ssr) role (& dj) 'dfjoin)
    (+& (& ssr) role (& rec) 'rec)
    (+& (& body) plantype 'ssr-body)))

;there might be something funny here with multiple exits,
;and the kind of joins they are supposed to have.
```

```
(defun 3predize (rsegs)
  (let ((seg (group (Rlist (car (Elist rsegs))) 'pred)))
    (>& (& seg) plantype 'pred)
    (3make-roles rsegs seg)
    seg))

(defun 3predize-i (rsegs)
  (let* ((seg (3predize rsegs))
         (init (caar rsegs))
         (cflowin (car (--& (&* seg) cflow * (&* init)))))
    (if cflowin
        (assert-fact (&~to cflowin (~to (-& (&* seg) cflow (&* init))))))
;this is a cludge to get mnemonic output cases in this situation.
;note cludge to avoid renaming clobbering
    (letS* (((cf outside-cf)
             (Elist (letS* ((cflowF (Elist (--& (&* seg) cflow * (&* seg)))))
                      (Rbag (list cflowF (-& * cflow (~to cflowF))))))))
      (>& (& seg) cflow (~from cf) (& seg (~case (~from cf))))
      (>& (~segcase outside-cf) cflow
          (& seg (~case (~from cf))) (~to outside-cf)))
    seg))

(defun 3exprize (rsegs)
  (let ((seg (group (Rlist (car (Elist rsegs))) 'expr)))
    (>& (& seg) plantype 'expr)
    (--& (&* seg) cflow)
    (3make-roles rsegs seg)
    seg))


(defun 3xorize (rsegs)
  (let ((seg (group (Rlist (car (Elist rsegs))) 'xor)))
    (>& (& seg) plantype 'xor)
    (3make-roles rsegs seg)
    (3xor-simp seg)
    seg))

;_____________________________________________________________________

(defun 3make-roles (rsegs seg)
  (dolist (rseg rsegs)
    (let (((dseg name . nil) rseg))
      (>& (& seg) role (& dseg) name))))
;_____________________________________________________________________

;the xor simplification must happen before the dflow movement

(defun 3xor-simp (xor)
  (let* ((pred (?role xor 'pred))
         (join (?role xor 'join))
         (innards (??role xor 'action)))
    (if (null ;leave alone if dflow from pred
         (??& (&* xor) dflow (&* (&* pred) output)))
        (dolist (iseg innards)
          (when (and (eq (?plantype iseg) 'xor) (null (?role iseg 'init)))
            (let* ((ipred (?role iseg 'pred))
                   (ijoin (?role iseg 'join))
                   (iinnards (??role iseg 'action)))
              (when (null ;no good if ipred dflow
                     (??& (&* iseg) dflow (&* (&* ipred) output)))
                (ungroup iseg)
                (dolist (iinnard iinnards)
                  (>& (& xor) role (& iinnard) 'action))
                (setq pred (3predize [[pred 'pred] [ipred 'pred]]))
                (>& (& xor) role (& pred) 'pred)
                (setq join (fuse-joins ijoin join))
                (format-trace *ANALYZE-Debug* "~%")))))))))
```

```
;This program takes a loop which has been located, and converts it
;into a SSR.  it also constructs the plantype and role statements
;which are appropriate.  (the transformation is detailed in an
;illustration in the thesis.)

(def-alistq upport)

(defun 3loop-ssrize (rsegs)
  (let* ((body (group2 (Rlist (car (Elist rsegs))) 'tcbody 'always))
         (bodycase (& body))
         (bodypat (&* body))
         (oldjoin (dolist (rseg rsegs)
                    (let (((seg role) rseg))
                      (if (eq role 'l-join) (return seg)))))
         (oldjoinpat (&* oldjoin))
         (ijcase (~to (-& bodypat cflow bodypat oldjoinpat)))
         (ssr (group [body] 'tc))
         (ssrpat (&* ssr))
         (ssrcase (& ssr))
         (rec (genseg 'rec))
         (reccase (& rec))
         (dj (genseg 'dj))
         (djcase (& dj)))
;get rid of the cflow connections to the old join in the loop.
    (+& bodycase cflow bodycase (~to (-& bodypat cflow oldjoinpat)))
    (+& bodycase cflow (~from (-& bodypat cflow * oldjoinpat)) reccase)
;set up relationship between recurrence and body
    (+& bodycase recursive reccase)
    (+& bodycase subseg reccase)
    (+& reccase segtype (?segtype body))
    (+& reccase @seg 'recursion nil nil)
;set up dfjoin
    (+& bodycase subseg djcase)
    (+& djcase segtype 'join)
    (+& djcase @seg 'join nil nil)
    (dolist (df (--& bodypat dflow * (&* bodypat output)))
      (let* ((obj (~obj (~to df)))
             (nout (+& djcase output nil (genobj dj obj) nil)))
        (+& bodycase dflow (~from df)
            (+& (& dj 'body) input 'id (genobj dj obj) (~obj nout)))
        (+& bodycase dflow
            (+& reccase output nil (genobj rec obj) nil)
            (+& (& dj 'rec) input 'id (genobj dj obj) (~obj nout)))
        (+& bodycase dflow nout (~to df))))
;set up plantype and roles
    (+& ssrcase plantype 'ssr)
    (+& ssrcase role bodycase 'body)
    (+& ssrcase role reccase 'rec)
    (+& ssrcase role djcase 'dfjoin)
    (+& bodycase plantype 'ssr-body)
```

```
;setup appropriate dflow to and from body and rec
(let* ((joined-idfs
        (--& bodypat dflow (&* bodypat input) (&* ijcase input)))
       rec-inputs-used)
;make all of the inputs which go to other than the input case of the
;join available on each cycle of the ssr, and remember which
;ones these are.
      (dolist (df (??& bodypat dflow (&* bodypat input)))
        (let* ((input (~from df)))
          (cond ((not (member input rec-inputs-used))
                 (push input rec-inputs-used)
                 (>& bodycase dflow input
                     (>& reccase input nil (genobj rec (~obj input)) nil)))))))
;now take care of the dflows going to the input part of the old join
      (dolist (df joined-idfs)
        (let* ((dest (~to df))
               (input (~from df)))
          (if ;must duplicate the input to stop trouble.
              (memq input rec-inputs-used)
              (+& ssrcase dflow (~from (?& ssrpat dflow * input))
                  (setq input
                        (+& bodycase input nil
                            (genobj body (~obj input)) nil))))
          (push input rec-inputs-used)
          (dolist (df (--& bodypat dflow (&* oldjoinpat output * (~id dest))))
            (>& bodycase dflow input (~to df)))
          (>& bodycase dflow
              (~from
                (-& bodypat dflow * (&* oldjoinpat input * * (~id dest))))
              (>& reccase input nil (genobj rec (~obj input)) nil)))))
;get rid of everything about the old join
    (kill-seg-and-its-subsegs oldjoin)

;now for each pred in the loop, the cflow has to be rerouted
;through new joins to cases of the body.
    (dolist (rseg rsegs)
      (let (((pred role) rseg))
        (when (eq role 'l-pred)
          (let* ((pbcf (-& bodypat cflow (&* pred) bodypat))
                 (join (genseg 'join)))
            (+& bodycase subseg (& join))
            (+& (& join) segtype 'join)
            (+& (& join) @seg 'join nil nil)
            (+& bodycase cflow (& rec (~case (~to pbcf))) (& join 'rec))
            (+& bodycase cflow (~from pbcf) (& join 'pred))
            (+& bodycase cflow (& join) (~to pbcf)))))))))
```

```
;this performs the transformation depicted on the next page.  the key
;question is whether of not the two preds can just be commuted or not.
;they can if and only if the condition under which p2 goes to p is
;IMPLIED by the condition under which p takes the branch outside.  And
;the input data to the two preds is compatable, here we assume they
;must be the same.  of the 36 ways the predicates _lt _eq and _gt can
;interact, 12 of them satisfy this criterion.  However, 6 of these are
;uninteresting because they correspond to situations where one of the
;three choices can never be taken.  this leaves 6 cases of interrest.
;(the first (_lt no _eq no) is sometimes created by _AIF in FREAD)
;(another situation (_lt no _eq yes) which is not commutable can also
;be produced by _AIF.)

(defun 3commutable-preds (p p2 a)
  (let* ((ppat (&* p))
         (p2pat (&* p2))
         (apat (&* a))
         (supseg (?supseg p))
         (supsegpat (&* supseg))
         (p2pcflow (?& supsegpat cflow p2pat ppat))
         (pacflow (?& supsegpat cflow ppat apat))
         (-pdflows (??& supsegpat dflow * (&* ppat input)))
         (-p2dflows (??& supsegpat dflow * (&* p2pat input))))
    (and (= (length -p2dflows) 1)
         (= (length -pdflows) 1)
         (equal (~from (car -p2dflows)) (~from (car -pdflows)))
         (member [(~01 ~name (??& p2pat @seg))
                  (~case (~from p2pcflow))
                  (~01 ~name (??& ppat @seg))
                  (~case (~from pacflow))]
                 '((_lt no _eq no)
                   (_lt no _gt no)
                   (_gt no _eq no)
                   (_eq no _lt no)
                   (_gt no _lt no)
                   (_eq no _gt no))))))

;_____________________________________________________________________

;this routine actually commutes the order of two predicates.  It takes
;an argument which is a list of segments and role names.  there can be
;from 3 to 5 segments in this list, the order is very important. the
;lower pred is first, the higher pred is second, the segment which
;follows the lower pred must be third, and the seg which follows the
;higher pred must be last.

(defun 3commute-preds (segs)
  (let* ((((p) (p2) (a)) segs)
         (pcase (& p))
         (ppat (&* p))
         (p2case (& p2))
         (p2pat (&* p2))
         (apat (&* a))
         (supseg (?supseg p))
         (supsegpat (&* supseg))
         (p2pcflow (-& supsegpat cflow p2pat ppat))
         (pacflow (-& supsegpat cflow ppat apat))
         (-p2cflow (-& supsegpat cflow * p2pat)))
    (pp-trace *ANALYZE-Debug* [['3commute-preds ! segs]])
    (assert-fact (&~to p2pcflow (~to pacflow)))
    (assert-fact (&~to -p2cflow pcase))
    (assert-fact (&~to pacflow p2case))
    (3xorize (cdr segs))))
```

```
;this has the same argument order as the fn above.

(defun 3unfactor-preds (segs)
  (let* ((((p) (p2) (a)) segs)
         (pcase (& p))
         (ppat (&* p))
         (p2case (& p2))
         (p2pat (&* p2))
         (apat (&* a))
         (b (caar (last segs)))
         (bpat (&* b))
         (supseg (?supseg p))
         (supsegcase (& supseg))
         (supsegpat (&* supseg))
         (p2pcflow (-& supsegpat cflow p2pat ppat))
         (pacflow (-& supsegpat cflow ppat apat))
         (p2bcflow (?& supsegpat cflow p2pat bpat))
         (-p2cflow (-& supsegpat cflow * p2pat))
         (p2c (copyseg p2 p2))
         (p2cacase (& p2c (~case (~from p2pcflow))))
         (p2cbcase (& p2c (~case (~from p2bcflow))))
         (p2ccase (& p2c))
         (j (genseg 'join))
         (jcase (& j)))
    (pp-trace *ANALYZE-Debug* [['3unfactor-preds ! segs]])
    (+& supsegcase subseg p2ccase)
    (+& supsegcase subseg jcase)
    (+& jcase segtype 'join)
    (+& jcase @seg 'join nil nil)
    (+& jcase plantype 'join)
    (assert-fact (&~to p2pcflow (~to pacflow)))
    (assert-fact (&~to -p2cflow p2ccase))
    (+& supsegcase cflow p2cacase pcase)
    (+& supsegcase cflow p2cbcase (& j p2c))
    (assert-fact (&~to pacflow (& j p)))
    (+& supsegcase cflow jcase p2case)
    (3predize [[p2c 'pred][p 'pred][j 'join]])
    (3xorize (cdr segs))))
```

```
;   this little fn looks at a segment and all its decendents trying to
;factor the dflow forward.  it works from the topmost subsegs downward.
;it returns nil if no change instigated by it propogated up to it.
;otherwise it returns the seg which is the highest level that must be
;backed up to in order to continue the simplification.
;   in the first part (the computation of high) it looks at each subseg in
;turn and tests whether it can be dflow moved.  If it finds one which can
;be, it moves it then stops looking at the subsegs.  Instead, it returns
;the name of the highest seg which could
;have been modified by the move.
;   If no subsegs are moved, then in the second part, it calls itself
;recursively on each subseg.  if a subseg is modified, then if that seg is
;the highest modified point, it just calls itself again to try for further
;moves. (note the memory variable segfacts is reset).  if the modification
;reached to a higher level, then the problem is kicked upstares.  this
;results in the stack unwinding up to the appropriate point to begin
;another pass.
;   The complexity of this fn stems mostly from the fact that the control
;stack is used to encode a rather complex order of simplyfication of segs
;which is dynamically modified as the situation warents.  this complexity
;was introduced because this simplification runs around the program an
;awful lot.  It takes advantage of the fact that nothing can ever be moved
;between the inside and the outside of an ssr-body in order to break up
;its work.

(defun 3dflow-simp (seg)
  (let* (SEGFACTS)
    (3dflow-simp1 nil seg)
    (dolist (ssr (3ssrs seg))
      (3dflow-simp1 nil (?role ssr 'body)))))

(defun 3dflow-simp1 (supseg seg)
  (repeat ()
    (if (*catch seg (3dflow-simp2 seg)) (return))
    (when (null (?segtype seg)) ;ie got ungrouped by the move
      (if (null supseg)
          (barf "3DFLOW-SIMP1: Top seg got ungrouped.~% Seg: ~S" seg))
      (*throw supseg nil)))) ;go up to supseg - else just look again.

(defun 3dflow-simp2 (seg)
  (let ((subsegs (??subseg seg)))
    (dolist (dseg subsegs)
      (let* ((cpath (3check-path dseg (3suggest-dmove-path dseg))))
        (when cpath
          (3do-dflow-move dseg cpath)
          (setq segfacts nil)
          (*throw (3high cpath seg) nil))))
    (dolist (dseg subsegs)
      (when (and (??subseg dseg) (not (eq (?plantype dseg) 'ssr-body)))
        (3dflow-simp1 seg dseg)))
    t))

(defun 3high (cpath init-high)
  (repeat ((i 0) (high init-high) (rpath cpath) dir seg)
    (if (null rpath) (return high))
    (desetq (dir seg . rpath) rpath)
    (cond ((eq dir 'out)
           (cond ((zerop i) (setq high seg))
                 (t (incf i))))
          (t (decf i)))))
```

```
(defun 3check-path (seg path)
  (cond ((not (eq (third (3segfacts seg)) 'simple)) nil)
        (t (3check-step seg nil (car path) (cdr path)))))

;this takes as arguments, the seg that is moving, the reverse of the path
;up to this point, the seg which marks this point in the path, and the
;rest of the path after this point.  first the rest of the path is
;checked, and then the current portion is checked.

(defun 3check-step (seg pre-path oseg path-info)
  (let* ((path path-info)
         ((dir nseg . rest-path) path))
    (if path
        (let* ((npath (3check-step seg
                                   [dir oseg ! pre-path] nseg rest-path))
               ((oterm otype ostype) (3segfacts oseg))
               ((nterm ntype nstype) (3segfacts nseg)))
          (prog () (go start)
           error (barf "3CHECK-STEP: Error.~% Data: ~S" [oseg ! path])
           not-ok (return nil)
           ok-if-npath (if (null npath) (return nil))
           ok    (return [dir nseg ! npath])
           start (cond ((eq dir 'OUT)
                        (cond ((memq otype '(PRED XOR SSR AUG FILTER
                                             TERM TEMPCOMP SSR-BODY))
                               (go not-ok))
                              ((memq otype '(EXPR COMP AND))
                               (go ok-if-npath))))
                       ((not nterm) ;and (eq dir 'IN)
                        (cond ((memq ntype '(SSR AUG FILTER TERM TEMPCOMP))
                               (go ok))
                              ((memq ntype '(PRED XOR))
                               (cond ((3side-effects seg) (go not-ok))
                                     (T (go ok-if-npath)))) ;no xor init
                              ((eq ntype 'SSR-BODY)
                               (go not-ok))
                              ((memq ntype '(EXPR COMP AND))
                               (cond ((memq otype '(EXPR COMP AND))
                                      (go ok-if-npath))
                                     (t
                                      (go ok))))))
                       ((eq ntype 'REC) ;and (not (null nterm))
                        (go not-ok))
                       ((eq ntype 'JOIN)
                        (cond ((and (eq otype 'XOR)
                                    (not (??& (&* oseg) role
                                              (&* (cond (pre-path
                                                         (cadr pre-path))
                                                        (t seg)))
                                              'ACTION)))
                               (go ok))
                              (t (go not-ok))))
                       ((eq ntype 'PRED)
                        (cond ((not (eq otype 'PRED)) (go ok))
                              ((cdr (??role oseg 'PRED)) (go ok))
                              (t (go not-ok))))
                       ((eq ntype 'EXPR)
                        (cond ((memq otype '(EXPR COMP AND))
                               (go not-ok))
                              (t (go ok)))))
           (go error))))))
```

```
(defun 3side-effects (seg)
  (Ror-fast (??& (&* (& (Elist (all-subsegs seg))) output)
                 @io 'side-effect)))

(defun 3segfacts (seg)
  (memoizeq segfacts seg
    (let* ((terminal (null (??subseg seg)))
           (type (?plantype seg))
           (stype (cond ((??& * recursive (&* seg)) 'rec)
                        (t (?segtype seg)))))
      (if terminal (setq type (cdr (assq stype '((split . pred)
                                                 (simple . expr)
                                                 (join . join)
                                                 (rec . rec))))))
      (if (or (null type) (null stype))
          (barf "3SEGFACTS: Error.~% Data: ~S" seg))
      [terminal type stype])))


;this routine generates a listing of the common portion of the dflow
;paths out of a segment each step on the list is a seg and a
;direction.  for example a result of ((s1 output) (s2 output) (s3
;input) (s4 input)) means that all of the output from the segment goes
;out of s1 , then out of s2 then into s3 and finally into s4 at which
;point the dflow paths either diverge, or all terminate.

(defun 3common-dflow-path (segcase)
  (if (3is-mobile (~seg segcase))
      (letS* ((ports (enumerateS
                       #'(lambda (ps)
                           (or (null ps)
                               (progn (setq segcase (~segcase (car ps)))
                                      (Ror-fast
                                        (not (equal (~segcase
                                                      (Elist (cdr ps)))
                                                    segcase))))))
                       #'(lambda (ps)
                           (Rnconc (~nset ~to (??& * dflow (Elist ps)))))
                       (~nset ~to (??& * dflow (&* (&* (~seg segcase))
                                                   output))))))
        (Rlist (list (~seg segcase) (~keyword (car ports)))))))


;certain kinds of segs cannot be moved.  In particular roles that don't
;have any dflow from them to say were they should really be.
;Also probably some kind of side-effect guys cannot move either,
;but we aren't dealing with that now.

(defun 3is-mobile (seg)
  (cond ((and (??& * slot (&* seg))
              (null (??& * dflow (&* (&* seg) output))))
         nil)
        (T (Rand-fast (3is-mobile (Elist (??subseg seg)))))))

;this routine transforms the common dflow path above into a segment
;movement path this path would be (s1 out s2 out s' in s3 in s4) in
;the above example.  s' is the superseg of s2. this path is a list of
;the series of supersegs of seg as it is moved and indications of the
;directions to be taken.  if the dflow path is nil then the movement
;path will be (sup) where sup is the superior of seg.

(defun 3suggest-dmove-path (seg)
  (cdr (over [[tseg code] ! cdp [nseg ncode]]
             <- [[seg 'output] ! (3common-dflow-path (& seg))]
             by cdp until (null tseg)
             nconc
             (cond ((eq code 'output)
```

```
(cond ((eq ncode 'output) ['out nseg])
      (t (let ((sup (?supseg tseg)))
           (cond (sup ['out sup])
                 (t (return (nreverse over-acc))))))))
(t ['in tseg])))))
```

```
(defun 3do-dflow-move (seg path)
  (when (not (null path))
    (letS* (((dir nseg . rpath) (generateS #'cddr path)))
      (cond (rpath
             (cond ((eq dir 'OUT) (move-seg-up seg))
                   (t (move-seg-down nseg seg))))
            (t
             (cond ((eq dir 'OUT)
                    (move-seg-up seg)
                    (let ((sseg (?supseg seg)))
                      (>& (& sseg) role (& seg)
                          (cond ((memq (?plantype sseg) '(EXPR COMP AND))
                                 'ACTION)
                                (t 'INIT)))))
                   (t
                    (let (((nterm ntype nstype) (3segfacts nseg)))
                      (cond (nterm
                             (cond ((eq ntype 'JOIN)
                                    (let* ((nsegcase
                                            (~segcase
                                             (~to (a?& * dflow
                                                       (&* (&* seg) output)
                                                       (&* (&* nseg) input)))))
                                           (nsegcase-cf
                                            (?& * cflow * nsegcase))
                                           (action (~seg (~from nsegcase-cf)))
                                           ((aterm atype) (3segfacts action)))
                                      (if (not
                                           (??& * role (&* action) 'action))
                                          (setq action nil))
                                      (cond (action
                                             (cond ((and
                                                     (not aterm)
                                                     (memq atype
                                                           '(EXPR COMP AND)))
                                                    (move-seg-down action seg)
                                                    (>& (& (?supseg seg)) role
                                                        (& seg)
                                                        'ACTION))
                                                   (t
                                                    (>&
                                                     (& (~seg~segcase
                                                         nsegcase-cf))
                                                     role
                                                     (& (3exprize
                                                         [[seg 'action]
                                                          [action 'action]]))
                                                     'action))))
                                            (t
                                             (--& * role (&* seg))
                                             (>& (& (~seg~segcase nsegcase-cf))
                                                 role (& seg) 'action)
                                             (make-fact-be-out nsegcase-cf)
                                             (assert-fact (&~from nsegcase-cf
                                                                  (& seg)))
                                             (assert-fact (&~to nsegcase-cf
                                                                (& seg)))))))
                                   (t (let ((role (car (--& * role (&* nseg))))
                                            (newseg
                                             (cond ((eq ntype 'pred)
                                                    (3predize-i
                                                     [[seg 'init]
                                                      [nseg 'pred]]))
                                                   ((eq ntype 'expr)
                                                    (3exprize
                                                     [[seg 'action]
                                                      [nseg 'action]]))
                                                   (t
```

```
                                        (barf "3DO-DFLOW-MOVE: ~
                                                  Error 1.~
                                                ~% Seg: ~S~
                                                ~% Path: ~S"
                                                 seg path)))))
                                    (if role
                                        (assert-fact
                                         (&~roleseg role (& newseg))))))))
                                (t ;(not nterm)
                                 (cond ((memq ntype '(EXPR COMP AND))
                                        (move-seg-down nseg seg)
                                        (>& (& (?supseg seg))
                                            role (& seg) 'action))
                                       ((memq ntype '(PRED XOR SSR AUG TERM
                                                      FILTER TEMPCOMP))
                                        (let* ((init (car (--role nseg 'init))))
                                          (move-seg-down nseg seg)
                                          (if init
                                              (setq seg (3exprize
                                                         [[seg 'action]
                                                          [init 'action]])))
                                          (>& (& (?supseg seg))
                                              role (& seg) 'init)))
                                       (t (barf "3DO-DFLOW-MOVE: Error 2.~
                                                 ~% Seg: ~S~% Path: ~S"
                                                  seg path))))))))
                (done))))
  (format-trace *ANALYZE-Debug* "~%"))
nil)
```

```
;this works bottom up simplifying and analyzing exprs.  if an expr has
;an expr as supseg, then the lower expr is ungrouped.  if an expr has
;a non expr superseg then the expr is analyzed to see if it can have
;an AND at the top level.  for the convenience of modular calling, it works
;on an ssr by ssr basis.

;NOTE THAT THE PROCESSING OF EXPRS TO SELECT ANDs HAS BEEN TEMPORARILY
;COMMENTED OUT.  IT DOESN'T SEEM TO MAKE ANYTHING WORK BETTER AT THE
;CURRENT TIME, AND MAKES THINGS A BIT MORE CONFUSING IN THE PLAN EDITOR.
;SEE THE NEXT PAGE.

(defun 3expr-simp (seg)
  (mapc #'3expr-simp1 (qsetU [seg] (3ssrs seg))))

(defun 3expr-simp1 (seg)
  (cond ((eq (?plantype seg) 'expr) (3expr-simp-top seg))
        (t (mapc #'3expr-simp2 (??subseg seg)))))

(defun 3expr-simp2 (seg)
  (cond ((eq (?plantype seg) 'ssr) nil)
        ((eq (?plantype seg) 'expr) (3expr-simp-top seg))
        (t (mapc #'3expr-simp2 (??subseg seg)))))

(defun 3expr-simp-top (seg) ;seg must be an expr
  (dolist (s (??subseg seg)) (3expr-simp-ungroup seg s)))

(defun 3expr-simp-ungroup (supseg seg)  ;supseg must be an expr
  (let* ((subsegs (??subseg seg)))
    (cond ((not (eq (?plantype seg) 'expr)) (mapc #'3expr-simp2 subsegs))
          (subsegs
           (dolist (s subsegs) (3expr-simp-ungroup seg s))
           (setq subsegs (??subseg seg)) ;may have changed if ungrouping
           (ungroup seg)
           (dolist (s subsegs) (>& (& supseg) role (& s) 'action))
           (format-trace *ANALYZE-Debug* "~%")))))
```

```
;(defun 3expr-simp1 (seg)
;  (cond ((eq (?plantype seg) 'expr) (3expr-simp-and seg))
;        (t (mapc #'3expr-simp2 (??subseg seg)))))
;
;(defun 3expr-simp2 (seg)
;  (cond ((eq (?plantype seg) 'ssr) nil)
;        ((eq (?plantype seg) 'expr) (3expr-simp-and seg))
;        (t (mapc #'3expr-simp2 (??subseg seg)))))
;
;(defun 3expr-simp-and (seg) ;seg must be an expr
;  (dolist (s (??subseg seg)) (3expr-simp-ungroup seg s))
;  (let ((split (3dflow-split (??subseg seg))))
;    (when (cdr split)
;      (dolist (set split)
;        (cond ((cdr set)
;               (>& (& seg) role (& (3exprize (Rlist (list (Elist set)
;                                                          'action))))
;                   'action))
;              (t (assert-fact (&~rolename (-& (&* seg) role (&* (car set)))
;                                          'action)))))
;      (--& (&* seg) plantype)
;      (>& (& seg) plantype 'and)
;      (format-trace *ANALYZE-Debug* "~%"))))
;
;;this takes a set of segs which form a directed graph connected be
;;dflow arcs, and partitions them into unconnected subgraphs.  this is
;;done by coloring each seg and taking a transitive closure of the
;;dflow arcs in order to propagate the colors to regions.  at the end
;;each region of one color is a disjoint subgraph.  a list of the
;;subgraphs is returned.
;
;(defun 3dflow-split (segs)
;  (let* ((segcolors (letS* ((s (Elist segs)))
;                      (nreverse (Rlist (cons s s))))))
;    (over segs -> seg
;          (over (??& * dflow (&* (&* seg) output) (&* * input))
;                (~seg (~segcase (~to (car _)))) -> nseg
;                (cdr (assq nseg segcolors)) -> color
;            select color
;            makeqset init [(cdr (assq seg segcolors))]
;            color) -> colors
;      (when (cdr colors)
;        (dolist (item segcolors)
;          (when (memq (cdr item) (cdr colors))
;            (setf (cdr item) (car colors))))))
;    (over (over segcolors -> [seg ! key] classifyq [key ! seg])
;      nlist (cdr _))))
```

```
;this function calls 3ssr-analyze on all of the SSRs in a plan

(defun 3ssr-simp (seg)
  (dolist (ssr (3ssrs seg))
    (3ssr-analyze ssr)))


;this part of the file has functions which make it possible to analyze
;an SSR as a TEMP COMP.

  ;; a clasification of the roles of an xor being worked on
(def-alistq xrole-fillers)

  ;; an alist of fsets showing what segs each seg affects
(def-alistq 3AFFECTED)

  ;; shows what tjoin goes with each term pred (each term is
  ;; assumed to be binary for now)
(def-alistq term-join-cor)

  ;; shows what join goes with each filter pred
(def-alistq filter-join-cor)

  ;; shows the actions that are dominated by
  ;; each filter pred
(def-alistq pred-action-cor)

  ;; a mapping of ports to sets of ports to show the
  ;; effect of the relationship between a seg and
  ;; its recursive instances.
(def-alist porttrans)

;this are just special because I had to break up the fn in order for
;the lispm to compile it.

(declare (special terms tjoins body bodypat rec dj tc porttrans
                  3AFFECTED todo non-join-segs segtypes ssr term-join-cor
                  filter-join-cor seg-order pred-action-cor))

;note as of 1984 we are doing this a new way where we don't separate
;terms from generators.  As soon as we get to the point where we would
;have taken out a terminator we just leave everything as a bssr.
;the following variable can switch this feature on and off.

(defvar *separate-gens-and-terms* nil)

(defun 3ssr-analyze (ssr)
  (let* (terms                    ;a set of the termination preds
         tjoins          ;the termination joins
         (body (?role ssr 'body))
         (bodypat (&* body))
         (rec (?role ssr 'rec))
         (dj (?role ssr 'dfjoin))
         tc porttrans 3AFFECTED todo non-join-segs segtypes
         term-join-cor filter-join-cor seg-order pred-action-cor)
    (format-trace *ANALYZE-Debug* "~%")
    (pp-trace *ANALYZE-Debug* ['3ssr-analyze ssr])
    (3ssra-build-tc)
    (3ssra-build-porttrans)
    (3ssra-classify-subsegs)
    (3ssra-init-affects)
    (3ssra-ungroup)
    ;force it to copy
    (setq non-join-segs (makefset [!! todo !! terms !! nil]))
    (3ssra-compute-seg-order)
    (3ssra-build-affected)
    (pp-trace *ANALYZE-Debug* ['3AFFECTED-relation ! 3AFFECTED])
```

```
    (repeat ()
      (when (null 3AFFECTED)
        (cond ((not *separate-gens-and-terms*))
              ((and (null (??input ssr))
                    (null (??output ssr))
                    (null (??& (&* ssr) toutput)))
               (kill-seg-and-its-subsegs ssr))
              (t
               (barf "SSR-ANALYZE: Couldn't completely analyze.~
                      ~% TC: ~S"
                     tc)))
        (return))
      (3ssra-remove-fragment))
;distribute the init of the tc into the pieces as much as possible.
    (let ((dummy (genseg 'dummy)))
      (>& (& tc) subseg (& dummy)) ;to prevent collapse of tc
      (>& (& dummy) segtype 'simple) ;needed by 3segfacts
      (3dflow-simp1 nil tc)
      (kill-seg-and-its-subsegs dummy))
    tc))
```

```
;build up the tempcomp seg.  Make a new seg so that we can have
;a temp comp.  This is done inside out in case the ssr is an outermost seg.

(defun 3ssra-build-tc ()
  (let* ((role-facts (??& (&* ssr) role)))
    (setq tc  ssr
          ssr (group2 [body] 'bssr 'always))
    (dolist (role-fact role-facts)
      (cond ((not (eq (~rolename role-fact) 'init))
             (assert-fact (&~segcase role-fact (& ssr))))))
    (>& (& ssr) plantype 'ssr)
    (-& (&* tc) plantype)
    (>& (& tc) plantype 'tempcomp)
    (>& (& tc) role (& ssr) 'bssr)
    (-& (&* tc) cflow (&* tc) (&* ssr))))

;construct the mapping between the ports of the rec and the ports of
;the body

(defun 3ssra-build-porttrans ()
  (dolist (out (??output body))
    (setf (porttrans out)
          (??& (&~seg (~segcase out) rec) output * (~obj out))))
  (dolist (in (??input rec))
    (setf (porttrans in) (??& bodypat input * (~obj in)))))

;classify the subsegs of the body

(defun 3ssra-classify-subsegs ()
  (dolist (seg (??subseg body))
    (when (and (not (eq seg rec)) (not (eq seg dj)))
      (cond ((eq (3segtype seg) 'split) (push seg terms))
            ((eq (3segtype seg) 'join) (push seg tjoins))
            (t (push seg todo)))))
  (setq todo (makefset todo)))

;set up the affects relation and term-join-cor for the termination preds
;note affects just holds the next pred in line for a term.

(defun 3ssra-init-affects ()
  (dolist (term terms)
    (setf (3AFFECTED term) nil)
    (repeat ((waiting [term]) seg)
      (setq seg (when-pop-fails waiting (return)))
      (cond ((and (not (eq seg term)) (memq seg terms))
             (setf (3AFFECTED term) [seg]))
            ((memq seg tjoins)
             (setf (term-join-cor term) seg)
             (setf (term-join-cor seg) term))
            ((and (not (eq seg body)) (not (eq seg rec)))
             (setq waiting
                   (append (Rlist (~seg (~to (Elist (??& bodypat
                                                    cflow (&* seg))))))
                           waiting)))))))
```

```
;compute how much inner ungrouping to do

(defun 3ssra-ungroup ()
  (repeat ((prescan todo) ptype seg subs)
    (setq seg   (when-pop-fails prescan (return))
          ptype (?plantype seg))
    (cond ((memq ptype '(expr and comp))        ;ungroup these for sure
           (setq subs (makefset (??subseg seg))
                 todo (fsetU (delq seg todo) subs))
           (setq prescan (append subs prescan))
           (letS* (((aseg . asegs) (Elist 3AFFECTED)))
             (cond ((memq seg asegs)
                    (setf (3AFFECTED aseg) [!! subs !!(delq seg asegs)])))
           (letS* (((pred . actions) (Elist pred-action-cor)))
             (cond ((memq seg actions)
                    (setf (pred-action-cor pred)
                          [!! subs !! (delq seg actions)])))
           (ungroup seg))
          ((eq ptype 'xor)                      ;check if can ungroup
           (let* ((newptran (letS* ((o (Elist (??output seg))))
                              (Rlist [o !(3dfaffect body [o] porttrans)])))
                  xrole-fillers subs)
             (when (and (letS* ((affects (cdr (Elist newptran))))
                          (Rand-fast
                           (letS* ()
                             (Rand-fast
                              (eq (~seg~segcase (Elist affects)) seg)))))
                        ;xor affects only itself
                        (letS* ((sub (Elist (??role seg 'action))))
                          (Rand-fast
                           (letS* ()
                             (Rand-fast
                              (eq (~seg~segcase
                                   (Elist (3dfaffect seg (??output sub)
                                                     newptran)))
                                  sub))))))
               ;each action affects only itself
               (setq xrole-fillers (??roles seg))
               (setf (3AFFECTED (first (xrole-fillers 'pred)))
                     (xrole-fillers 'action))
               (setq subs (makefset [!!(xrole-fillers 'init)
                                     !!(xrole-fillers 'action)
                                     ! nil]) ;must protect actions!
                     todo (fsetU (fsetU (delq seg todo) subs)
                                 (xrole-fillers 'pred))
                     prescan [!! subs ! prescan])
               (setf (filter-join-cor (first (xrole-fillers 'pred)))
                     (first (xrole-fillers 'join)))
               (setf (filter-join-cor (first (xrole-fillers 'join)))
                     (first (xrole-fillers 'pred)))
               (setf (pred-action-cor (car (xrole-fillers 'pred)))
                     (xrole-fillers 'action))
               (letS* (((aseg . asegs) (Elist 3AFFECTED)))
                 (cond ((memq seg asegs)
                        (setf (3AFFECTED aseg) [(car (xrole-fillers 'pred))
                                                !! subs
                                                !! (delq seg asegs)])))
               (ungroup seg)))))))
```

```
;construct a list of the non-join segs in cflow order
;(note trouble with filter joins).

(defun 3ssra-compute-seg-order ()
  (repeat ((segs [(~seg (~from (?& bodypat cflow * (&* rec))))]) seg)
    (setq seg (pop segs))
    (if (eq seg body) (return))
    (push seg seg-order)
    (let ((prior-segs (~set ~seg ~from (??& bodypat cflow * (&* seg)))))
      (cond ((null (cdr prior-segs))
             (if ;if filter pred it is there
              (not (memq (car prior-segs) segs))
              (push (car prior-segs) segs)))
            (t
             (push (or (filter-join-cor seg)
                       (barf "3SSR-ANALYZE: Bad join in SSR.~% Seg: ~S"
                             seg))
                   segs)
             (setq segs (append (delq (top segs) prior-segs) segs)))))))

;now construct the data flow part of the "3AFFECTED" relation

(defun 3ssra-build-affected ()
  (dolist (seg non-join-segs)
    (repeat ((bodypat (&* body))
             (tdo (??output seg))
             (affects (3AFFECTED seg))
             done port)
      (when (null tdo)
        (setf (3AFFECTED seg) (makefset affects))
        (return))
      (setq port (pop tdo))
      (when (not (member port done))
        (push port done)
        (dolist (df (??& bodypat dflow port))
          (let* ((to (~to df))
                 (toseg (~seg~segcase to))
                 (trans (porttrans to)))
            (if (not (memq toseg affects))
                (cond (trans (setq tdo (append trans tdo)))
                      ((3joinp toseg)
                       (setq tdo
                             (append (??& (&* toseg) output * (~id to)) tdo)))
                      (t
                       (push toseg affects)
                       (letS* (((pred . actions) (Elist pred-action-cor)))
                         (if (and (memq toseg actions)
                                  (not (memq seg actions))
                                  (not (memq pred affects)))
                             (push pred affects)))))))))))

;this returns a list of the ports reachable from the ports given.
;porttrans is used to account for the identity of the rec and the body

(defun 3dfaffect (supseg ports porttrans)
  (repeat ((supsegpat (&* supseg))
           (todo ports)
           done affected port)
    (setq port (when-pop-fails todo (return affected)))
    (when (not (member port done))
      (push port done)
      (dolist (df (??& supsegpat dflow port))
        (let* ((to (~to df))
               (toseg (~seg~segcase to))
               (trans (porttrans to)))
          (cond (trans (setq todo (append trans todo)))
                ((3joinp toseg)
                 (setq todo
```

```
                (append (??& (&* toseg) output * (~id to))
                        todo)))
        (t (push to affected))))))))
```

```
;look for an augmentation filter or term and pull it out.

(defun 3ssra-remove-fragment ()
 (prog ()
  (let* (headseg othersegs terminal? filter? junk)
;find a removable seg (ignore possibility of a set of segs on first pass.)
    (dolist (bucket 3AFFECTED)
      (let* (((seg . affects) bucket))
        (when (and (not (term-join-cor seg))
                   (fset< (fset^ affects non-join-segs) [seg]))
          (setq headseg seg)
          (push bucket junk)
          (return))))
;if didn't find anything, look for a set of segs
    (if (null headseg)
        (let* ((buckets (3ssr-find-removable-group
                          3AFFECTED term-join-cor non-join-segs))
               (segs (Rbag (car (Elist buckets)))))
          (when buckets
            (setq headseg (car segs))
            (dolist (seg (cdr segs))
              (if (memq seg (memq headseg seg-order))
                  (psetq headseg seg seg headseg))
              (push seg othersegs)
              (assert-fact (&~to (-& (&* body) cflow * (&* seg))
                                 (~to (-& (&* body) cflow (&* seg))))))
            (setq junk (append buckets junk)))))
;if still didn't find anything try for a term. (note it is important that
;you look for groups of segs first before terms.)
    (when (null headseg)
        (when (and (not *separate-gens-and-terms*)
                   (< (length (lets ((seg (car (Elist 3affected))))
                                (Rlist (Fselect seg (term-join-cor seg)))))
                      2))
          (setq 3affected nil) (return))
        (dolist (bucket 3AFFECTED)
          (let* (((seg . affects) bucket))
            (when (and (term-join-cor seg)
                       (fset< (fset^ affects non-join-segs) [seg])
                       (let (affected-before now-after)
                         (letS* ((s (Elist seg-order)))
                           (cond (now-after
                                  (if (memq seg (3AFFECTED s)) (done nil)))
                                 (t (cond ((eq seg s)
                                           (if (not affected-before)
                                               (done t))
                                           (setq now-after t))
                                          ((and (not (term-join-cor s))
                                                (memq seg (3AFFECTED s)))
                                           (setq affected-before t)))))
                                 T)))
              (setq headseg seg)
              (push bucket junk)
              (return)))))
;if still didn't find anything, you are out of luck.
    (cond ((null headseg)
           (grumble "SSR-ANALYZE: Couldn't completely analyze.~% SSR: ~S"
                    ssr)
           (setq 3AFFECTED nil))
```

```
;check if headseg is filter or termination and if it is remember
;the corresponding join
          (t (setq filter?   (filter-join-cor headseg)
                   terminal? (term-join-cor headseg))
;look and see if there are any other segs that can come out with the
;headseg. Note: you don't have to look for a term or filter, because
;anything with them is already grouped as a predicate.
;(also DCB doesn't want this.)
             (if (not (or filter? terminal? NO-GROUP-IN-TC
                          (??& * slot (&* headseg))))
                 (repeat ()
                   (if (not
                        (letS* ((bucket (Elist 3AFFECTED))
                                ((seg . affects) bucket))
                          (Ror (cond ((and (not (memq bucket junk))
                                           (not (term-join-cor seg))
                                           (not (filter-join-cor seg))
                                           (not (??& * slot (&* seg)))
                                           (qset< affects
                                                  [headseg ! othersegs])
                                           (null (??& (&* ssr) toutput
                                                      (&* seg))))
                                      (push bucket junk)
                                      (if (memq seg (memq headseg
                                                          seg-order))
                                          (psetq headseg seg seg headseg))
                                      (push seg othersegs)
                                      (assert-fact
                                       (&~to
                                        (-& (&* body) cflow * (&* seg))
                                        (~to (-& (&* body) cflow
                                                 (&* seg)))))
                                      t)))))
                       (return))))
;update the information about the inside of the body.
             (setq 3AFFECTED (qset- 3AFFECTED junk)
                   non-join-segs (qset- non-join-segs
                                        [headseg ! othersegs]))
;if there are other segs, then combine them with the head seg.
;note cannot be a filter or term
             (if othersegs
                 (let ((newheadseg
                        (3exprize [[headseg 'action]
                                   ! (Rlist (list (Elist othersegs)
                                                  'action))])))
                   (dolist (seg (cons headseg othersegs))
                     (dolist (toutputF (--& * toutput (&* seg)))
                       (let* ((newtoutputF
                               (assert-fact
                                (&~lower-env toutputF (& newheadseg)))))
                         (dolist (dflowF (--& * dflow toutputF))
                           (assert-fact (&~from dflowF newtoutputF)))))
                     (dolist (toutputF
                              (--& * toutput * * (&* (&* seg) output)))
                       (let* ((newport (>& (& newheadseg) output nil
                                           (genobj newheadseg
                                                   (~obj toutputF)) nil))
                              (newtoutputF
                               (assert-fact
                                (&~lower-io toutputF newport))))
                         (>& (& newheadseg) dflow (~lower-io toutputF)
                                                  newport)
                         (dolist (dflowF (--& * dflow toutputF))
                           (assert-fact (&~from dflowF newtoutputF))))))
                   (setq headseg newheadseg)))
;do the temporal decomposition
             (temporal-decompose [headseg ! (or filter? terminal?)]
```

```
(cond (filter? 'filter)
      (terminal? 'term)
      (t 'aug))))))))
```

```
;this finds a list of buckets so that
;1 every non-join seg affected by a seg in the list is in the list.
;2 no term are effected by any of the segs.
;3 no shorter such list exists.

(defun 3ssr-find-removable-group (3AFFECTED term-join-cor non-join-segs)
  (let* ((segs (letS* ((x (car (Elist 3affected))))
                 (Rbag (Fselect x (not (term-join-cor x))))))
         (subsets (Rlist (3ssr-subsets1 (Elist segs)
                           3AFFECTED term-join-cor non-join-segs)))
         (shortest
           (car (sort (Rbag (filterS #'cdr (Elist subsets)))
                      #'(lambda (x y) (< (length x) (length y)))))))
    (Rlist (assq (Elist shortest) 3affected))))

;starting with a given seg this makes a list of every seg
;it affects and transative closes on this ignoring term-join-cor segs.
;note 3affected can be circular it seems

(defun 3ssr-subsets1 (start-seg 3AFFECTED term-join-cor non-join-segs)
  (prog (to-do result seg)
        (setq result (ncons start-seg))
        (setq to-do (3AFFECTED start-seg))
      L (if (null to-do) (return result))
        (setq seg (pop to-do))
        (if (or (memq seg result) (not (memq seg non-join-segs))) (go L))
        (if (term-join-cor seg) (return nil)) ;effecting a term.
        (push seg result)
        (setq to-do (append to-do (3affected seg)))
        (go L)))
```

```
; this removes a filter, aug, or term from an ssr.  The basic operation is
;that a lot of stuff is coied fom the ssr into a new ssr, and some
;stuff is removed from the ssr.  The alist newmap shows what segs are
;being copied.  The keyseg, and join (if any) are moved across
;wholesale.
;the ssr, body, df-join, and rec are copied.  Newmap shows what
;the new seg names are.
; the variable newportmap keeps track of ports that are copied.
;the ports on the keyseg and join (if any) move across wholesale, because
;the segs themselves do.  Starting from this basis, every port that
;has dflow from or to a port that is being copied is also copied,
;if and only if the seg they are on is being copied.  (there is a
;cludge with filter joins in that the interaction with them isn't,
;copied, but rather just elided.)
; Any port that has no dflow to any port that is not copied,
;is deleted.  any tport that touches a port that is deleted, is
;copied.
; next we look at dflows.  there are three basic cases.
;1. a dflow is from a copied port to a copied port.  the dflow itself
;   is also copied.  if the ~to is deleted, then the dflow is to.
;   (note that it is not possible for the ~from to be deleted without
;    the ~to)
;2. only the ~to is being copied.  in this case temporal flows are set up
;   in order to encode the dflow.
;3. only the from is being copied (this is an error).
; lastly the apropriate cflow is copied, and the keyseg and join are
;actually put in the new seg.

(def-alistq newmap)
(def-alistq newportmap)
(declare (special copy-cflow term-join-cor))

;these additional specials are needed in order to break this fn up
;so that it can compile on the lispm
(declare (special newmap datatouts newportmap dominant-segcase
                  keyseg join type))

(defun temporal-decompose (stuff type) ;type is aug/term/filter
;help passes in a lot of data that would otherwise have to be recomputed.
  (let* (((keyseg . join) stuff)
         (newmap [!(3copy-ssr-skeleton ssr tc) [keyseg ! keyseg]
                  !(if join [[join ! join]])])
         datatouts newportmap dominant-segcase copy-cflow
         (*DB-CHECK-FACT-PORTS* NIL))
    (pp-trace *ANALYZE-Debug*
              [['temporal-decompose [keyseg ! join] type]
               (newmap ssr)])
    (td-find-dominant)
    (td-copy-output-dflow)
    (td-copy-input-dflow)
    (3copy-cflow (&* body) body (& body) newmap [body rec])
    (td-move-toutputs)
;now if there is a dominant-segcase, and there are no tinputs to the
;new ssr, a dummy temp dflow must be inserted to show the dominance of
;the filter.
    (if (and dominant-segcase (null (??& (&* (newmap ssr)) tinput)))
        (>& (& tc) dflow
            (>& (& ssr) toutput dominant-segcase (genobj ssr 'dummy) nil)
            (>& (& (newmap ssr)) tinput (& keyseg)
                (genobj (newmap ssr) 'dummy) nil)))
    (td-move-segs)
    (td-maybe-has-become-simple-seg)
    (newmap ssr)))
```

```
;this checks to see if the ssr was a split, but only has one output case
;left in which case it converts it to being a simple seg.

(defun td-maybe-has-become-simple-seg ()
  (when (and (??& (&* ssr) segtype 'split)
             (= (length (??& (&* (?supseg ssr)) cflow (&* ssr))) 1))
    (assert-fact (&~from (-& (&* body) cflow (&* rec)) (& rec)))
    (assert-fact (&~to (-& (&* body) cflow * (&* body)) (& body)))
    (assert-fact (&~from (-& (&* ssr) cflow (&* body)) (& body)))
    (assert-fact (&~to (-& (&* ssr) cflow * (&* ssr)) (& ssr)))
    (assert-fact (&~from (-& (&* (?supseg ssr)) cflow (&* ssr)) (& ssr)))
    (assert-fact (&~type (-& (&* rec) segtype) 'simple))
    (assert-fact (&~type (-& (&* body) segtype) 'simple))
    (assert-fact (&~type (-& (&* ssr) segtype) 'simple))))
```

```
(def-alistq ssr-roles)

;this makes an ssr with a body dfjoin and recs, having the same
;pattern as the ssr it is given as an argument.  note that this does
;not give a role to the new seg produced. this is done later.

(defun 3copy-ssr-skeleton (ssr tc)
  (let* ((ssr-roles (??roles ssr))
         (newssr (genseg 'ssr))
         (newssrcase (& newssr))
         (newbody (genseg 'body))
         (newbodycase (& newbody))
         (newdj (genseg 'dfj))
         (newdjcase (& newdj))
         (newmap [[ssr ! newssr]
                  [(car (ssr-roles 'body)) ! newbody]
                  [(car (ssr-roles 'dfjoin)) ! newdj]
                  ! (letS* ((seg (Elist (ssr-roles 'rec)))
                            (nrec (genseg seg)))
                      (+& newbodycase recursive (& nrec))
                      (+& (& nrec) segtype 'simple)
                      (+& newssrcase role (& nrec) 'rec)
                      (+& newbodycase subseg (& nrec))
                      (Rlist (cons seg nrec)))]))
    (+& (& tc) subseg newssrcase)
    (+& newssrcase segtype 'simple)
    (+& newbodycase segtype 'simple)
    (+& newssrcase plantype 'ssr)
    (+& newbodycase plantype 'ssr-body)
    (+& newssrcase subseg newbodycase)
    (+& newssrcase role newbodycase 'body)
    (+& newbodycase subseg newdjcase)
    (+& newssrcase role newdjcase 'dfjoin)
    (+& newdjcase segtype 'join)
    (+& newssrcase cflow newssrcase newbodycase)
    (+& newssrcase cflow newbodycase newssrcase)
    newmap))

;find the dominant filter test, term pred, or term join if there is one

(defun td-find-dominant ()
  (repeat ((seg keyseg) segcase)
    (setq segcase (~from (a?& (&* body) cflow * (&* seg)))
          seg     (~seg segcase))
    (if (eq seg body) (return))
    (cond ((term-join-cor seg)
           (setq dominant-segcase segcase)
           (return))
          ((filter-join-cor seg)
           (cond ((eq 'join (?segtype seg))
                  (setq seg (filter-join-cor seg)))
                 (t
                  (setq dominant-segcase segcase)
                  (return)))))))
```

```
;Next, copy all dflow leaving the keyseg the idea here is that
;there is a net of df arcs which originates at the output of the
;keyseg.  and that this net is all moved to the new seg being
;built.  this net touches on 4 kinds of segs: it can touch the body,
;and recs which have already been copied, in this case the ports
;are moved allong with the df.  it can touch the keyseg, or join,
;in which case the ports already naturally go across with the segs
;themselves.  it can touch a filter join, in which case the join is
;not copied, rather the df is modified to skip the join. if
;it touched anything else, decomposition would not be posible.  While
;the moving of df is going on, the variable datatouts is used to
;collect the temporal outputs which must also be copied [this is
;done later].  the variable newportmap is an alist showing what each
;moved port is copied to.
```

```
(defun td-copy-output-dflow ()
  (do ((todo (let ((outs (??output keyseg)))
               (setq newportmap (letS ((out (Elist outs)))
                                      (Rlist (cons out out))))
               outs))
       (done nil)
       (tokill nil))
      (nil)
    retry
    (when (null todo)
      (dolist (f tokill) (make-fact-be-out f))
      (return))
    (when (memq (top todo) done)
      (pop todo)
      (go retry))
    (push (top todo) done)
    (setq datatouts
          (nconc (--& (&* ssr) toutput * * (top todo))
                 datatouts))
    (dolist (df (--& (&* body) dflow (pop todo)))
      (let* ((to (~to df))
             (toseg (~seg~segcase to))
             (totype (cond ((eq toseg rec) 'rec)
                           (t (?segtype toseg))))
             (newfrom (newportmap (~from df))))
        (cond ((or (eq toseg keyseg) (eq toseg join))
               (>& (& (newmap body)) dflow newfrom to))
              ((and (eq totype 'join) (filter-join-cor toseg))
               (--& to @io)
               (make-fact-be-out to)
               (dolist (out (??& (&* toseg) output * (~id to)))
                 (--& out @io)
                 (make-fact-be-out out)
                 (push out todo)
                 (setf (newportmap out) newfrom)))
              ((eq toseg dj)
               (let ((id-port (~id-port to)))
                 (when (null (newportmap id-port))
                   (setf (newportmap id-port)
                         (=port-seg id-port (newmap dj)))
                   (push id-port todo)
                   (push id-port tokill))
                 (make-fact-be-out to)
                 (setf (newportmap to)
                       (>& (& (newmap toseg) (~case (~segcase to))) input 'id
                           (genobj (newmap toseg) (~obj to))
                           (~obj (newportmap id-port))))))
               (>& (& (newmap body)) dflow newfrom (newportmap to)))
              ((eq toseg rec)
               (setf (newportmap to) (=port-seg to (newmap toseg)))
               (make-fact-be-out to)
               (>& (& (newmap body)) dflow newfrom (newportmap to))
               (dolist (p (porttrans to))
                 (push p todo)
                 (setf (newportmap p)
                       (=port-seg p (newmap (~seg~segcase p))))
                 (make-fact-be-out p)
                 (dolist (idf (--& (&* ssr) dflow * p))
                   (let* ((from (~from idf))
                          (tcdf (?& (&* tc) dflow * from)))
                     (when (null (??& (&* ssr) dflow from))
                       (make-fact-be-out from)
                       (make-fact-be-out tcdf))
                     (when (not (newportmap from))
                       (setf (newportmap from) (=port-seg from (newmap ssr)))
                       (assert-fact (&~to tcdf (newportmap from))))
                     (>& (& (newmap ssr)) dflow
                         (newportmap from) (newportmap p))))))))
```

```
((eq toseg body)
 (setf (newportmap to) (=port-seg to (newmap toseg)))
 (make-fact-be-out to)
 (>& (& (newmap body)).dflow newfrom (newportmap to))
 (dolist (p (porttrans to))
   (push p todo)
   (setf (newportmap p)
         (=port-seg p (newmap (~seg~segcase p))))
   (make-fact-be-out p)
   (dolist (odf (--& (&* ssr) dflow to))
     (let* ((oto (~to odf)))
       (setf (newportmap oto) (=port-seg oto (newmap ssr)))
       (make-fact-be-out oto)
       (>& (& (newmap ssr)) dflow
           (newportmap to) (newportmap oto))
       (dolist (dflowF (--& (&* tc) dflow oto))
         (assert-fact (&~from dflowF (newportmap oto))))))))
(t
 (barf "TEMPORAL-DECOMPOSE: Trouble with dflow.~% DF: ~S"
       df)))))))
```

```
;for all of the dflow entering keyseg, copy it, or make a
;temporal df.  note that dfs which come from the outputs of keyseg
;have already been copied in the last stage and are not seen
;here.  if a df comes directly from an input to the body, and that
;input also flows directly to the corresponding inputs of the recs,
;then all that can be moved across (must check for more datatouts).
;Otherwise, a temporal output is constructed on the old ssr in order
;to supply the value for the new ssr.

(defun td-copy-input-dflow ()
  (dolist (df (--& (&* body) dflow * (&* (&* keyseg) input)))
    (let* ((from (~from df)))
      (cond ((and (eq (~seg~segcase from) body)
                  (letS* (((rec mapto) (Elist porttrans)))
                    (Rand-fast (or (not (equal mapto from))
                                   (??& (&* body) dflow from rec)))))
             (setf (newportmap from) (=port-seg from (newmap body)))
             (let* ((idf (?& (&* ssr) dflow * from))
                    (idfrom (~from idf))
                    (tcdf (?& (&* tc) dflow * idfrom))
                    (otherdfs (??& (&* body) dflow from))
                    (move-all?
                     (Rand-fast (memq from
                                      (porttrans (~to (Elist otherdfs)))))))
               (when move-all? (make-fact-be-out from))
               (setf (newportmap idfrom) (=port-seg idfrom (newmap ssr)))
               (assert-fact (&~to tcdf (newportmap idfrom)))
               (>& (& (newmap ssr)) dflow
                   (newportmap idfrom) (newportmap from))
               (>& (& (newmap body)) dflow (newportmap from) (~to df))
               (when move-all?
                 (make-fact-be-out idf)
                 (when (null (??& (&* ssr) dflow idfrom))
                   (make-fact-be-out idfrom)
                   (make-fact-be-out tcdf))
                 (setq datatouts
                       (nconc (--& (&* ssr) toutput * * from) datatouts)))
               (dolist (df otherdfs)
                 (let* ((to (~to df)))
                   (cond ((memq from (porttrans (~to df)))
                          (when move-all?
                                (make-fact-be-out df)
                                (make-fact-be-out to))
                          (setf (newportmap to)
                                (=port-seg to (newmap (~seg~segcase to))))
                          (>& (& (newmap body))
                              dflow
                              (newportmap from)
                              (newportmap to))))))))
            (t ;does not come copyably from body
             (>& (& tc) dflow
                 (let ((le (repeat ((segcase (~segcase (~to df)))
                                    seg
                                    (from-seg (~seg~segcase from)))
                             (setq seg (~seg segcase))
                             (cond ((eq seg from-seg) (return segcase))
                                   ((eq seg body)
                                    (barf
                                     "TEMPORAL-DECOMPOSE: Can't find 1-env~
                                     ~% From: ~:="
                                     from))
                                   ((eq seg (~seg dominant-segcase))
                                    (3update-affected from seg porttrans body)
                                    (return segcase)))
                             (setq segcase
                                   (~from (a?& (&* body) cflow
                                               * (&* (~seg segcase))))))))))))
```

```
  (or (~01 (??& (&* ssr) toutput le * from))
      (>& (& ssr) toutput le (genobj ssr (~obj from)) from)))
(>& (& (newmap ssr)) tinput (& keyseg)
   (genobj (newmap ssr) (~obj (~to df))) (~to df))))))))
```

```
;this copies all the necessary cflow.
;the special variable copy-cflow is used to keep track of what segs
;have been visited.

(defun 3copy-cflow (usegpat seg source newmap br)
  (cond ((memq seg copy-cflow) nil)
        (t (push seg copy-cflow)
           (dolist (cf (??& usegpat cflow (&* seg)))
             (let* ((to (~to cf))
                    (the-source (or source (~from cf))))
               (cond ((newmap (~seg to))
                      (>& (& (newmap (~seg usegpat))) cflow
                          (& (newmap (~seg the-source))
                             (if (not (memq (~seg the-source) br))
                                 (~case the-source)))
                          (& (newmap (~seg to))
                             (if (not (memq (~seg to) br)) (~case to))))
                      (3copy-cflow usegpat (~seg to) nil newmap br))
                     ((not (and (term-join-cor (~seg to))
                                (eq (?segtype (~seg to)) 'join)))
                      (3copy-cflow usegpat (~seg to)
                                   the-source newmap br))))))))
```

```
;now move across all of the temporal outputs in datatouts

(defun td-move-toutputs ()
  (dolist (dt datatouts)
    (let* ((newsegcase (& (newmap ssr) (~case (~segcase dt))))
           (le (~lower-env dt))
           (newle (cond ((eq (~seg le) keyseg) le)
                        ((null (cdr le)) (& (newmap (~seg le))))
                        (t (barf "TEMPORAL-DECOMPOSE: Cannot move lower io~
                                  ~% DT: ~S"
                                 dt))))
           (newlio (newportmap (~lower-io dt)))
           (newdt (>& newsegcase toutput newle
                      (genobj (newmap ssr) (~obj dt)) newlio)))
      (if (not (and newle newlio))
          (barf "TEMPORAL-DECOMPOSE: Can't move toutput.~% DT: ~S" dt))
      (dolist (dflowF (--& (&* tc) dflow dt))
        (assert-fact (&~from dflowF newdt)))))
;now move across any more touputs which refer to the environment of
;the keyseg even though they do not refer to data of the keyseg.
  (dolist (ct (??& (&* ssr) toutput (&* keyseg)))
    (cond ((null (~lower-io ct)) ;if is dummy tdflow port
           (let* ((nct (>& (& (newmap ssr)) toutput (~lower-env ct)
                           (genobj (newmap ssr) (~obj ct)) nil)))
             (dolist (dflowF (--& (&* tc) dflow ct))
               (assert-fact (&~from dflowF nct)))))
          (t ;is normal toutput
           (let* (new-le nct2 ncti nct)
             (repeat ((seg keyseg)
                      segcase
                      (lio (~lower-io ct))
                      (lioseg (~seg~segcase (~lower-io ct))))
               (setq segcase (~from (a?& (&* body) cflow * (&* seg)))
                     seg     (~seg segcase))
               (cond ((eq seg lioseg)
                      (setq new-le segcase)
                      (return))
                     ((eq seg body)
                      (barf
                       "TEMPORAL-DECOMPOSE: Can't find lower-env~% CT: ~S"
                       ct))
                     ((eq seg (~seg dominant-segcase))
                      (3update-affected lio seg porttrans body)
                      (setq new-le segcase)
                      (return))))
             (setq nct2
                   (or (car (??& (&* ssr) toutput new-le * (~lower-io ct)))
                       (assert-fact (&~lower-env ct new-le)))
                   ncti
                   (cond ((let ((df (car (??& (&* tc) dflow nct2
                                              (&* (& (newmap ssr)) tinput
                                                  (& keyseg))))))
                            (if df (~to df))))
                         (t
                          (>& (& (newmap ssr)) tinput (& keyseg)
                              (genobj (newmap ssr) (~obj ct)) nil)))
                   nct
                   (>& (& (newmap ssr)) toutput (~lower-env ct)
                       (genobj (newmap ssr) (~obj ct)) ncti))
             (>& (& tc) dflow nct2 ncti)
             (dolist (dflowF (--& (&* tc) dflow ct))
               (assert-fact (&~from dflowF nct))))))
    (erase-fact ct)))
```

```
;this is used to update the affected relation, when a temporal output
;is being made that links a source with a different dominant seg.
;it makes sure that the dominating filter/term will be pulled out
;before the actual source is.

(defun 3update-affected (from dominant-seg porttrans body)
  (repeat (from-seg)
    (setq from-seg (~seg~segcase from))
    (when (not (eq from-seg body))
      (setf (3AFFECTED from-seg)
            (fsetU (3AFFECTED from-seg) [dominant-seg]))
      (return))
    (setq from (letS* (((fport . tports) (Elist porttrans)))
                 (cond ((member from tports) (done fport)))))
    (setq from (~from (?& (&* body) dflow * from)))))

;now we must actually moved the segs across, and fix up the cflow in the
;old ssr (this is the hard part)

(defun td-move-segs ()
  (cond ((eq type 'aug)
         (-subseg body keyseg)
         (+subseg (newmap body) keyseg)
         (>& (& body) cflow
             (~from (-& (&* body) cflow * (&* keyseg)))
             (~to (-& (&* body) cflow (&* keyseg)))))
        ((eq type 'filter)
         (-subseg body keyseg)
         (+subseg (newmap body) keyseg)
         (-subseg body join)
         (+subseg (newmap body) join)
         (--& (&* body) cflow (&* keyseg) (&* join))
         (>& (& body) cflow
             (~from (-& (&* body) cflow * (&* keyseg)))
             (~to (-& (&* body) cflow (&* join)))))
        (t :(eq type 'term)
         (-subseg body keyseg)
         (+subseg (newmap body) keyseg)
         (--& (&* body) cflow (&* keyseg) (&* join))
         (>& (& body) cflow
             (~from (-& (&* body) cflow * (&* keyseg)))
             (~to (-& (&* body) cflow (&* keyseg))))
         (-subseg body join)
         (+subseg (newmap body) join)
         (let* ((cf1 (car (??& (&* body) cflow (&* rec) (&* join))))
                (cf2 (car (??& (&* body) cflow (&* join) (&* body))))
                (segcase (cond ((and cf1 cf2) ;this will be true in loops
                                (make-fact-be-out cf1)
                                (make-fact-be-out cf2)
                                (prog1
                                 (~to (-& (&* tc) cflow
                                          (~to (-& (&* ssr) cflow
                                                   (~to cf2)))))
                                 (if (null (??& (&* body) cflow (&* rec)))
                                     (>& (& body) cflow (& rec) (& body)))
                                 (if (null (??& (&* ssr) cflow (&* body)))
                                     (>& (& ssr) cflow (& body) (& ssr)))))
                               (t (repeat ((segcase (& join)))
                                    (if (eq (~seg segcase) tc)
                                        (return segcase))
                                    (setq segcase
                                          (~to (?& * cflow segcase)))))))))
           (>& (& tc) cflow (& (newmap ssr)) segcase))))
;indicate the role of the moved keyseg.
  (>& (& (newmap ssr)) role (& keyseg) 'op)
  (cond ((memq type '(term filter))
         (>& (& tc) role (& (newmap ssr)) type))
```

```
((null (??& (& (newmap ssr)) tinput))
 (>& (& tc) role (& (newmap ssr)) 'gen))
((null (??& (& (newmap ssr)) output))
 (>& (& tc) role (& (newmap ssr)) 'trn))
(t
 (>& (& tc) role (& (newmap ssr)) 'acc))))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp REPEAT Indent:1;
;;; Lisp REPEAT* Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts: MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;this file (which uses the prefix 5) contains functions for
;making LISP code corresponding to a plan.
;  the coding process works in three stages.  First temporal
;compositions are undone.  Then the plan is annotated with @seg, @io,
;and @creatvar information.  Lastly code is produced.
;finally the annotation produced is moved back to the temporally decomposed
;version, and the layers containing the composed version are discarded.
; the key to the matter is the annotation produced.
;DB contains the official definitions of the assertions.  This comment
;describes them in more detail.  (Some of the types created by the surface
;analyzer are not used here, and vis versa.)
;
;       ( <deep> @SEG <kind> <name> <code> )
;  <deep>    = a segcase
;  <kind>    = LAMBDA-EXP ! CONSTANT ! FUNCALL ! OPENCODE ! JOIN ! RECURSION
;  <code>    = a list of CONSs corresponding to the code for the seg
;  <name>    = if FUNCALL the name of the function
;              if CONSTANT the constant value
;              if top level opencode then the name the function is to have
;              otherwise nil
;this is used by the central dispatcher of the coder.  it classifies each
;segment.  Terminal segs are all of type CONSTANT or FUNCALL (except for joins
;and recursions).  All of the non-terminal segs are of type OPENCODE.  The
;code field does double duty showing what to display, and what to execute
;in order to get the return value.
;
;       ( <deep> @IO <kind> <name> <argno> )
;  <deep>    = an input or output
;  <kind>    = LAMBDA-ARG ! ACTUAL-ARG ! FREE-VAR ! RETURNVAL ! SIDE-EFFECT
;  <name>    = the variable name if the item
;              happend to be carried in a variable.
;  <argno>   = if LAMBDA-ARG or ACTUAL-ARG the argument position number.
;This controls how dflow will pass through the annotated port.
;LAMBDA-ARG is used only for inputs to the top level lambda
;expression, ACTUAL-ARG is used for inputs to a FUNCALL.  RETURNVAL is
;used when the data flow is going to be implemented by nesting of the
;returnval producing code in the receiving code. (precautions have to
;be taken so that this doesn't interact badly with FREE-VAR dataflow.)
;FREE-VAR indicates that the dflow is being carried in a variable.  If
;the same name is assigned to more than once you have to be careful.

;       ( <deep> @CREATEVAR <varname> )
;  <deep>    = the port where the variable is to be created
;  <varname> = the var name
;This indicates that a variable is being created by a setq.  It always happens
;AFTER the port (eg inside an input and outside an output).  It serves as a
;transducer to give something another name, or a name in the first place.
;Its primary use is to assign the returnval of a funcall to a variable for
;later use.
```

```
;the annotator which constructs the above assertions must construct
;a pattern of assertions which the coder can then code.  To do this it
;must follow certain restrictions.  The key coding difficulty is deciding
;how to implement the dataflow.  This is embedded in the @IO and @CREATEVAR
;assertions.  The problem for the coder comes when a data flow goes between
;two ports which are not of the same type.  (note that a join is constrained
;so that all of its inputs are the same type as the corresponding outputs
;so that no code is needed to correspond with the inside of the join.)
;the coder makes interfaces as follows
;
;NOTE a VAR-THING is a LAMBDA-ARG, FREE-VAR, or @CREATEVAR
;    from         to                  code produced
; VAR-THING  ACTUAL-ARG    nest var name in arglist
; VAR-THING  FREE-VAR      if name is the same then no code at all
;                         *do a setq before the code for the to seg
;                          or after it if free-var is an output
; VAR-THING  RETURNVAL     nest var name as last thing to be executed.
; VAR-THING  @CREATEVAR    var name must be different,
;                         *do a setq just inside the seg to the new var
;                          or just after the seg if thing is an output.
; RETURNVAL  ACTUAL-ARG    nest code for returnval in arg position
; RETURNVAL  FREE-VAR      *do a setq with the returnval code nested in it
;                          before or after the to seg code.
; RETURNVAL  RETURNVAL     nest returnval code as last thing to evaluate
; RETURNVAL  @CREATEVAR    *do a setq with the returnval code imbedded where you
;                          would have put the seg otherwise.
;
;There are two problamatical things in this.
;1. when a setq is to be done (the *ed items) you can get into two kinds of
;trouble.  first if a setq has to go after some seg then that seg can't
;have a returnval too unless you use PROG2s.
;Second if you have more than one setq in the
;same spot they may clash with each other.  You can't assign to a variable
;which you are going to use in another setq later.  The simple solution
;which will be used for now is using a PSETQ.
;2. if a returnval is used without an @CREATEVAR so somthing realy gets nested
;this can cause problems with other dataflows which are being carried in
;variables.  (the one time you are sure a returnval is completely safe is
;when it is the only output.)  this is delt with by really looking at
;what added constaints this produces.
;
;In order to be very safe on the above, and hopefully make the code prettier
;the annotator tries to follow the following guidelines
;1. use as few free-vars as possible
;2. don't assign to the same var twice unless you can prove that the domains
;   of use do not overlap.
;3. attempt to keep setqs to a minimum by avoiding name changes at
;   a port
```

```
;The coder has been extended to allow for PARTIAL plans.  This
;comment discusses just what ways the plan can be partial.  As will be
;seen, the key thing is that the data flow can be incomplete.  Each
;kind of thing that EDIT works with will be discussed seperately.

;1. A SEGMENT - On the one hand, if a segment isn't there, it just isn't
;there, and there is no problem.  On the other hand, to be well formed,
;some plans require some roles to definitely be there.  As a result,
;the coder will require them to be there.  (they don't have to have
;anything useful in them however)  These required actions include the
;PRED and JOIN of an XOR, and the BODY, REC and DFJOIN of an SSR.
;   Another way in which a segment can be incomplete is that it is just
;a place holder.  It has ports and cases, but no insides at all.  to
;represent this, you make it a funcall with a name field of NIL.  In
;this situation, it must be a SLOT, and the slot name will be used to
;code it up in #{} form.

;2. a CASE  In general whenever there are cases they are strongly
;interrelated with surrounding segments.  I cannot think of any
;situation where you could delete a case without having to change a lot
;of other things too in order to get a plan the coder would be able to
;code.  (Note that in a sense a case only exists because a cflow connects
;to it.)

;3. a CFLOW  This is the major place where the coder does not allow
;anything to be missing.  cflows are only allowed in xors, preds, ssrs,
;and tempcomps, and in these contexts they are absolutely mandatory,
;because the coder follows them around in order to tell what is going
;on.

;4. a DFLOW  This is the major place where the coder works to provide
;for incompleteness.  There are two ways in which a dflow can be
;missing, a port can have no dflow ending on it, or a port can have no
;dflow starting from it.  (The latter case doesn't even indicate that
;the program is actually incomplete and is no problem)
;the former case is treated by just trying to ignore it.  It
;shouldn't actually happen in normal programs since input slots
;are segs which properly interact with dflow.

;5. a PORT  If a port isn't there, then it just isn't there, and there
;is no problem.  I don't think that there is any situation where a port
;is required, except on a join, where a certain consitancy must be
;maintained.  (Note that if you remove an input from a call on some
;system fn, you may get an odd looking result.)  also note that if you
;delete a port, you must delete the dflow refering to it.
```

```
;this takes a seg and codes up a top level lambda expression corresponding
;to it.  any old @assertions are suppressed by pushing a context and
;deleting them.  they are used later to suggest variable names, and to
;give function names.

(declare (special SEGTYPES PSEGS NSEGS SEGFACTS
                  5TERMINAL? 5ORDERID REGIONS TOP-SEG TCs 5CLASSIFY
                  NAME-CLASSES SEGS-WITH-RETURNVALS VAR-NAMES-USED
                  5PTYPE 5FLAG 5IMMEDIATE 5PATTERN 5RELATED
                  5MAXID 5NEXTBIT 5LIMIT 5PLY 5BIT-PATTERN 5CLOSED 5IDS
                  NUM-SEGS CODE-FOR)
         (*expr db-verify-context1)
         (*fexpr dr))

(define-trace-option *CODER-Debug*
                     "Show trace of CODER internals"
  nil :boolean)

(def-alistq ORDERING)
(def-alistq roles)
(def-alistq mpred-roles)
(def-alistq lower-rets)
(def-alistq CODE-FOR)
(def-alist  REGIONS)
(def-alist  case-action)

(defun code (segcase)
  (let* ((mode (code-mode segcase))
         (result (code1 segcase mode))
         (def-name
           (caseq mode
             (defun (?@seg-name (~seg result)))
             (define-cliche (~cliche-def-name (?& result cliche-def))))))
    (if def-name
        (coder-confirmation def-name
                            (copytree (sel01 (?@seg-code (~seg result))))))
    result))

;Due to bad design of the assertions in the data base, there is
;no good way to tell the difference between a cliche instance and
;a cliche definition.  The following is a gross kludge that
;at least works much of the time.  Note that only compound non-top level
;forms can be cliches by the following definition.

(defun code-mode (segcase)
  (cond ((??& segcase @seg 'lambda-exp) 'defun)
        ((??& segcase cliche-def) 'define-cliche)
        (T 'opencode)))

(defun code1 (segcase mode)
  (declare (special *force-slots-up*))
  (let* (REGIONS 5CLASSIFY
         (TOP-SEG (~seg segcase))
         (original-context *db-context*) ;contains original version.
         5TERMINAL? VAR-NAMES-USED 5PTYPE ioc (NUM-SEGS 0)
         5FLAG 5IMMEDIATE 5PATTERN 5RELATED CODE-FOR
         5MAXID 5NEXTBIT 5LIMIT 5PLY 5BIT-PATTERN 5CLOSED 5IDS)
    (db-context-was <CODE)
    (5init-order)
    (push-db-context) ;make ply for the composed version.
    (setq ioc (5temporal-compose-all TOP-SEG))
    ;Must must make sure slot info is not lost.
    (let ((*force-slots-up* t))
      (3expr-simp TOP-SEG)) ;kill unneeded exprs created by editing.
    (5annotate TOP-SEG)
    (5codetop TOP-SEG mode)
    ;; restores & pushes context:
    (5fix-@assertions TOP-SEG ioc original-context)
```

```
    (db-context-was >CODE)
    segcase))
```

```
;run over the plan combining all tempcomps

(defun 5temporal-compose-all (seg)
  (let (TCs)
    (5find-tcs seg)
    (over TCs
      nconc (temporal-compose _))))

(defun 5find-tcs (seg)
  (cond ((eq (?plantype seg) 'tempcomp)
         (push seg TCs)
         (5tc-collapse seg)))
  (over (??subseg seg) (5find-tcs _)))

;This ungroups any nested TCs.

(defun 5tc-collapse (tc)
  (over (??& (&* tc) role) -> fact
    (let* ((seg (~seg~roleseg fact)))
      (5tc-fix-slots seg)
      (when (and (not (eq (~rolename fact) 'init))
                 (eq (?plantype (~seg~roleseg fact)) 'tempcomp))
        (if (eq (?plantype seg) 'tempcomp) (5tc-collapse seg))
        (let* ((roles (--& (&* seg) role))
               (dummy (genseg 'dummy))
               (dummy2 (genseg 'dummy)))
          (+subseg tc dummy)          ;prevents plan info from going up.
          (+subseg seg dummy2)        ;prevents plan info form going down.
          (ungroup seg)
          (5kill-dummy tc dummy)
          (5kill-dummy tc dummy2)
          (over roles (assert-fact (&~segcase _ (& tc)))))
        (let ((inits (~nset ~seg ~roleseg (??& (&* tc) role * 'init))))
          (if (cdr inits)
              (let ((new-init (group inits 'expr)))
                (+& (& new-init) plantype 'expr)
                (+& (& tc) role (& new-init) 'init)
                (over inits (+& (& new-init) role (& _) 'action)))))))))

;this saves any slot info so it will come out in the program text.
;note we are making this name with spaces in it that
;will have to be parsed out later.

(defun 5tc-fix-slots (seg)
  (let ((down-slotFs (??& (&* seg) slot))
        (up-slotF (~01 (??& * slot (&* seg)))))
    (when down-slotFs
      (letS ((down-slotF (Elist down-slotFs))
             (new-name (if (and (not-implemented? seg) up-slotF)
                           (intern (format nil "~A OF THE ~A"
                                           (~slotname down-slotF)
                                           (~slotname up-slotF))
                                   *kbe-package*)
                           (~slotname down-slotF))))
        (>& (if up-slotF
                (~segcase up-slotF)
                (& (?supseg seg)))
            slot (~slotseg down-slotF) new-name (~slottype down-slotF))
        (make-fact-be-out down-slotF))
      (if up-slotF (make-fact-be-out up-slotF)))))


(defun 5kill-dummy (sseg dummy)
  (let ((from (~01 ~from (??& (&* sseg) cflow * (&* dummy))))
        (to   (~01 ~to   (??& (&* sseg) cflow (&* dummy)))))
    (if (and from to) (>& (& sseg) cflow from to))
    (kill-seg dummy)))
```

```
(declare (special tcomp-order tcomp-order-tc))

;this takes in a tc and combines all of its fragments together into a
;single ssr.  it only works for loops at the moment.  the list (ioc
;[I/O Correspondances]) which is returned is a list of pairs of ports
;in the temporal plan, and the new ports they correspond to.  this is
;used by the coder.  (note that there are several places where less than
;the maximal amount of simplification is done when things are combined
;so that as few segments as possible have to be deleted.  this helps
;the correspondance between old and new ports.  [this is done primarily
;with the inits as they are combined.])

;NOTE we assume that all of the subsegs of the TC can be combined.
;This means that any slots must have dummy ssr insides!

(defun temporal-compose (tc)
  (let* ((tc-init (?role tc 'init))
         (tcomp-order-tc tc)
         (tcomp-order nil)
         (tc-bssr (?role tc 'bssr))
         (tc-subsegs (tc-sorted-ssrs tc))
         new-ssr
         ioc)
    (cond (tc-bssr
           (setq new-ssr tc-bssr)
           (setq tc-subsegs (delq tc-bssr (append tc-subsegs nil)))
           (td-maybe-should-be-split-seg new-ssr))
          (T (setq new-ssr (tcomp-prototype tc))))
    (setq ioc (over tc-subsegs
                nconc (temporal-combine tc new-ssr _)))
    (if tc-init
        (let* ((new-ssr-init (?role new-ssr 'init))
               (dummy-subseg (genseg '*dummy*)))
          (+subseg tc dummy-subseg)
          (move-seg-down new-ssr tc-init) ;dummy stops ungrouping of tc
          (-subseg tc dummy-subseg)
          (cond (new-ssr-init
                 ;note no simplification is done here!!
                 (let ((new-init (group [new-ssr-init tc-init] 'expr)))
                   (+& (& new-ssr) role (& new-init) 'init)
                   (+& (& new-init) plantype 'expr)
                   (+& (& new-init) role (& new-ssr-init) 'action)
                   (+& (& new-init) role (& tc-init) 'action)))
                (t
                 (>& (& new-ssr) role (& tc-init) 'init)))))
    (mini-analyze (& (?role new-ssr 'body)))
    (let* ((dummy-subseg (genseg '*dummy*)))
      (+subseg new-ssr dummy-subseg)
      (ungroup new-ssr) ;note dummy makes plantype info go up for sure!
      (5kill-dummy tc dummy-subseg))
    ioc))
```

```
;this checks to see if the new-ssr should be a split because it
;goes to a split case on the TC.

(defun td-maybe-should-be-split-seg (new-ssr)
  (let* ((cflows (??& (&* tc) cflow (&* new-ssr) (&* tc)))
         (case (and cflows (null (cdr cflows))
                    (~case (~to (car cflows))))))
    (when case
      (let ((body (?role new-ssr 'body))
            (rec (?role new-ssr 'rec)))
        (assert-fact
          (&~from (-& (&* body) cflow (&* rec)) (& rec case)))
        (assert-fact
          (&~to (-& (&* body) cflow * (&* body)) (& body case)))
        (assert-fact
          (&~from (-& (&* new-ssr) cflow (&* body)) (& body case)))
        (assert-fact
          (&~to (-& (&* new-ssr) cflow * (&* new-ssr)) (& new-ssr case)))
        (assert-fact
          (&~from (-& (&* tc) cflow (&* new-ssr)) (& new-ssr case)))))))
```

```
(defun tc-sorted-ssrs (tc)
  (let ((ssrs (letS ((seg (Elist (??subseg tc))))
                (Rbag (Fselect seg (??& (&* seg) plantype 'ssr)))))
        result)
    (loop do
      (cond ((null ssrs) (return (nreverse result))))
      (letS ((ssr (Elist ssrs))
             (prior-segs (~set ~seg~segcase ~from
                               (??& * dflow * (&* (&* ssr) tinput)))))
        (cond ((null (qset^ prior-segs ssrs))
               (push ssr result)
               (setq ssrs (delq ssr ssrs))
               (done)))))))

;this brings into play some of the capabilities of the analyzer.
;the absolute minimum is done in order not to disturb the basic grouping.
;we just must do some grouping in the body in order for the coder to work.
;Note that we have to do some dflow moving in order to make the output
;code look aesthetic.

(defun mini-analyze (supsegcase)
  (let* ((supseg (~seg supsegcase))
         SEGTYPES PSEGS NSEGS SEGFACTS) ;note that this is critical!
    (3group-up supseg)
    (setq segfacts nil)
    (3dflow-simp1 nil supseg)
;this ungrouping takes away room to move in dflow and so I don't
;think we realy want to have it, but I don't know why it was
;put here in the first place and so am scared to take it out. -- DICK
    (prog ()
      L (dolist (seg (??subseg supseg))
          (cond ((eq (?plantype seg) 'expr)
                 (ungroup seg)
                 (go L)))))
    supsegcase))

;this sets up an empty ssr to add things into

(declare (*fexpr deffacts))

(defun tcomp-prototype (tc)
  (let (bssr)
    (setq bssr
          (~seg (plan-def bssr nil
                          (deffacts bssr
                            (segtype (simple)) (plantype (ssr))
                            (role (bssrbody body) (rec rec) (dj dfjoin))
                            (subseg (bssrbody))
                            (cflow (bssr bssrbody) (bssrbody bssr)))
                          (deffacts bssrbody
                            (segtype (simple)) (plantype (ssr-body))
                            (subseg (rec) (dj))
                            (recursive (rec))
                            (cflow (bssrbody rec) (rec bssrbody)))
                          (deffacts rec
                            (segtype (simple)))
                          (deffacts dj
                            (segtype (join))))))
    (+subseg tc bssr)
    bssr))
```

```
(declare (special st-cf-order st-cf-order-body ioc))

;this combines a fragment (aug, filter, or term) with an
;ssr in order to create a bigger ssr.  (at the moment
;this only works with tcs which are loops.) this produces
;ioc for each combination.

(defun temporal-combine (tc bssr frag)
  (if (null (??subseg frag))
      (barf "CODER plan to code has empty ssr: ~A" bssr))
  (let* (dest-segcase ioc (frag-body (?1role frag 'body)))
    (pp-trace *CODER-Debug* '(temporal-combine ,tc ,bssr ,frag))
;look at the temporal inputs in order to
;decide where to put the keyseg in the bssr.  basically it goes as
;high up as possible consistant with the dflow constraints.
    (let* ((l-envs (~nset ~lower-env ~from
                          (??& (&* tc) dflow * (&* (&* frag) tinput))))
           (st-cf-order-body (?1role bssr 'body))
           (st-cf-order nil))
      (setq l-envs
            (letS ((env (Elist l-envs)))
              (Rlist (Fselect env (eq st-cf-order-body
                                      (?supseg (~seg env)))))))
      (if (cdr l-envs) (setq l-envs (sortcar l-envs #'l-env-sort)))
      (setq dest-segcase
            (~to (?& (& st-cf-order-body) cflow
                     (cond (l-envs (car l-envs))
                           (t (& st-cf-order-body)))))))
    (5tc-move-frag-ports tc bssr frag)
    (5tc-move-frag-init bssr frag)
    (5tc-move-frag-body-ports bssr frag)
    (5tc-move-frag-dj bssr frag)
    (5tc-fix-remaining-tports bssr frag)
    (5tc-move-fragkeyseg tc bssr frag dest-segcase)
;check that we have gotten everything
    (if (or (??& (&* frag-body) dflow)
            (??& (&* frag) tinput)
            (??& (&* frag) toutput))
        (barf "TEMPORAL-COMBINE: Some junk left over.~
               ~% TC:   ~S~
               ~% BSSR: ~S~
               ~% Frag: ~S"
                tc bssr frag))
;kill all remaining junk
    (over (??subseg frag-body) (kill-seg _))
    (kill-seg frag-body)
    (kill-seg frag)
;return the ioc list
    ioc))
```

```
;move the inputs of the outer seg of the frag over.  some colapsing
;may occure because some of the values may already be going to the
;bssr.  move over the dflows which touch these inputs (e.g. switch to
;new port and change the supersegs).  Note that the stuff pushed onto ioc
;is odd due to the fact that the bssr will be ungrouped at the end.
;  Next move the outputs of the outer seg over.  no collapsing can happen.
;move dflow too.  Note some additional work has to be done on the
;body-ssr dflow later.

(defun 5tc-move-frag-ports (tc bssr frag)
  (over (--input frag) -> in
        (car (--& (&* tc) dflow * in)) -> df-to-in
        (and df-to-in
             (car (??& (&* tc) dflow (~from df-to-in)
                       (&* (&* bssr) input)))) -> parallel-df
        (cond (parallel-df (~to parallel-df))
              (t (+& (& bssr) input nil (genobj bssr (~obj in)) nil)))
          -> new-in
    (when df-to-in
      (if (null parallel-df) (assert-fact (&~to df-to-in new-in)))
      (push [in ! (~from df-to-in)] ioc))
    (over (--& (&* frag) dflow in) -> df
      (>& (& bssr) dflow new-in (~to df))))
  (over (--output frag) -> out
        (+& (& bssr) output nil (genobj bssr (~obj out)) nil) -> new-out
        (car (--& (&* frag) dflow * out)) -> df-to-out
        (??& (&* tc) dflow out) -> dfs-from-out
    (if dfs-from-out (push [out ! (~to (car dfs-from-out))] ioc))
    (cond (df-to-out (+& (& bssr) dflow (~from df-to-out) new-out))
          (t (barf "TC-MOVE-FRAG-INPUTS: Frag output with no dflow to it.~
                    ~% Out: ~:="
                    out)))
    (dolist (df dfs-from-out)
      (+& (& tc) dflow new-out (~to df)))))

;move init (if there is one) and dflow from it over (dflow to it came
;over above).  If there was a bssr init too, then combine them together
;into a single new init.  Note that the result must be an AND.
;note that no simplification is done, so that the old init will never
;be ungrouped.

(defun 5tc-move-frag-init (bssr frag)
  (let* ((fraginit (?role frag 'init))
         (bssrinit (?role bssr 'init)))
    (cond (fraginit
           (-& (&* frag) subseg (&* fraginit))
           (-& (&* frag) role (&* fraginit))
           (>& (& bssr) subseg (& fraginit))
           (>& (& bssr) role (& fraginit) 'init)
           (dolist (df (--& (&* frag) dflow (&* (&* fraginit) output)))
             (>& (& bssr) dflow (~from df) (~to df)))
           (if bssrinit
               (cond ((eq 'and (?plantype bssrinit))
                      (move-seg-down bssrinit fraginit)
                      (+& (& bssrinit) role (& fraginit) 'action))
                     (t
                      (let ((new-init (group [fraginit bssrinit] 'and)))
                        (+& (& bssr) role (& new-init) 'init)
                        (+& (& new-init) plantype 'and)
                        (+& (& new-init) role (& bssrinit) 'action)
                        (+& (& new-init) role (& fraginit) 'action)))))))))
```

```
;move the inputs of the body/rec over and all dflow touching them.
;Also move over temporal ports (note tinput only to rec, toutput only from
;body).  The tinputs and the associated tdflow are converted into normal
;dflow.  There can be collapsing of ordinary inputs iff there is direct
;body rec dflow (to coressponding ports) in both frag and bssr!
;Next move outputs of body/rec over. dflow and temporal ports too.
;(no collapsing is possible.)

(defun 5tc-move-frag-body-ports (bssr frag)
  (let* ((fragbody (?1role frag 'body))
         (fragrec (?1role frag 'rec))
         (bssrbody (?1role bssr 'body))
         (bssrrec (?1role bssr 'rec)))
    (over (??input fragbody) -> in
          (?& (&* fragrec) input * (~obj in)) -> rec-in
          (car (--& (&* bssr) dflow * in)) -> df-to-in  ;note moved
          (if df-to-in (car (??& (&* bssr) dflow (~from df-to-in)
                                 (&* (&* bssrbody) input))))
            -> parallel-df
          (and parallel-df
               (??& (&* bssrbody) dflow (~to parallel-df)
                    (&* (&* bssrrec) input * (~obj (~to parallel-df))))
               (??& (&* fragbody) dflow in rec-in))
            -> can-share
          (cond (can-share (~to parallel-df))
                (t (+& (& bssrbody) input nil
                       (genobj bssrbody (~obj in)) nil)))
            -> new-in
          (cond (can-share (?& (&* bssrrec) input * (~obj new-in)))
                (t (+& (& bssrrec) input nil
                       (genobj bssrrec (~obj new-in)) nil)))
            -> new-rec-in
      (if (and df-to-in (null can-share))
          (assert-fact (&~to df-to-in new-in)))
      (push [rec-in ! new-rec-in] ioc)
      (push [in     ! new-in    ] ioc)
      (over (--& * dflow in) -> df         ;note might have moved already!
        (>& (& bssrbody) dflow new-in (~to df)))
      (over (--& * dflow * rec-in) -> df   ;note might have moved already!
        (>& (& bssrbody) dflow (~from df) new-rec-in))
      (5tc-move-toutput in new-in bssr)
      (5tc-move-tinput rec-in new-rec-in bssrbody)
      (kill-port-and-commentary in)
      (kill-port-and-commentary rec-in))
    (over (??output fragbody) -> out
          (?& (&* fragrec) output * (~obj out)) -> rec-out
          (+& (& bssrbody) output nil
              (genobj bssrbody (~obj out)) nil) -> new-out
          (+& (& bssrrec) output nil
              (genobj bssrrec (~obj new-out)) nil) -> new-rec-out
      (push [rec-out ! new-rec-out] ioc)
      (push [out     ! new-out]     ioc)
      (over (--& (&* fragbody) dflow rec-out) -> df
        (>& (& bssrbody) dflow new-rec-out (~to df)))
      (over (--& * dflow * out) -> df  ;may have moved already
        (>& (& bssrbody) dflow (~from df) new-out))
      (over (--& (&* bssr) dflow out) -> df  ;half moved above.
        (assert-fact (&~from df new-out)))
      (5tc-move-toutput rec-out new-rec-out bssr)
      (5tc-move-tinput out new-out bssrbody)
      (kill-port-and-commentary out)
      (kill-port-and-commentary rec-out))))
```

```
;move dfjoin over e.g. move all its ports over to the bssr-dfjoin and
;move all of the dflow (most of which will aready probably have been
;moved).  No collapsing possible.  move tios too as above.
;moved-out is an alist linking old outs with new ones.

(def-alistq moved-out)

(defun 5tc-move-frag-dj (bssr frag)
  (let* (moved-out
         (fragdj (?1role frag 'dfjoin))
         (bssrdj (?1role bssr 'dfjoin))
         (bssrbody (?1role bssr 'body)))
    (over (??output fragdj) -> out
          (>& (& bssrdj) output nil
              (genobj bssrdj (~obj out)) nil) -> new-out
      (setf (moved-out out) new-out)
      (push [out ! new-out] ioc)
      (over (--& * dflow out) -> df
        (>& (& bssrbody) dflow new-out (~to df)))
      (5tc-move-toutput out new-out bssr))
;
    (over (--input fragdj) -> in
          (>& (& bssrdj (~case (~segcase in))) input 'id
              (genobj bssrdj (~obj in))
              (~obj (moved-out (~id-port in)))) -> new-in
      (push [in ! new-in] ioc)
      (over (--& * dflow * in) -> df
        (>& (& bssrbody) dflow (~from df) new-in))
      (5tc-move-tinput in new-in bssrbody))
    (over moved-out -> [out ! _] (make-fact-be-out out))))

;this little fn compares two segs in order to see which is further down
;in the control flow.

(defun 1-env-sort (s1 s2)
  (memoize st-cf-order [s1 ! s2]
    (cond ((eq s1 st-cf-order-body) nil)
          ((eq s1 s2) t)
          (t
           (1-env-sort
            (~seg (~from (a?& (&* st-cf-order-body) cflow * (&* s1))))
            s2)))))
```

```
;these take care of moving tports when a port they might be touching
;is moved.

(defun 5tc-move-toutput (port new-port bssr)
  (dolist (tout (??& * toutput * * port))
    (5tc-move-toutput1 tout new-port bssr)))

(defun 5tc-move-toutput1 (tout new-port bssr)
  (let* ((port (~lower-io tout))
         (new-tout
          (>& (& bssr) toutput
              (cond ((and port (equal (~lower-env tout) (~segcase port)))
                     (~segcase new-port))
                    (t
                     (~lower-env tout)))
              (genobj bssr (~obj tout)) new-port)))
    (over (--& * dflow tout) (assert-fact (&~from _ new-tout)))
    (kill-port-and-commentary tout)))

(defun 5tc-move-tinput (port new-port bssrbody)
  (dolist (tin (??& * tinput * * port))
    (5tc-move-tinput1 tin new-port bssrbody)))

(defun 5tc-move-tinput1 (tin new-port bssrbody)
  (let* ((port (~lower-io tin))
         (source (~01 ~lower-io ~from (--& * dflow * tin))))
    (cond (port
           (when (and (null source) (??& * toutput * * tin))
             (let* ((seg (genseg 'dummy))
                    (in (>& (& seg) input nil
                            (genobj seg (~obj tin)) nil))
                    (out (>& (& seg) output nil
                             (genobj seg (~obj tin)) nil))
                    (cflow (-& (&* bssrbody) cflow (&* bssrbody))))
               (+& (& seg) @seg 'funcall '5noop nil)
               (+& (& seg) segtype 'simple)
               (+& in @io 'actual-arg nil 1)
               (+& out @io 'returnval nil nil)
               (+& (& bssrbody) subseg (& seg))
               (+& (& bssrbody) cflow (~from cflow) (& seg))
               (+& (& bssrbody) cflow (& seg) (~to cflow))
               (setq source out)))
           (cond (source
                  (cond ((or (eq bssrbody (?supseg (~seg~segcase source)))
                             (eq bssrbody (~seg~segcase source)))
                         (>& (& bssrbody) dflow source new-port)
                         (5tc-move-toutput tin source
                                           (?supseg bssrbody)))
                        (T ;special init->init dflow case
                         (implement-dflow source new-port))))))
          (source (5tc-move-toutput tin source (?supseg bssrbody)))
          (t
           (let* ((bssr (?supseg bssrbody))
                  (new-tin (+& (& bssr) tinput (& bssr)
                               (genobj bssr (~obj tin)) nil)))
             (5tc-move-toutput tin new-tin bssr))))
    (kill-port-and-commentary tin)))

;fix all of the remaining tinputs to frag.  by converting
;them to ordinary dflow.  Note that if a tinput goes
;directly to a toutput, then this is done by changing the
;toutput.  (note that a tinput can be used both ways.  if
;it is only used with a tout, then it will not have a
;~lower-io.)   Then move over the remaining temporal outputs.

(defun 5tc-fix-remaining-tports (bssr frag)
  (let* ((fragkeyseg (?role frag 'op))
         (bssrbody (?1role bssr 'body)))
```

```
(dolist (tin (??& (&* frag) tinput))
  (5tc-move-tinput1 tin (~lower-io tin) bssrbody))
(when fragkeyseg
  (dolist (tout (??& (&* frag) toutput (&* fragkeyseg)))
    (5tc-move-toutput1 tout (~lower-io tout) bssr)))
(dolist (tout (??& (&* frag) toutput))
  (5tc-move-toutput1 tout (~lower-io tout) bssr))))
```

```
;now actually move the keyseg into the bssr (this is the only part
;which differs based on what the frag's plantype is).
;Note that in a few special pure dflow ssrs there is no keyseg!

(defun 5tc-move-fragkeyseg (tc bssr frag dest-segcase)
  (let* ((bssrbody (?1role bssr 'body))
         (bssrrec (?1role bssr 'rec))
         (fragtype (~rolename (?& (&* tc) role (&* frag))))
         (fragbody (?1role frag 'body))
         (fragrec (?1role frag 'rec))
         (fragkeyseg (?role frag 'op))
         (controlling-segcase
          (~from (?& (&* bssrbody) cflow * dest-segcase))))
    (if fragkeyseg
        (let ((old-cf (-& (&* bssrbody) cflow controlling-segcase)))
          (>& (& bssrbody) cflow controlling-segcase (& fragkeyseg))
          (>supseg bssrbody fragkeyseg)
          (cond ((memq fragtype '(gen acc trn))
                 (>& (& bssrbody) cflow (& fragkeyseg) (~to old-cf)))
                ((eq fragtype 'filter)
                 (let ((fjoin (~seg (~to (a?& (&* fragbody) cflow
                                              (&* fragkeyseg))))))
                   (>supseg bssrbody fjoin)
                   (>& (& bssrbody) cflow (& fjoin) (~to old-cf))
                   (dolist (cf (--& (&* fragbody) cflow (&* fragkeyseg)))
                     (assert-fact (&~segcase cf (& bssrbody))))))
                (t ;(eq fragtype 'term)
                   ;new case is taken from the case of the tc
   (dolist (cf (--& (&* fragbody) cflow (&* fragkeyseg)))
     (cond ((eq (~seg (~to cf)) fragrec)
            (>& (& bssrbody) cflow (~from cf) (~to old-cf)))
           (t (assert-fact (&~segcase cf (& bssrbody)))
              (>supseg bssrbody (~seg (~to cf)))
              (let* ((cf0 (-& (&* fragbody) cflow * (&* (~seg (~to cf)))))
                     (cf1 (-& (&* fragbody) cflow (&* (~seg (~to cf)))))
                     (cf2 (-& (&* frag) cflow (~to cf1)))
                     (cf3 (-& (&* tc) cflow (~to cf2)))
                     (ocf0 (car (??& (&* bssrbody) cflow (& bssrrec))))
                     newcf1 newcf2)
                (cond (ocf0
                       (make-fact-be-out ocf0)
                       (assert-fact
                         (&~from ocf0 (& bssrrec (~case (~to cf3)))))
                       (--& (&* bssr) cflow (& bssrbody) (& bssr))
                       (setq ocf0 (-& (&* bssrbody) cflow * (& bssrbody)))
                       (assert-fact (&~to ocf0 (~to cf0))))
                      (t (>& (& bssrbody) cflow
                             (& bssrrec (~case (~to cf3))) (~to cf0))))
                (setq newcf1 (>& (& bssrbody) cflow (~from cf1)
                                 (& bssrbody (~case (~to cf3)))))
                (setq newcf2 (>& (& bssr) cflow (~to newcf1)
                                 (& bssr (~case (~to cf3)))))
                (>& (& tc) cflow (~to newcf2) (~to cf3)))
              (when (cdr (??& (&* tc) cflow (&* bssr)))
                (-& (&* bssr) segtype)
                (>& (& bssr) segtype 'split)
                (-& (&* bssrbody) segtype)
                (>& (& bssrbody) segtype 'split)
                (-& (&* bssrrec) segtype)
                (>& (& bssrrec) segtype 'split)))))))))

(declare (unspecial st-cf-order st-cf-order-body ioc))
```

```
;the programs which annotate the code work on two basic
;stores of knowledge a partial order keeping track of all
;of the ordering constraints embodied in the plan and in
;the variable names chosen, and the set of all the dflow
;regions in the program.  (a dflow region is a source
;port, together with all of the ports the data value
;travels through and to.)  the key facts are that each
;dflow region must be internally consistant (eg use the
;same variable name everywhere) and that if the same name
;is used in two regions, then they cannot over lap.

;each region has the following form
;  [SOURCE NAME [1ST 2ND 3RD] INTERMEDIATES ! DESTINATIONS]
;see 5FIND-REGIONS and 5SUGGEST-NAME for descriptions of the parts.

;note that this assumes that terminal segs are essentially already
;annotated right.

;note that the inputs to an ssr-body are treated specially all over the
;place.  this is due to the fact that they must be treated as
;starting regions even though they appear to be intermediate in
;the plan.

;{{there is some extent to which terminal segs which take
;  freevar inputs and outputs can still mess things up a lot.
;  if the value is carried to that seg in some other variable,
;  then a setq will have to be done, and the program doesn't check
;  that this will really work.}}
```

```
(defun 5annotate (seg)
  (let* (NAME-CLASSES SEGS-WITH-RETURNVALS (counter 0)
         (temp-vars '(x y z u v w a b c)))
    (5abstract-ordering seg nil)
    (5close)
    (5find-regions seg)
    (5suggest-names) ;uses old @ios
    (5clean@assertions seg) ;important is just here
    (setq REGIONS (nreverse REGIONS)) ;put important ones first.
    (over REGIONS (5determine-name-class _))
    (5determine-nesting seg
     (~01 ~deep (??& (&* (&* seg) output) @io 'returnval)))
    (5close)
    (format-trace *CODER-Debug* "~&The regions are ~Q~
                                 ~&The name classes are ~Q" #'dr #'dn)

;first annotate every region which has a good guess at the data flow.
    (over REGIONS -> [![_ name [1st 2nd 3rd] ! _] = region]
      select (and (null name) 1st)
      (cond ((eq 1st '_return) (5try-make-be-returnval region))
            (t (5try-make-be-name region 1st))))

;use name classes to suggest names for related regions.
    (5use-name-classes 20)

;next try any other names which have been suggested
    (over REGIONS -> [![_ name [1st 2nd 3rd] ! _] = region]
      select (and 2nd (null name)
                  (dolist (3name 2nd)
                    (return-if (5try-make-be-name region 3name)))))

;use name classes to suggest names for anybody who is left
    (5use-name-classes 0)

;next try and see if we can figure out a name from the function that
;creates the value.
   (over REGIONS -> [![_ name ! _] = region]
      select (null name)
      (dolist (var (3make-up-name region))
        (return-if (5try-make-be-name region var))))

;use name classes to suggest names for anybody who is left
    (5use-name-classes 0)

;next try any blah names that were suggested.
    (over REGIONS -> [![_ name [1st 2nd 3rd] ! _] = region]
      select (and 3rd (null name)
                  (dolist (3name 3rd)
                    (return-if (5try-make-be-name region 3name)))))

;use name classes to suggest names for anybody who is left
    (5use-name-classes 0)

;lastly, if you must, make up a temporary variable name and use it
    (over REGIONS -> [![_ name ! _] = region]
      select (null name)
      (let* ((temp (repeat (name)
                     (setq name (when-pop-fails temp-vars
                                  (symbolconc 'TEMP (incf counter))))
                     (if (not (memq name VAR-NAMES-USED))
                         (return name)))))
        (5try-make-be-name region temp)             ;cannot fail
        (dolist (r (5get-related-regions region 0)) ;use the name as many
          (when (null (cadr r))                     ;times as you can
            (5try-make-be-name r temp)))))
;
    (5cleanup seg)))
```

```
(defun 5use-name-classes (min)
  (prog (did-something)
    L (setq did-something nil)
      (over REGIONS -> [![_ name ! _] = region]
        select (null name)
        (over (5get-related-regions region min) -> [_ n ! _]
          select (and n (not (memq n '(_return _))))
          (when (5try-make-be-name region n)
            (setq did-something T)
            (return))))
      (if did-something (go L))))

(defun 3make-up-name (region)
  (let* ((source (car region))
         (seg (~seg~segcase source))
         var)
    (errset (and (??& source @io 'returnval)
                 (setq var (3make-up-name-for-returnval seg))) nil)
    (cond (var (list var)))))

;this is only a stub now.  It could do all kinds of things
;for more reasonable guessing. (this is used other places too.)

(defun 3make-up-name-for-returnval (seg)
  (caseq (?@seg-name seg)
    ((ada-fn~query_user_for_date query_user_for_date) 'date)
    (T nil)))
```

```
;this cleans up all the @ assertions, keeping only the minimally necessary
;information.  The information which must be kept is the @SEGs on the
;terminal segs, and the @IOs on the terminal segs and the TOP-SEG.
;in addition, it creates new @segs for every non-terminal seg.

(defun 5clean@assertions (seg)
  (--& (&* (&* seg) port) @createvar)
  (let* ((@seg (~01 (--& (&* seg) @seg)))
         (@segkind (if @seg (~kind @seg)))
         (@segname (if @seg (~name @seg))))
    (if (memq @segkind '(opencode funcall constant lambda-exp))
        (incf NUM-SEGS)) ;this is displayed by the coder
    (cond ((eq @segkind 'lambda-exp)
           (>& (& seg) @seg 'lambda-exp @segname nil))
          ((5terminal? seg)
           (cond ((??& (&* seg) segtype 'join) ;might not have had one
                  (>& (& seg) @seg 'join nil nil)
                  (--& (&* (&* seg) port) @io))
                 ((eq @segkind 'funcall)
                  (>& (& seg) @seg 'funcall (5get-lisp-name @segname) nil))
                 ((eq @segkind 'constant)
                  (>& (& seg) @seg 'constant
                      (5get-lisp-constant @segname) nil))
                 ((??& * recursive (&* seg)) ;might not have had one
                  (>& (& seg) @seg 'recursion nil nil)
                  (--& (&* (&* seg) port) @io))
                 (t
                  (>& (& seg) @seg 'opencode nil nil)
                  (if (?supseg seg) (--& (&* (&* seg) port) @io)))))
          ;; part of the gross cliche definition hack
          ((and (eq @segkind 'opencode) (get @segname 'cliche-comments))
           (>& (& seg) @seg 'opencode @segname nil))
          (t
           (>& (& seg) @seg 'opencode nil nil)
           (if (?supseg seg) (--& (&* (&* seg) port) @io))))
    (over (??subseg seg) (5clean@assertions _))))

;this runs over the whole plan and abstracts out all of
;the partial ordering information encoded in the cflow
;and dflow.  This is used below to determine whether or
;not two dflow regions overlap.  note special encodings
;of joins.

(defun 5abstract-ordering (seg supseg)
  (dolist (dflow (??& (&* seg) DFLOW))
    (5port-make-precede (~from dflow) (~to dflow)))
  (over (??& (&* seg) cflow) -> cflow
        (~from cflow) -> fsegcase
        (~to cflow) -> tsegcase
    (5make-precede
      (5order-id fsegcase (cond ((eq (~seg fsegcase) seg) 'input)
                                (t 'output)))
      (5order-id tsegcase (cond ((eq (~seg tsegcase) seg) 'output)
                                (t 'input)))))
  (let* ((segin (5order-id (& seg) 'input)) ;note segin=segout if terminal
         (segout (5order-id (& seg) 'output))
         (segins (makeset  ;segins=(& segin)/nil  if not join
                   [!(over (if supseg (??& (&* supseg) cflow * (&* seg)))
                       bag (5order-id (~to _) 'input))
                    !(over (??& (&* seg) input) bag (5port>order-id _))]))
         (segouts (makeset  ;segouts=(& segout)/nil if not split
                    [!(over (if supseg (??& (&* supseg) cflow (&* seg)))
                       bag (5order-id (~to _) 'output))
                     !(over (??output seg) bag (5port>order-id _))])))
    (over segins (5make-precede _ segin))
    (over segouts (5make-precede segout _))
    (dolist (subseg (??subseg seg))
      (5make-precede segin (5order-id (& subseg) 'input))
```

```
(5make-precede (5order-id (& subseg) 'output) segout)
(5abstract-ordering subseg seg))))
```

```
;this runs over the whole plan, and finds all of the dflow regions which
;it stores in REGIONS.  Each region is of the form
;   [source name [1st 2nd 3rd] intermediates ! destinations]
;regions are started by every port which does not have dflow to it
;(ie a top level input, a bottom level output, or an intermediate level
;port where the dflow is incomplete).  In addition, a region is started for
;every input of an SSR-BODY that doesn't have dflow directly to the
;corresponding input of the rec (ie which changes in the loop).
;(Note that data flow that carries data to places where it isn't
;actually used, is ignored in all this.  EG if some data flow starts
;at some output and then goes through some joins and stuff but
;isn't actually used anywher then no regions will be made for it.)

(defun 5find-regions (seg)
  (mapc #'5find-region (??input seg))
  (mapc #'5find-region (??output seg))
  (mapc #'5find-regions (??subseg seg)))

;returns the region for a port if any.
(defun 5find-region (port)
  (cond ((cdr (assq port regions)))
        ((eq (5region-classify port) 'head)
         (let (((int . dest) (5find-successors port)))
           (when dest
             (push [port nil nil int ! dest] REGIONS)
             (car regions))))))

;returns port description
(defun 5region-classify (port)
  (memoizeq 5CLASSIFY port
    (let* ((seg (~seg~segcase port))
           (ptype (5ptype seg))
           (dflow-from (??& * dflow port))
           (dflow-to (??& * dflow * port))
           (complex-bssr-port
             (and (input-fact? port)
                  (eq ptype 'ssr-body)
                  (null (??& (&* seg) dflow port
                             (&* (&* (?recurrence seg))
                                 input * (~obj port))))))))
      (cond ((and dflow-from dflow-to)
             (if complex-bssr-port 'head 'inter))
            ((and (not dflow-from) (not dflow-to)) 'ignore)
            (dflow-from 'head)
            ((output-fact? port)
             (if (or (null (?supseg seg))
                     ;;needed for trailing-pointer list enumeration
                     (memq ptype '(ssr-body ssr)))
                 'dest 'ignore))
            ((??subseg seg) 'ignore)
            ((eq (?segtype seg) 'join) ;NOTE RECURSION HERE
             (if (5find-region (?& (&* seg) output * (~id port)))
                 'dest 'ignore))
            (T 'dest))))))

;returns intermediate and destination ports if any.
(defun 5find-successors (port)
  (let ((inter) (dest))
    (letS ((to (~to (Elist (??& * dflow port)))))
      (caseq (5region-classify to)
        ((head dest) (push to dest))
        (inter (let (((sub-inter . sub-dest) (5find-successors to)))
                 (setq inter (append sub-inter [to] inter))
                 (setq dest (append sub-dest dest))))))
    (cons inter dest)))

;this is for debugging, it prints out REGIONS in a readable way.
(defun dr (&optional (full? nil) (supress_returns nil))
```

```
  (dolist (region REGIONS)
    (let (((s n suggestions is . ds) region))
      (when (not (and supress_returns (eq n '_return)))
        (pp:pp [(compress-port s) n suggestions
                !(if full? [(over is bag (compress-port _))
                           (over ds bag (compress-port _))])]))))
  (values))

;this prints out name-classes

(defun dn ()
  (dolist (class name-classes)
    (let (((a n b) class))
      (pp:pp [(compress-port (car a)) n (compress-port (car b))])))
  (values))
```

```
;this is one of the most critical parts of the coder,  It should really
;work by constraint propogation so that it can balance the choice in
;one region against choices in others in order to reduce the number of
;setqs.  It should also not reuse the same name unless there is some
;logical reason to do so.  (The name classes are a (not to great)
;attempt at this.)
;As a hack more or less, it looks at what the original user did in order
;to try and pick reasonable variable names.

;This fn looks at a region and makes suggestions as to the var name to use
;for dflow in the region.  it returns a triple [1st 2nd 3rd] where
;1st - 3rd -- are suggested names in order of how good an idea they are.
;1st -- name chosen because the source has a name specified for it, and
;    most of the uses have the same name specified.  (Determination of
;    nesting may eventually put _RETURN in this spot indicating that a
;    retval should be used)  The idea here is that in either case we
;    have solid evidence that this name was actually used before.
;2nd -- not used at the moment
;3rd -- a list of every other name that occures in some @ assertion for
;       the region.  they are ordered according to how often they
;       appeared.
;
;note that names beginning with "_" are assumed to be internally
;generated and are ignored.

;if the source is an intermediate level port with no dflow to it, then
;it is given the suggested name "_" which is then used.  (NOTE how this
;is not a suggestion that will get reused when the thing is recoded
;later.)

(defvar *blah-names* '("STRUCTURE" "DATA" "DATUM" "FILE" "RECORD")
  "list of var names suffixes we really don't want to use")

(defun blah-name? (name)
  (let* ((string (string name))
         (start (string-reverse-search-set "-_" string))
         (suffix (if start (substring string (1+ start)) string)))
    (member suffix *blah-names*)))

(defun 5suggest-names ()
  (letS ((region (Elist regions))
         ((port nil nil inter . dest) region)
         (name-info (5suggest-name port inter dest)))
    (setf (caddr region) name-info)))

(defun 5suggest-name (source intermediates dests)
  (let* ((source-name (and (null (?supseg (~seg~segcase source)))
                           (~01 5goodname ~name (??& source @io))))
         (other-names (over [source !! intermediates !! dests] -> port
                        (or (~01 5goodname ~slotname
                                 (??& * slot (~segcase port) * 'output))
                            (~01 5goodname ~varname (??& port @createvar))
                            (~01 5goodname ~name (??& port @io))) -> name
                        select name bag name))
         (blah-names (letS ((name (EList other-names)))
                       (Rlist (Fselect name (blah-name? name))))))
    (when source-name (setq other-names (delq source-name other-names)))
    (setq other-names (qset- other-names blah-names))
    (setq other-names (sort (over other-names classifyq [_])
                            #'(lambda (x y) (> (length x) (length y)))))
    (list source-name (mapcar #'car other-names) blah-names)))

;;; (5GOODNAME sym)
;;;
;;;  Used to filter out system generated names.
;;;
;;;  Returns NIL if sym is NIL or a system name that begins with
;;;  an underscore. Otherwise, returns the sym (which will obviously
```

```
;;;  be non-NIL).

(defun 5goodname (sym)
  (cond ((or (null sym)) nil)
        ((not (eq (aref (get-pname sym) 0) #/_)) sym)))
```

```
;Looks at a source of a region and checks to see if it is a side-effected
;thing.  if so it enters the appropriate partial order restrictions, and if
;the thing isn't specified directly as a free variable, then it indicates
;that this region should have the same var name as the linked region.
;Note that at this point all of the ordering is completly specified
;except in EXPRs and ANDs.  as a result ordering restrictions only have to
;be introduced if the two regions reach into the same EXPR/AND.  It is
;important that they only be introduced then, or things in CONDs can get
;messed up since the two branches of a COND must remain unordered.
;  In a similar vein, if the head of the region is an output of a join,
;then it suggests that the same name should be used for
;the regions terminating on the corrsponding inputs.
;Also, if the region starts on an input of an ssr-body
;[note no regions pass through an input of an ssr-body]
;then the program suggests that the same name should be
;used for the region ending on the input, and the one
;going to the corresponding input of the rec.  But note
;that we propogate names out of loop, but don't use
;outside name to modify in loop!  (ie we check that one won't
;propogate into the body unless it is really coming
;from a real thing in the init.)
;Note that in these last two cases,
;mere suggestions are being made, while the side-effect stuff is much more
;serious.
;  special care is taken so that the fact that something initializes two
;different ports of an ssr-body will not cause spurious collapsing of
;name-classes.  this is done by including the input region only in
;the port region which has the best chance of suggesting a good name
;[note that other dflow factors can still cause the regions to collapse
;we are ignoring this for now.]
; We also try to propogate the same name across any instances of 5noop.

(defun 5add-name-class (reg1 reg2 binding
                        &optional (reverse-binding binding))
  (when (and reg1 reg2)
    (if (plusp binding)
        (push (list reg1 binding reg2) NAME-CLASSES))
    (if (plusp reverse-binding)
        (push (list reg2 reverse-binding reg1) NAME-CLASSES))))

(defun 5determine-name-class (source-region)
  (let* ((source (car source-region))
         (e-input (and (??& source @io 'side-effect) (~id-port source))))
    (cond (e-input
           (let* ((prior-region (dolist (region REGIONS)
                                  (if (member e-input (cddddr region))
                                      (return region))))
                  (this-id (5port>order-id e-input))
                  (dest-ids (over (cddddr prior-region) -> dest
                                  (5port>order-id dest) -> dest-id
                                  select (not (eq dest-id this-id))
                                  bag dest-id))
                  (other-ports [!!(cadddr prior-region)
                                  !!(cddddr prior-region) ! nil])
                  (these-ports (over p <- e-input
                                     by (~from (?& * dflow * p))
                                     until (not (member p other-ports))
                                     bag (setq other-ports
                                               (delete p other-ports))
                                     p)))
             (cond ((over dest-ids fastor (5precedes? this-id _))
                    (warn
                     "5DETERMINE-NAME-CLASS: dflow and side-effects clash.~
                     ~% Source: ~:="
                     source))
                   (t
                    (5constrain< other-ports these-ports '(expr and comp))
                    (5close)))
```

```
(cond ((not (??& e-input @io 'free-var))
       (5add-name-class source-region prior-region 90)))))
```

```
          ((??& (&* (~seg~segcase source)) @seg 'funcall '5noop)
           (over (??& (&* (~seg~segcase source)) input) -> in
                 (dolist (region REGIONS)
                   (if (member in (cddddr region))
                       (return region))) -> prior-region
                 (5add-name-class prior-region source-region 80)))
          ((??& (&* (~seg~segcase source)) segtype 'join)
           (over (??& (&* (~seg~segcase source)) input
                      'id * (~obj source)) -> in
                 (dolist (region REGIONS)
                   (if (member in (cddddr region))
                       (return region))) -> prior-region
                 (5add-name-class prior-region source-region 80 60)))
          ((and (input-fact? source)
                (eq (5ptype (~seg~segcase source)) 'ssr-body))
           (let* ((rec (?recurrence (~seg~segcase source)))
                  (rec-port (?& (&* rec) input * (~obj source)))
                  (prior-region (dolist (region REGIONS)
                                  (if (member source (cddddr region))
                                      (return region))))
                  (rec-region (dolist (region REGIONS)
                                (if (member rec-port (cddddr region))
                                    (return region)))))
             (if (not (eq (~seg~segcase source)
                          (~seg~segcase (car rec-region))))
                 (5add-name-class source-region rec-region 50 -100))
             (when (and prior-region
                        (over (cddddr prior-region) -> port
                              select (not (equal source port))
                              fastand (not (eq (~seg~segcase port)
                                               (~seg~segcase source)))))
;propogate name out of loop, but don't modify outside name in loop!
               (5add-name-class prior-region source-region 40 -100)
;can pull in a name from a true init however.
               (when (and (not (??& (~segcase (car prior-region))
                                    @seg 'funcall '5noop))
                          (2seg-inside-seg
                            (~seg~segcase (car prior-region))
                            (?supseg (~seg~segcase source))))
                 (5add-name-class
                   prior-region source-region -100 30))))))))

(defun 5constrain< (portsa portsb ptypes)
  (let* ((segsa (5constrain-segs portsa ptypes))
         (segsb (5constrain-segs portsb ptypes)))
    (over segsa -> [sega ! supa]
      (over segsb -> [segb ! supb]
        (if (and (eq supa supb) (not (eq sega segb)))
            (5make-precede (5order-id (& sega) 'output)
                           (5order-id (& segb) 'input)))))))

(defun 5constrain-segs (ports ptypes)
  (over ports -> p
        (~seg~segcase p) -> seg
        (?supseg seg) -> sup
    select (and sup (memq (?plantype sup) ptypes))
    bag [seg ! sup]))
```

```
;The next pgm runs over the plan for the seg, and decides what are all
;of the things which can be made returnvals.  As a side effect of
;this, it is determining a complete ordering in each expr, by adding
;orderings to ORDERING.  Note that due to this side-effect, it is very
;important that it looks at EVERY expr.  (note that since all of its
;activities are confined to single segments, and it keeps track of the
;changes it is making to ORDERING, the ordering does not have to be
;reclosed until after all of the nesting has been computed.)  When it
;finds something which can be a returnval, it signals this by putting
;_RETURN as the first suggestion for the region.

;The fn works recursively top down, seg by seg.  it works
;keying off of the plan type.  it takes advantage of the
;fact that since the analyzer did dflow simplyfication,
;NESTING WILL NEVER MOVE A SEGMENT OVER A SEGMENT
;BOUNDRY.  Note that this implies that the intermediate
;ports of a region which is marked as _RETURN must all be
;outputs!

;note that at each point, action is taken with respect to
;the subsegs, and not with respect to the seg you are on.
;info comes down as to whether any port on this seg
;should be a retval, and this info is passed down to the
;appropriate other place.  if the appropriate lower place
;cannot form a retval output, then the region in question
;is broken in two, and the upgoing portion is marked as
;_RETURN, while the lower portion is left alone.  Thus by
;hook or crook, the request for a retval is always
;satisfied.  for this to work, it must be that case that
;whenever 5determine-nesting is called on a terminal seg,
;then a retval can only be asked for where it is
;possible.  (the fact that there is no interlevel look
;ahead as to what can be conveniently be made a retval,
;speeds things up and misses some opportunities for
;optimal nesting.)

;the fn keys off of the following plantypes
;1. ssr -- something from the body may be able to be a retval.
;2. xor -- outputs from actions, and/or join might be a retval (note not init)
;3. pred -- an output of the init may be able to be a returnval.
;          (note it isn't minor that all preds have only one arg!
;           for otherwise we might miss some nesting here. with AND inits)
;          note that niether a pred (or an ssr used as one) can have a retval.
;4. and -- one of the inner pieces may have a retval.
;5. expr, comp --  here is the big work, where the heavy decisions about
;    nesting get made.  The algorithm works backwards from the retval output
;    (if any) trying to nest as many things as possible.
;6. ssr-body -- there is complete cflow in here, and nothing is last, so
;    there cannot be a real retval.  as a result, if one is needed above,
;    the region will have to be broken.  [Note we don't have to do anything
;    here because temporal-compose regroups the interior after it acts.
;    if there are any and/expr/xor regions in there they will have
;    been grouped.]

(declare (special SEGS-TO-DO FIRST-SEG-ID SEGS-PATTERN))
```

```
(defun 5determine-nesting (seg retoutput)
 (cond
  ((5terminal? seg)
   (if retoutput (5mark-returnval retoutput)));this had better be ok!!
  (t
   (let* ((ptype (5ptype seg))
          (inner-ret-source
            (if retoutput (~01 ~from (??& (&* seg) dflow * retoutput))))
          (inner-ret? (and inner-ret-source
                           (memq ptype '(and expr comp xor ssr ssr-body))
                           (5can-be-ret inner-ret-source)))
          (inner-ret-seg
            (if inner-ret? (~seg~segcase inner-ret-source)))
          (subsegs (??subseg seg))
          lower-rets)
     (if inner-ret?
         (setf (lower-rets inner-ret-seg) inner-ret-source))
     (if (and retoutput (not inner-ret?))
         (5mark-returnval (5break-region retoutput)))
     (cond ((memq ptype '(expr comp))
            (let* ((SEGS-TO-DO subsegs)
                   (SEGS-PATTERN (5make-multiple-pattern SEGS-TO-DO))
                   (FIRST-SEG-ID (5order-id (& seg) 'output)))
              (if inner-ret-seg
                  (cond ((5any-follow inner-ret-seg SEGS-PATTERN)
                         (5mark-returnval (5break-region retoutput)))
                        (t
                         (5expr-nesting inner-ret-source
                                        inner-ret-seg))))
              (repeat ()
                (if (null SEGS-TO-DO) (return))
                (let* ((possible-segs
                        (over SEGS-TO-DO
                              select (not (5any-follow _ SEGS-PATTERN))
                              list _)))
                  ;; some time we may want to sort possible-segs.
                  (5expr-nesting nil (car possible-segs))))))
           (t
            (caseq ptype
              ((AND)
               (if inner-ret-seg
                   (let* ((id (5order-id (& inner-ret-seg) 'input)))
                     (over subsegs -> seg
                       select (not (eq seg inner-ret-seg))
                       (5make-precede (5order-id (& seg) 'output) id)))))
              ((SSR-BODY)
               (let ((rec (?recurrence seg)))
                 (if retoutput
                     (setf (lower-rets rec)
                           (?& (&* rec) output * (~obj retoutput))))
;;; this isn't the place to be doing this we should be getting the
;;; call on the rec right some other way.  This gets wrong stuff
;;; some times  --DICK
                 (repeat ((ssr-sub rec) ports)
                   (cond ((eq ssr-sub seg) (return))
                         ((and (setq ports (??output ssr-sub))
                               (null (cdr ports))
                               (5can-be-ret (car ports)))
                          (setf (lower-rets ssr-sub)
                                (car ports))))
                   (setq ssr-sub
                         (~seg (~from (?& (&* seg) cflow *
                                          (&* ssr-sub)))))))))
              ((PRED)
               (over (~1st ??output (??role seg 'init))
                     select (5can-be-ret _)
                     (setf (lower-rets (~seg~segcase _)) _)
                     (return)))
```

```
((XOR)
 (if inner-ret?
     (if (eq (?segtype inner-ret-seg) 'join)
         (over (??& (&* inner-ret-seg) input
                    'id * (~obj inner-ret-source)) -> input
               (~from (?& (&* seg) dflow * input)) -> source
           select (and (5can-be-ret source)
                       (??& (&* seg) role
                            (~segcase source) 'action))
           (setf (lower-rets (~seg~segcase source))
                 source))))))
(over subsegs (5determine-nesting _ (lower-rets _)))))))))
```

```
;helping fns for the above

(defun 5mark-returnval (output)
  (let ((region (assoc output REGIONS)))
    (if region (rplaca (caddr region) '_return))))

;takes in an intermediate or destination port in some region, and
;breaks it off to start its own region.  (the port is returned)

(defun 5break-region (output)
  (cond ((not (5find-region output))
         (push [output nil [nil nil nil] nil output] regions)))
  (let* ((old-region (over (cdr REGIONS) ;note the cdr isn't just for speed
                       select (or (member output (cadddr _))
                                  (member output (caddddr _)))
                       (return _)))
         (new-region (assoc output REGIONS)))
    (cond (old-region
           (rplacd (cddr old-region)
                   [(delete output (set- (cadddr old-region)
                                         (cadddr new-region)))
                    !(setU [output] (set- (caddddr old-region)
                                          (caddddr new-region)))])
           (5add-name-class old-region new-region 10 0))))
  output)

;this checks to see if a random port can be a returnval in that it has
;dflow to only one place, it is an output, and if it has an @io, then it
;is a returnval.  PROBABLY NEED TO BE MORE SUFFISTICATED HERE.
;note we don't have ssrs have rets for aesthetic reasons.
;note gross kludge by just saying that all rec outputs can be
;nested.  this isn't really right I suppose but works for now.

(defvar 5un-nestable-fns
  '(ada-constant-dcl~pseudo ada-init-dcl~pseudo ada-dcl~pseudo
                            ada-fn~read))

(defun 5can-be-ret (port)
  (and (output-fact? port)
       (or (??& * recursive (~segcase port))
           (and (or (assoc port REGIONS)
                    (??& (~segcase port) dflow * port))
                (not (eq (?segtype (~seg~segcase port)) 'split))
                (not (memq (~01 ~name (??& (~segcase port) @seg))
                           5un-nestable-fns))
                ;note middle level boxes don't say returnval!
                (memq (?@io-kind port) '(nil returnval))
                (null (cdr (??& * dflow port)))
                (null (cdr (??& (&* (~segcase port) output)
                                @io 'returnval)))
                (not (eq (5ptype (~seg~segcase port)) 'ssr))
                (or (??& (~segcase port) @seg 'funcall '5noop)
                    (not (5goes-to-side-effect port)))))))

(defun 5goes-to-side-effect (port)
  (or (??& (&* (&* (~seg~segcase port)) output 'id * (~obj port))
           @io 'side-effect)
      (Ror-fast (5goes-to-side-effect (~to (Elist (??& * dflow port)))))
      (let ((upper (?recursive-supseg (~seg~segcase port))))
        (cond (upper (5goes-to-side-effect
                       (?& (&* upper) input * (~obj port))))))))

;this computes the basic nesting of things in an expression.
;it also goes beyond this to provide order stability to things
;that give data flow to a seg even when they cannot be nested.

(defun 5expr-nesting (retoutput seg &aux maybe-later)
  (setq SEGS-TO-DO (delq seg SEGS-TO-DO)
```

```
      SEGS-PATTERN (5remove-from-pattern seg SEGS-PATTERN))
  (let ((this-id (5order-id (& seg) 'output)))
    (5make-precede this-id FIRST-SEG-ID)
    (setq FIRST-SEG-ID this-id))
  (5determine-nesting seg retoutput)
  (over (sort (??& (&* (&* seg) input) @io 'actual-arg)
              #'(lambda (x y) (> (~argno x) (~argno y)))) -> @io
        (car (??& * dflow * (~deep @io))) -> source-df
        (if source-df (~from source-df)) -> source
    select (and source (eq (~keyword source) 'output)
                (not (5any-follow (~seg~segcase source) SEGS-PATTERN)))
    (cond ((5can-be-ret source)
           (5expr-nesting source (~seg~segcase source)))
          ((??& (&* seg) output 'id * (~obj (~deep @io)))
           (push (~seg~segcase source) maybe-later))))
  (dolist (seg (nreverse maybe-later))
    (if (not (5any-follow seg SEGS-PATTERN))
        (5expr-nesting nil seg))))
```

```
;this fn takes in a region and a name, and tries to use it as the
;variable name.  The only reason the name might not be able to be used
;is if the region overlaps with some other region that also uses the
;name.  If the name can be used, it puts in the correct @ios, marks the
;region by putting the name in its name field, and then returns T.

;this is a very gross kludge to be sure.
;it lists names that come directly from cliches and should be just
;taken on faith.
(defvar *unquestionable-vars*
        '(current_output units repairs defects models
          unit_key repair_index repair))

(defun 5try-make-be-name (region name)
  (cond ((or (eq name '_)
             (if (not (memq name VAR-NAMES-USED))
                 (push name VAR-NAMES-USED))
             (memq name *unquestionable-vars*)
             (over REGIONS -> reg
              select (and (eq name (cadr reg))
                          (not (5noop-related region reg)))
              fastand (not (5regions-overlap region reg))))
         (setf (cadr region) name)
         (5annotate-var-dflow name region)
         t)))

(defun 5noop-related (region1 region2)
  (or (5nr region1 region2) (5nr region2 region1)))

;this should really be doing a transitive closer type of thing.

(defun 5nr (region1 region2)
  (and (??& (~segcase (car region2)) @seg 'funcall '5noop)
       (memq (?& (~segcase (car region2)) input)
             (cddddr region1))))
```

```
;this checks to see that two regions don't overlap in time.
;It is important, that things not get
;moved around any after this check is made, because nothing is
;checking that this doesn't get violated later.  Note I think this
;depends on the fact that regions don't go through joins.

;Note also that at this time, there is complete control flow in every seg
;except ANDs.  (This is always true of xors, preds, and ssrs; compose got
;rid of tcs, and determining the nesting put complete ordering constraints
;in comps and exprs.)  Reusing a variable name here, can cause ordering
;constraints in an and to appear.  These must be entered so that the coder
;will obay them, and we won't make a contradictory decision later.
;Unfortunately this is a messy process.

;I don't understand this anymore and don't think that it is right
;really but it seem to be working sort of -- DICK

(defun 5regions-overlap (region1 region2)
  (let* (((sourcea nil nil intera . destsa) region1)
         ((sourceb nil nil interb . destsb) region2)
         (heada (5port>order-id sourcea))
         (tailsa (over destsa bag (5port>order-id _)))
         (headb (5port>order-id sourceb))
         (tailsb (over destsb bag (5port>order-id _)))
         (heada<headb (5precedes? heada headb))
         (headb<heada (5precedes? headb heada)))
    (cond ((cond (heada<headb (over tailsa fastor (5precedes? headb _)))
                 (headb<heada (over tailsb fastor (5precedes? heada _)))
                 (t (over [headb ! tailsb] -> idb
                      fastor (over [heada ! tailsa] -> ida
                                   fastor (or (5precedes? ida idb)
                                              (5precedes? idb ida)))))))
          (t
           (cond (heada<headb
                  (5constrain< [sourcea !! intera ! destsa]
                               [sourceb !! interb ! destsb] '(and)))
                 (t
                  (5constrain< [sourceb !! interb ! destsb]
                               [sourcea !! intera ! destsa] '(and))))
           (5close)
           nil))))

;this fn takes in a region, and tries to make it implemented by
;returnvals.  (it assumes that it is only called when it is possible.
;it annotates the region, marks it, and returns T.)

(defun 5try-make-be-returnval (region)
  (when (5annotate-return-dflow region)
    (setf (cadr region) '_return)
    T))
```

```
;this fn takes a name and a region, and annotates all of the ports in the
;region so that the name will be used for the dflow in the region in
;a completely consistant way.

;note we are assuming that 5kill@assertions was called so that there are
;no @createvars, and no @ios except on terminal segs to start with

(defun 5annotate-var-dflow (name region)
  (let* (((source nil nil intermediates . dests) region)
         (source@io (car (??& source @io))))
    (cond ((null source@io)
           (>& source @io 'free-var name nil))
          ((and (eq name (~name source@io)) (eq (~kind source@io) 'free-var)))
          ((memq (~kind source@io) '(free-var returnval))
           (>& source @createvar name))
          ((memq (~kind source@io) '(lambda-arg side-effect actual-arg))
           (assert-fact (&~name (make-fact-be-out source@io) name)))
          (t
           (barf "5ANNOTATE-VAR-DFLOW: Bad source @io.~% Data: ~:="
                 source@io)))
    (dolist (port intermediates)
      (>& port @io 'free-var name nil))
    (over dests -> port
          (car (??& port @io)) -> @io
      (cond ((and ;taken care of elsewhere (is head of region)
                  (??& * dflow port)
                  (eq (5ptype (~seg~segcase port)) 'ssr-body)))
            ((null @io)
             (>& port @io 'free-var name nil))
            ((or (eq (~name @io) name)
                 (memq (~kind @io) '(returnval side-effect free-var))))
            ((eq (~kind @io) 'actual-arg)
             (assert-fact (&~name (make-fact-be-out @io) name)))
            (t
             (barf "5ANNOTATE-VAR-DFLOW: Bad dest @io.~% Data: ~:=" @io))))))

;this sets things up compatably for returnval nesting to be used.

(defun 5annotate-return-dflow (region)
  (let* (((source nil nil intermediates . dests) region)
         (source@io (car (??& source @io)))
         (dest@io (car (??& (car dests) @io))))
    (prog ()
      (if (cdr dests) (return nil))
      (if (not (or (null source@io) (eq (~kind source@io) 'returnval)))
          (return nil))       ;maybe can relax this!
      (if (Ror-fast (input-fact? (Elist intermediates)))
          (return nil))
      (cond ((null dest@io) (>& (car dests) @io 'returnval nil nil))
            ((memq (~kind dest@io) '(returnval side-effect free-var)))
            ((eq (~kind dest@io) 'actual-arg)
             (assert-fact (&~name (make-fact-be-out dest@io) nil)))
            (t (return nil)))
      (if source@io (make-fact-be-out source@io))
      (>& source @io 'returnval nil nil)
      (dolist (port intermediates)
        (>& port @io 'returnval nil nil))
      (return T))))
```

```
;this runs over the plan and checks on some final compatabilites
;1. it checks to see that the inputs to a rec are annotated the same way
;   as the inputs of the seg it is an instance of.  (Note that in loops
;   what the outputs are like is ignored)
;2. it checks that the corresponding ports on a join are annotated the same
;   if not, then the inputs are changed to fix the problem.
;3. it checks that a side-effected output has the same var name as its
;   input (if it has any dflow out of it at all).

(defun 5cleanup (seg)
  (over (??subseg seg) -> subseg
        (?recursive-supseg subseg) -> supseg
    (cond (supseg
           (over (??& (&* (&* subseg) input) @io) -> in@io
                 (?& (&* (&* supseg) input *
                         (~obj (~deep in@io))) @io) -> top@io
             select (or (not (eq (~kind top@io) (~kind in@io)))
                        (not (eq (~name top@io) (~name in@io))))
             (make-fact-be-out in@io)
             (assert-fact (&~deep top@io (~deep in@io)))))
;
          ((eq (?segtype subseg) 'join)
           (dolist (out@io (??& (&* (&* subseg) output) @io))
             (let ((in@ios (??& (&* (&* subseg) input 'id
                                    * (~obj (~deep out@io))) @io)))
               (dolist (in@io in@ios)
                 (when (or (not (eq (~kind out@io) (~kind in@io)))
                           (not (eq (~name out@io) (~name in@io))))
                   (make-fact-be-out in@io)
                   (assert-fact (&~deep out@io (~deep in@io))))))))
;
          (t (over (??& (&* (&* subseg) output) @io 'side-effect) -> se@io
                   (?& (~id-port (~deep se@io)) @io) -> in@io
             (when (and (??& * dflow (~deep se@io))
                        (~name in@io)
                        (not (eq (~name se@io) (~name in@io))))
               (assert-fact (&~name (make-fact-be-out se@io)
                                    (~name in@io)))
               (>& (~deep se@io) @createvar (~name se@io))))))
    (5cleanup subseg)))
```

```
;some helping fns

(defun 5terminal? (seg)
  (memoizeq 5TERMINAL? seg (null (??subseg seg))))

(defun 5ptype (seg)
  (memoizeq 5PTYPE seg (?plantype seg)))


;these just make calling the order things more concise

(defun 5port>order-id (port)
  (5order-id (~segcase port) (~keyword port)))

(defun 5port-precedes? (porta portb)
  (5precedes? (5order-id (~segcase porta) (~keyword porta))
              (5order-id (~segcase portb) (~keyword portb))))

(defun 5port-make-precede (porta portb)
  (5make-precede (5order-id (~segcase porta) (~keyword porta))
                 (5order-id (~segcase portb) (~keyword portb))))

;---------------------------------------

;this fn manages a little data structure (NAME-CLASSES) which indicates
;other regions which can be used to give you an idea of what name to use,
;if you don't get a name from the region itself.  Each entry is of the form
; (region1 priority region2)
;the priority tells you how likely (the bigger the better 0-100) it is
;that looking at region2 is a good idea.

;these tripples are just pushed onto the special variable (this is done in
;5determine-name-class)

;this fn gives you an ordered list that transatively
;(depth first) goes through the structure and tells you
;every region you might want to look at.  it takes
;advantage of the fact that regions are side-effect and
;therefore stay EQ over time.

(declare (special SO-FAR))

(defun 5get-related-regions (region min)
  (let ((SO-FAR [region]))
    (5get-related-regions1 region min)
    (cdr (nreverse SO-FAR))))

(defun 5get-related-regions1 (region min)
  (let* (hits rs)
    (over NAME-CLASSES -> [region1 num region2]
      select (> num min)
      (cond ((eq region region1) (push [num ! region2] hits))))
    (setq hits (sortcar hits #'<))
    (setq rs (over hits revlist (cdr _)))
    (dolist (r rs)
      (when (not (memq r SO-FAR))
        (push r SO-FAR)
        (5get-related-regions1 r min)))))
```

```
;this group of functions makes it fast to keep track of an acyclic
;partial order.  Which can increase in specificity, but cannot decrease
;(eg no arcs can be removed.)  The basic algorithm works on keeping a
;relation defined on numbers.  alists are used to convert non-numeric
;things into proper numeric indices.  the relation is encoded in bit strings
;in vectors.  these bit strings are represented as lists of fixnums
;(36 bits each).  The algorithm runs off of the following
;special variables
;5MAXID - the largest number id used so far (0 is the first one used)
;5NEXTBIT - (1+ (\ (1+ 5MAXID) 5LIMIT)) the bit counting from 1 corresponding
;        to (1+ 5MAXID) taken mode 5LIMIT so we know when to add length.
;5BIT-PATTERN - 2**(5MAXID+2) the bit string corrsponding to 5NEXTBIT
;        (a bignum)
;5LIMIT - specifes how many items are crammed into each element of
;        the bit string lists.  it is also used for the descretization
;        on the size of the arrays used.
;5PATTERN - array of bignum bit stings this shows the bit pattern corresponding
;        to the index fetching it.
;5RELATED - array of list of fixnums this is the or of the bit pattern for
;        every index the fetching index is FOLLOWED by.  they are stored this
;        way instead of through who precedes, in order to make
;        5any-follow fast.
;5IMMEDIATE - array of list of indices this is a list of each of the indices
;        the fetching index is immediately related to.
;5FLAG - array of T/NIL used by 5close when it is computing the transitive
;        closure of the ordering.
;5IDS - alist encodeing the relationship between names and indeces.
;5CLOSED - T if 5RELATED is known to be completely accurate.

(arraydefs (5flag t) (5immediate t) (5pattern t) (5related t))

(defun 5init-order ()
  (setq 5LIMIT       23. ;makes debugging easier due to no minus!
        5FLAG        (*array nil t 5LIMIT)
        5IMMEDIATE   (*array nil t 5LIMIT)
        5PATTERN     (*array nil t 5LIMIT)
        5RELATED     (*array nil t 5LIMIT)
        5MAXID       -1
        5NEXTBIT     1
        5BIT-PATTERN (list 1)
        5PLY         1
        5IDS         nil
        5CLOSED      t))

(defun 5order-id (segcase key)
  (let* ((seg (~seg segcase)) (case (~case segcase)))
    (setq key (cond ((and (null case) (5terminal? seg)) seg)
                    (t [seg key ! case])))
    (or (cdr (assoc key 5IDS)) (5new-id key))))

;decoders for debugging

(defun 5bd (bit-pattern)
  (5nd (over i <- 5maxid by (1- i) until (minusp i)
         select (5bit^ (5pattern i) bit-pattern)
         bag i)))

(defun 5nd (num) ;number or list of numbers
  (cond ((numberp num)
         (over 5IDS -> [key ! i]
               (if (= i num) (return key))))
        (t
         (over num bag (5nd _)))))
```

```
(defun 5new-id (key)
  (when (> 5NEXTBIT 5LIMIT)
    (setq 5PLY         (1+ 5PLY)
          5NEXTBIT     1
          5BIT-PATTERN [1 0 ! (cdr 5BIT-PATTERN)])
    (let ((i (* 5LIMIT 5PLY)))
      (adjust-array-size 5FLAG i)
      (adjust-array-size 5IMMEDIATE i)
      (adjust-array-size 5PATTERN  i)
      (adjust-array-size 5RELATED  i))
    (over i <- 5MAXID by (1- i) until (minusp i)
       (setf (5PATTERN i) (cons 0 (5PATTERN i)))
       (setf (5RELATED i) (cons 0 (5RELATED i)))))
  (setq 5MAXID (1+ 5MAXID))
  (setf (5PATTERN 5MAXID) (append 5BIT-PATTERN nil))
  (setf (5RELATED 5MAXID) [0 !! (cdr 5BIT-PATTERN) ! nil])
  (setq 5NEXTBIT (1+ 5NEXTBIT))
  (setf (car 5BIT-PATTERN) (lsh (car 5BIT-PATTERN) 1))
  (push [key ! 5MAXID] 5IDS)
  5MAXID)

;note that we are winning extra big here because the
;calls on 5add-immediate and on 5ordered are both much
;more frequent that the calls on 5close.  further 5close
;can be very fast because the ordering is known to be
;non-cyclic and no immediate orderings are ever removed
;from the ordering.

(defun 5make-precede (ida idb) ;assumes (not (5ordered idb ida))
  (cond ((not (or (= ida idb)                ;all to save time in 5close
                  (5precedes? ida idb)
                  (member idb (5immediate ida))))
         (setq 5CLOSED nil)
         (push idb (5IMMEDIATE ida)))))

(defun 5precedes? (ida idb)
  (cond ((= ida idb) nil)
        (t (5bit^ (5PATTERN idb) (5RELATED ida)))))

(defun 5bit^ (bitsa bitsb)
  (over bitsa -> a
   over bitsb -> b
    fastor (not (zerop (logand a b)))))

(defun 5close ()
  (cond ((not 5CLOSED)
         (over i <- 5MAXID by (1- i) until (minusp i)
               (setf (5FLAG i) nil))
         (over i <- 5MAXID by (1- i) until (minusp i)
               (5CLOSE1 i))
         (setq 5CLOSED t))))

(defun 5close1 (i)
  (let ((related (5RELATED i)))
    (when (not (5FLAG i))
      (setf (5FLAG i) t)
      (over (5IMMEDIATE i) -> j
            (5close1 j) -> bits
        (over b <- bits
         over rel <- related
         over p <- (5PATTERN j)
          (rplaca rel (logior (car b) (car rel) (car p))))))
    related))
```

```
;these are some extended fns used by 5determine-nesting

(defun 5make-multiple-pattern (segs)
  (repeat ((result
             (copylist (5PATTERN (5order-id (& (car segs)) 'input)))))
    (setq segs (cdr segs))
    (when (null segs) (return result))
    (over r <- result
     over p <- (5PATTERN (5order-id (& (car segs)) 'input))
      (rplaca r (logior (car r) (car p))))))

(defun 5remove-from-pattern (seg pattern)
  (over r <- pattern
   over p <- (5PATTERN (5order-id (& seg) 'input))
    (rplaca r (boole 4 (car r) (car p))))
  pattern)

;this depends on the fact that 5RELATED holds who follows the seg.

(defun 5any-follow (seg pattern)
  (5bit^ (5RELATED (5order-id (& seg) 'output)) pattern))
```

```
;this takes a piece of plan which is already annotated with info in
;@seg, @io, and @creatvar assertions, and conses up actual LISP code
;corresponding to it.  NOTE that there are
;several key assumptions about what the annotation must look like. eg.
;this program runs locally, and assumes that the annotation already
;assures global consistancy.

(defun 5codetop (seg mode)
  (let ((code (5codeup-top seg)))
    (caseq mode
      (define-cliche
        (let* ((cliche-defF (?& (&* seg) cliche-def))
               (comments
                 (append '(nil) (~cliche-def-comments cliche-defF) nil)))
          (remprop comments 'role-tree)
          (remprop comments 'roles)
          (setq code
                '(define-cliche ,(~cliche-def-name cliche-defF)
                    ,(cdr comments)
                  ,@code))
          (>@seg-code seg [code])))
      (defun
        (let* ((name (?@seg-name seg)))
          (setq code '(defun ,name ,@(cdar code)))
          (>@seg-code seg [code])))
      (T (>@seg-code seg code)))
    (mapc #'5coder-post-phase-hacks (?@seg-code top-seg))
    (5clean-noops seg)))

;YOU MUST BE VERY CAREFULL HERE NOT TO LET PROGRAM CONSTANT LISTS
;GET INTO THE DATA BASE!

(defun 5coder-post-phase-hacks (code)
  (DECLARE (SPECIAL *input-language*))
  (if (and (listp code) (eq (car code) 'defun)) (5fix-depending-on code))
  (caseq *input-language*
    (lisp (5with-open-file-hack code)
          (5let-hack code)
          (5pretty-conds code)
          (5supress-output-roles code))
    (ada (5supress-output-roles code)
         (5no-cond-rets code)
         (cleanup-ada-code code))))

;this is here because we use a non-standard deffn of LET in KBE:
;note that we should do something special to get a vanilla LET
;in the code above.  We should have these format things somewhere else.

(pp:define-same-as kbe:let global:let)
(pp:define-same-as kbe:let* global:let*)
```

```
;this looks over the code and finds any instances of
;(setq var (cond ...)) if it finds any, then it pushes
;the setq into each clause

(defun 5no-cond-rets (thing)
  (cond ((and (listp thing) (not (eq (car thing) 'quote)))
         (5ncr-1 thing)
         (do ((it thing (cdr it))) ((or (not (listp it)) (null it)))
           (5no-cond-rets (car it))))))

(defun 5ncr-1 (thing)
  (when (and (eq (car thing) 'setq)
             (eq (length thing) 3)
             (listp (caddr thing))
             (eq (caaddr thing) 'cond))
    (let* ((clauses (cdaddr thing))
           (var (cadr thing)))
      (mapc #'(lambda (clause)
                (let ((last (last clause)))
                  (rplaca last (list 'setq var (car last)))))
            clauses)
      (rplacd thing clauses)
      (rplaca thing 'cond))))
```

```
;This looks for instances of (SETQ var {code, output name})
;and converts them to just (SETQ var code).
;It also looks for instances of {var, output name} which are
;nested as functional arguments, and converts them to just var.

(defvar 5list-car nil)

(defun 5supress-output-roles (thing)
  (let ((5list-car nil))
    (5supress-output-roles0 thing)))

(defun 5supress-output-roles0 (thing)
  (cond ((and (listp thing) (not (eq (car thing) 'quote)))
         (5sor-1 thing)
         (let ((5list-car (car thing)))
           (do ((it thing (cdr it))) ((or (not (listp it)) (null it)))
             (5supress-output-roles0 (car it)))))))

(defun 5sor-1 (thing)
  (when (and (eq (car thing) 'setq)
             (symbolp (cadr thing))
             (listp (caddr thing)))
    (cond ((and (eq (car (caddr thing)) 'kbe-/#-comment-macro)
                (memq 'output (caddr (caddr thing))))
           (rplaca (cddr thing) (caadr (caddr thing))))
          ((and (eq (car (caddr thing)) '5noop)
                (listp (cadr (caddr thing)))
                (eq (car (cadr (caddr thing))) 'kbe-/#-comment-macro)
                (memq 'output (caddr (cadr (caddr thing)))))
           (rplaca (cddr thing) (caadr (cadr (caddr thing))))))
    (when (or (eq (cadr thing) (caddr thing))
              (and (listp (caddr thing))
                   (eq (caaddr thing) '5noop)
                   (eq (cadr thing) (cadr (caddr thing)))))
      (rplaca thing '5noop)
      (rplacd thing nil)))
  (when (and (eq (car thing) 'kbe-/#-comment-macro)
             (memq 'output (caddr thing))
             (or (symbolp (caadr thing))
                 (and (listp (caadr thing))
                      (eq (car (caadr thing)) '5noop)
                      (symbolp (cadr (caadr thing)))))
             (symbolp 5list-car)
             (not (memq 5list-car '(loop progn let)))
             (or (fboundp 5list-car)
                 (string-equal "ADA-FN~" 5list-car
                               0 0 7 (min 7 (string-length 5list-car)))
                 (string-equal "ADA-SELECT~" 5list-car
                               0 0 11
                               (min 11 (string-length 5list-car)))))
    (rplaca thing '5noop)
    (rplacd thing (ncons (caadr thing))))
  (when (and (eq (car thing) '5noop)
             (listp (cadr thing))
             (eq (car (cadr thing)) 'kbe-/#-comment-macro)
             (memq 'output (caddr (cadr thing)))
             (or (symbolp (caadr (cadr thing)))
                 (and (listp (caadr (cadr thing)))
                      (eq (car (caadr (cadr thing))) '5noop)
                      (symbolp (cadr (caadr (cadr thing))))))
             (symbolp 5list-car)
             (not (memq 5list-car '(loop progn let)))
             (or (fboundp 5list-car)
                 (string-equal "ADA-FN~" 5list-car
                               0 0 7 (min 7 (string-length 5list-car)))))
    (rplaca thing '5noop)
    (rplacd thing (ncons (caadr (cadr thing))))))
```

```
;this looks for SETQs inside of a let and tries to bring
;them inside as LET* initializations.

(defun 5let-hack (thing)
  (cond ((and (listp thing) (not (eq (car thing) 'quote)))
         (5lh-1 thing)
         (do ((it thing (cdr it))) ((or (not (listp it)) (null it)))
           (5let-hack (car it))))))

(defun 5lh-1 (thing)
  (when (eq (car thing) 'let)
    (let* ((vars (cadr thing))
           (statements (cddr thing))
           pairs extra-statements)
      (do () (NIL)
        (cond ((and (listp (car statements))
                    (eq (caar statements) 'setq)
                    (memq (cadar statements) vars))
               (let ((var (cadar statements)))
                 (push (list var (caddr (pop statements))) pairs)
                 (setq vars (delq var vars))))
              ((and (listp (car statements))
                    (eq (caar statements) 'format))
               (push (pop statements) extra-statements))
              (T (return))))
      (setq statements (nreconc extra-statements statements))
      (setq pairs (append (mapcar #'ncons vars) pairs))
      (setq pairs (partial-order-sort (sort pairs #'5lh-lessp)
                                      #'5lh-order))
      (rplaca thing 'let*)
      (rplaca (cdr thing) pairs)
      (rplacd (cdr thing) statements)))
  thing)

(defun 5lh-lessp (x y)
  (cond ((string-equal (car x) "DATA") nil) ;shouldn't be so specific
        ((string-equal (car y) "DATA") T) ;shouldn't be so specific
        (T (alphalessp (car x) (car y)))))

;note you must use partial-order-sort with this

(defun 5lh-order (pair1 pair2)
  (5po1 (cadr pair2) (car pair1)))
```

```
;this takes all of the DEPENDS-ON forms out of a DEFUN

(defun 5fix-depending-on (thing)
  (cond ((and (listp thing) (not (eq (car thing) 'quote)))
         (do ((it thing (cdr it))) ((or (not (listp it)) (null it)))
           (5fd-1 it)
           (5fix-depending-on (car it))))))

(defun 5fd-1 (list)
  (when (and (listp (car list)) (eq (caar list) 'depending-on))
    (let ((first-arg (cadr (car list))))
      (cond ((or (symbolp first-arg)
                 (and (listp first-arg)
                      (eq (car first-arg) '5noop)
                      (symbolp (cadr first-arg))))
             (rplaca list (caddr (car list))))
            (T (warn "cannot handed depending-on a non variable yet"))))))

;this finds all of the conds in a chunk of code and
;changes some to WHENs and IFs.

(defun 5pretty-conds (thing)
  (cond ((and (listp thing) (not (eq (car thing) 'quote)))
         (5pc-1 thing)
         (do ((it thing (cdr it))) ((or (not (listp it)) (null it)))
           (5pretty-conds (car it))))))

(defun 5pc-1 (thing)
  (when (eq (car thing) 'cond)
    (let* ((clauses (cdr thing))
           (first-clause (car clauses))
           (second-clause (cadr clauses)))
      (cond ((and (null (cdr clauses))
                  (cdr first-clause)
                  (null (cddr first-clause)))
             (rplaca thing 'if)
             (rplacd thing first-clause))
            ((and (null (cdr clauses))
                  (cddr first-clause))
             (rplaca thing 'when)
             (rplacd thing first-clause))
            ((and (null (cddr clauses))
                  (cdr first-clause)
                  (null (cddr first-clause))
                  second-clause
                  (eq (car second-clause) 'T)
                  (cdr second-clause)
                  (null (cddr second-clause)))
             (rplaca thing 'if)
             (rplacd thing (nconc first-clause (cdr second-clause))))))))
```

```
;this runs around and puts in a with-open-file
;for any file that is opened with a WITH-OPEN-FILE-OPEN

(defun 5with-open-file-hack (code)
  (cond ((and (listp code) (memq (car code) '(defun define-cliche)))
         (let* (((nil nil nil . body) code)
                (let? (and (null (cdr body))
                           (listp (car body))
                           (eq (caar body) 'let)))
                (let-vars (if let? (cadar body)))
                (file-tripples (5find-io-L body)))
           (cond (file-tripples
                  (letS ((var (car (Elist file-tripples))))
                    (setq let-vars (delq var let-vars)))
                  (cond (let?
                         (rplaca (cdar body) let-vars)
                         (cond ((null let-vars)
                                (rplaca body (car (cddar body)))
                                (rplacd body (cdr (cddar body))))))))
                  (let (((first . rest) body))
                    (rplaca body (car (5make-with-open-file-forms
                                        file-tripples (cons first rest))))
                    (rplacd body nil)))))))
  code)

(defun 5make-with-open-file-forms (file-tripples body)
  (if file-tripples
      '((with-open-file ,(car file-tripples)
          ., (5make-with-open-file-forms (cdr file-tripples) body)))
      body))
```

```
(declare (special 5opens-L))

(defun 5find-io-L (thing)
  (let (5opens-L)
    (5fio-L1 thing)
    (sort 5opens-L #'5cio-order-L)))

(defun 5fio-L1 (thing)
  (cond ((and (listp thing) (not (eq (car thing) 'quote)))
         (5fio-L thing)
         (do ((it thing (cdr it))) ((or (not (listp it)) (null it)))
           L (5fio-L1 (car it))
             (cond ((and (listp (car it)) (eq (caar it) '5kill-this))
                    (rplaca it (cadr it)) ;must not be last!
                    (rplacd it (cddr it))
                    (go L)))))))

(defun 5fio-L (list)
  (when (and (eq (car list) 'setq)
             (listp (caddr list))
             (symbolp (caaddr list))
             (equal (string (caaddr list)) "WITH-OPEN-FILE-OPEN"))
    (rplaca list '5kill-this)
    (push (list (cadr list)
                (cadr (caddr list))
                (caddr (caddr list)))
          5opens-L)))

(defun 5cio-order-L (io1 io2)
  (let* (((var1 file1 code1) io1)
         ((var2 file2 code2) io2))
    (cond ((equal code1 code2) (alphalessp file1 file2))
          (T (memq (cadr code2)
                   (cdr (memq (cadr code1) '(:in :out))))))))
```

```
;This runs over the code and gets rid of all instances
;of the fn 5NOOP.  it then updates the @seg assertions to
;reflect these changes, and to get rid of any bits of
;code that aren't really in the program at all (really
;this shouldn't ever happen, but I think it does now and
;then).  It also gets rid of calls on 5noop where the value isn't used.
;This also splices out calls on 5noop with no args completely.

(declare (special 5NOOP-TRAN ALL-CODE))
(def-alistq 5NOOP-TRAN)

(defun 5clean-noops (top-seg)
  (let* (5NOOP-TRAN ALL-CODE
         (code (?@seg-code top-seg)))
    (5clean-noops1 code nil nil)
    (over (all-subsegs top-seg) -> seg
          (car (??& (&* seg) @seg)) -> @seg
          (and @seg (~code @seg)) -> seg-code
          (over seg-code -> c
                (or (5NOOP-TRAN c) (car (memq c ALL-CODE))) -> new-c
            select new-c list new-c) -> new-seg-code
      select (not (qset= seg-code new-seg-code))
      (assert-fact (&~code (make-fact-be-out @seg) new-seg-code)))))

(defun 5clean-noops1 (code prior-code parent-code)
  (push code ALL-CODE)
  (cond ((and code (listp code)) ;note maclisp (listp nil) = T
         (cond ((and (listp (car code)) (eq (caar code) '5noop))
                (cond ((null (cdar code))
                       (cond (prior-code (rplacd prior-code (cdr code)))
                             (parent-code (rplaca parent-code (cdr code))))
                       (setq code (cdr code)))
                      (t (setf (5NOOP-TRAN (car code))
                               ['*code-is-car* ! code])
                         (rplaca code (cadar code)))) ;side-effect vital
                (5clean-noops1 code prior-code parent-code))
               (T (5clean-noops1 (car code) nil code)
                  (if (cdr code) (5clean-noops1 (cdr code) code nil)))))))
```

```
;This takes an annotated seg and makes inline code for it.  It stores
;a list of this code in CODE-FOR.  It also
;returns this list.  It is assumed that any ports which supply data to
;this seg have already been coded.

;This first part of the fn takes care of external interfacing

(defun 5codeup-top (seg)
  (let ((local-vars
         (sort (qset- VAR-NAMES-USED
                      (over (??& (&* (&* seg) port) @io)
                        select (memq (~kind _) '(lambda-arg free-var))
                        bag (~name _)))
                    #'alphalessp)))
    (cond ((??& (&* seg) @seg 'lambda-exp) (5codelambda seg local-vars))
          (T (5codeopencode seg local-vars)))))

(defun 5codeopencode (seg local-vars)
  (let ((final-code (5codeup seg T)))
    (if local-vars
        (setq final-code '((let ,local-vars ,@ final-code))))
    (>@seg-code seg final-code)
    final-code))

(defun 5codelambda (seg local-vars)
  (let* ((arglist (over (sort (??& (&* (&* seg) input) @io 'lambda-arg)
                              #'(lambda (x y)
                                  (< (~argno x) (~argno y)))) -> lin
                        list (~name lin)))
         (code (5codeupinner seg)))
    (if local-vars
        (setq code '((let ,local-vars ,@ code))))
    (setq code [['lambda arglist ! code]])
    (>@seg-code seg code)
    code))
```

```
;this is called from a lot of places!

(defun 5codeup (seg &optional (top? nil))
  (let* (final-code)
    ;note coding must happen in three steps to get retvals right
    (setf (CODE-FOR seg)
          (append (if (not top?) (5assign (& seg) 'outside-input))
                  (5codeupinner seg)))
    (setq final-code (5assign (& seg) 'outside-output))
    ;side-effects force the above line to be seperate (see 5REP)
    (setq final-code
          (cond ((car (??& (&* (&* seg) output) @io 'returnval))
                 (5make-prog2 (CODE-FOR seg) final-code))
                (t
                 (append (CODE-FOR seg) final-code))))
    (setf (CODE-FOR seg) final-code)
    final-code))
```

```
;this fn codes up the stuff corresponding to the innards of the seg.
;it is not expected to be called with an ssr-body.
;it is not expected to be called with a join, or recursion.
;these only appear in xors and ssrs and are taken care of elsewhere.
;it can get called with a pred if a cliche with a naked top level
;pred is being coded up.  In that case, it cludges up a cond-like
;thing to print for the pred.

(defun 5codeupinner (seg)
  (let* ((name (?@seg-name seg)) code)
   (caseq (?@seg-kind seg)
      ((funcall)
       (setq code
             [[(or name '...)
               !(over (sort (??& (&* (&* seg) port) @io 'actual-arg)
                            #'(lambda (x y) (< (~argno x) (~argno y))))
                      -> lin
                 append (cond ((input-fact? (~deep lin))
                               [!(if (and (null name)
                                          (??& (&* (&* seg) output 'id *
                                                   (~obj (~deep lin)))
                                               @io 'side-effect))
                                     ['&in-out])
                                 (5rep1 (5source lin))])
                              (t [!(if (null name) ['&out])
                                   (~name lin)]))]])
       (cond ((or (?empty-slot seg) (??& * slot (&* seg) * 'output))
              (setq code (make-kbe-/#-comment code seg))))
       (>@seg-code seg code)
       code)
      ((constant)
       (setq code (cond ((or (memq (cadr name) '(t nil))
                             (numberp (cadr name))
                             (stringp (cadr name)))
                         [['5noop (copytree (cadr name))]])
                        (t [(copytree name)])))
       (>@seg-code seg code)
       code)
      ((opencode lambda-exp)
       (caseq (?plantype seg)
         (xor [!!(5assign (& seg) 'inside-input) !!(5codexor seg)])
         (ssr (5codessr (5assign (& seg) 'inside-input) seg nil))
         ((nil expr and comp)
          [!!(5assign (& seg) 'inside-input) !!(5codesimple seg)])
         (pred
           [!!(5assign (& seg) 'inside-input) !!(5codepred seg nil)])
         (t (barf "5CODEUPINNER: Unexpected plantype.~% Seg: ~:="
                  seg))))
      (t (barf "5CODEUPINNER: Unexpected @seg-kind.~% Seg: ~:=" seg)))))

(defun make-kbe-/#-comment (code seg)
  (let* ((name (external-slot-name seg)))
    (cond ((equal code '((...))) (setq code nil)))
    '((kbe-/#-comment-macro ,code ,name))))

;this returns a name list to be put in a kbe-/#-comment-macro

(defun external-slot-name (seg)
  (let* ((slot-fact (~01 (??& * slot (&* seg))))
         (slot-name (~slotname slot-fact))
         (name (make-external-name-list slot-name))
         (cliche-fact (if slot-fact
                          (~01 (??& (~segcase slot-fact) cliche)))))
    (if (eq (~clichename cliche-fact) 'pseudo-subcliche)
        (setq name '(,@name of the ,@(external-slot-name
                                       (~seg~segcase cliche-fact)))))
    (if (memq (~slottype slot-fact) '(input output))
        (push (~slottype slot-fact) name))
```

```
    (cons 'the name)))

(defun make-external-name-list (name-symbol)
  (let ((first-blank (string-search-char #\space name-symbol)))
    (if (null first-blank) (ncons name-symbol)
        (let* ((name-string (string-append (string name-symbol) " ")))
          (letS* ((blank-spot
                    (enumerateS
                      #'null
                      #'(lambda (x)
                          (string-search-char #\space name-string (1+ x)))
                      first-blank)))
            (Rlist (intern (string-upcase
                             (substring name-string
                                        (1+ (Pvalue blank-spot -1))
                                        blank-spot))
                           (symbol-package name-symbol)))))))))
```

```
;this fn takes care of the three plantypes which can be implemented
;with no splits or joins of control flow.  the only complexity is
;the way things get nested together.  the annotator better make sure
;that this is going to work out ok.

;one other funny thing that goes on is that it works pretty hard to
;nest any computations that happen as exit computations after a loop,
;back into the loop.  This is controlled by a flag which is turned
;off by default because that looks better in the current demo.

(defvar *nest-loop-epilog* nil
  "controls inclusion of loop epilog code in the terminating cond")

(defun 5codesimple (seg)
  (prog (returnval@io subsegs subseg c code action-code exit-code plantype)
    (setq plantype (?plantype seg))
    (setq returnval@io (car (??& (&* (&* seg) output) @io 'returnval))
          subsegs (sort (~nset ~subseg (??& (&* seg) subseg))
                        #'(lambda (x y)
                            (5precedes? (5order-id x 'output)
                                        (5order-id y 'input)))))
 lp (if (null subsegs)
        (return (5simple-stuff-at-end seg code)))
    (setq subseg  (~seg (car subsegs))
          subsegs (cdr subsegs))
    (cond ((and *nest-loop-epilog*
                (not (eq plantype 'and))
                (eq (?plantype subseg) 'ssr)
                (5put-here subseg)
                (over subsegs
                  fastand (not (eq (?plantype (~seg _)) 'ssr))))
           (setq action-code (over subsegs -> segcase
                                   (~seg segcase) -> s
                                   (5codeup s) -> c
                                 select (5put-here s)
                                 append c))
           (cond (returnval@io
                  (setq exit-code
                        (5make-prog2 (5rep (5source returnval@io))
                                     (5assign (& seg) 'inside-output))))
                 (t
                  (setq action-code
                        (append action-code
                                (5assign (& seg) 'inside-output)))))
           (return [!! code
                    !!(5codessr nil subseg
                                [[nil action-code ! exit-code]])]))
          (t
           (setq c (5codeup subseg))
           (if (5put-here subseg) (setq code (append code c)))
           (go lp)))))

;this does some simple finishing up stuff at the end of things
;to make sure that the seg gets the right ~code field, and that
;the returnval and free-variables are right.

(defun 5simple-stuff-at-end (seg inside-code)
  (let* ((returnval@io (car (??& (&* (&* seg) output) @io 'returnval)))
         (ret-code (if returnval@io (5rep (5source returnval@io)))))
    (>@seg-code seg [!! inside-code !! ret-code])
    [!! inside-code
     !! (5make-prog2 ret-code (5assign (& seg) 'inside-output))]))
```

```
;Now we get to the first hard part.  coding up CONDs.
;This assumes that the inside of every pred may have an
;init, and may have >1 inner preds.  However, the inner
;preds must each be binary, and the inner connectivity
;must be a tree if there are more than 2 cases on the
;outer seg.  this insures that the xor can be coded
;simply with a cond.

;an important data structure used here is "case-action"
;this shows what code is to be executed for each case of
;the cond.  (note that the actions actually get coded up
;before the preds do.  this is ok since the actions don't
;get dflow from the preds.) the form of case-action is
;[case action-code ! ret-code] (note action-code is just
;executed, while  ret-code has a value to return [only
;code-ssr actually cares about this  distinction]). Note
;that each of these is a LIST of sexprs!

(defun 5codexor (seg)
  (let* ((roles (??roles seg))
         (join (car (roles 'join)))
         (mpred (car (roles 'pred)))
         (mpred-segcases (~nset ~from (??& (&* seg) cflow (&* mpred))))
         (initcode (over (roles 'init) -> i
                         (5codeup i) -> c
                   select (5put-here i)  append c))
         (case-action
          (over mpred-segcases -> mpsc
                (~case mpsc) -> case
                (~to (?& (&* seg) cflow mpsc)) -> actioncase
                (cond ((memq (~seg actioncase) (roles 'join))
                       (prog1 actioncase (setq actioncase nil)))
                      (t
                       (~to (?& (&* seg) cflow actioncase)))) -> joincase
                (if actioncase (5codeup (~seg actioncase))) -> actioncode
                (5assign joincase 'outside-input) -> jcode
                (car (??& (&* joincase input) @io 'returnval)) -> ret
                (if ret (5rep (5source ret))) -> retcode
             bag (if (not (and actioncode (5put-here (~seg actioncase))))
                     (setq actioncode nil))
                 (cond (retcode (setq retcode (5make-prog2 retcode jcode)))
                       (t (setq actioncode [!! actioncode !! jcode])))
                 [case actioncode ! retcode]))
         (basic-code [!! initcode !! (5codepred mpred case-action)]))
    (setf (CODE-FOR join) basic-code) ;a bit of a kludge
    (5simple-stuff-at-end seg (if (5put-here join) basic-code))))
```

```
;this codes up a pred to get the indicated actions done.  this is done
;either with a cond, or if the pred is an ssr, by pushing the actions
;into the loop exits.  it assumes all primitive preds are binary.

;Note that this doesn't use CODE-FOR but rather case-action in order
;to remember what code goes with what pred.
;also note that complex preds don't get their code fields set untill
;the final claenup pass at the very end.

(declare (special join-info))

(defun 5codepred (pred case-action)
  (let* (code then thencase else elsecase)
    (cond ((eq (?@seg-kind pred) 'funcall)
           (desetq ((thencase . then) (elsecase . else))
                   (5case-action-sort case-action
                                      (5need-not
                                        (& pred (caar case-action)))))
           (setq then [!!(car then) !!(cdr then)]
                 else [!!(car else) !!(cdr else)])
           (setq code (car (5codeupinner pred)))
           (if (5need-not (& pred thencase)) (setq code ['not code]))
           (setq code
                 . [['cond
                     [code ! then]
                     !(cond ((5null-computation else) nil)
                            ((and (listp (car else)) (eq (caar else) 'cond)
                                  (null (cdr else))
                                  (over (and (boundp 'join-info) join-info)
                                        fastand (not (equal else (cdr _)))))
                             (cdar else))
                            (t [[t ! else]]))]]))
          ((eq (?plantype pred) 'ssr)
           (setq code
                 (5codessr (5assign (& pred) 'outside-input)
                           pred
                           case-action)))
          (t (let* ((roles (??roles pred))
                    (icode (over (roles 'init) -> iseg
                                 (5codeup iseg) -> c
                                 select (5put-here iseg)
                                 append c))
                    (first-predcase (~to (?& (&* pred) cflow (&* pred))))
                    (join-info (over (roles 'join) bag [_])))
               (setq code (5codepred2 pred first-predcase case-action))
               (if join-info (setq code (5consolodate-cond code)))
               (setq code (append icode code)))))
    code))

(defun 5null-computation (stuff)
  (member stuff '(nil (nil) ((5noop nil)) ('nil) ('(5noop nil)))))

(defun 5need-not (segcase)
  (over (or (~nset ~wff (??& segcase when))
            (~nset ~wff (??& segcase assert))) -> wff
    fastor (and (not (atom wff)) (eq (car wff) 'not))))
```

```
;this fn looks at a two element case action, and decides which of the
;two alternatives is the more complex.  The complex alternative is
;then put in the front.  the second argument says whether the first predicate
;will need a NOT (ie will be rather complex itself) when coded up.

(defun 5case-action-sort (case-action negativethen)
 (let* ((((thencase . then) (elsecase . else)) case-action))
  (setq then (append (car then) (cdr then))
        else (append (car else) (cdr else)))
  (let* ((condelse (and (listp (car else)) (eq (caar else) 'cond)
                        (null (cdr else))
                        (over (and (boundp 'join-info) join-info)
                          fastand (not (equal else (cdr _))))))
         (condthen (and (listp (car then)) (eq (caar then) 'cond)
                        (null (cdr then))
                        (over (and (boundp 'join-info) join-info)
                          fastand (not (equal then (cdr _))))))
         (emptyelse (or (null else)
                        (and (null (cdr else)) (null (car else)))))
         (emptythen (or (null then)
                        (and (null (cdr then)) (null (car then))))))
    (cond ((cond (condelse (and condthen negativethen))
                 (emptyelse (or condthen (and emptythen negativethen)))
                 (t (or condthen emptythen negativethen)))
           (reverse case-action))
          (t case-action)))))

;this does the work of coding up the insides of complex preds It
;recurses with 5codepred in a pattern which matchs the tree pattern of
;tests in the pred.  it plays tricky games with case-action.  What it
;returns is used as the last component of a case-action slot.
;  The funnyness is particularly great with regard to the way it plays
;with joins!  It copies whole hunks of code which get
;coalessed together as ANDs and Ors which are created by 5consolodate-cond.
;The special variable Join-info is used to hold the stuff to put
;in the cond.

(defun 5codepred2 (pred subpredcase case-action)
  (let* ((subpred (~seg subpredcase))  (subcase (~case subpredcase)))
    (cond ((eq pred subpred)                              ;an outside case
           [!!(car (case-action subcase))
            !!(cdr (case-action subcase))])
          ((assq subpred join-info)
           (or (cdr (assq subpred join-info))
               (cdr (rplacd (assq subpred join-info)
                            (5codepred2 pred
                                        (~to (?& (&* pred) cflow (&* subpred)))
                                        case-action)))))
          (t
           (5codepred subpred
                      (over (??& (&* pred) cflow (&* subpred)) -> cf
                       revlist [(~case (~from cf))
                                nil
                                !(5codepred2 pred (~to cf) case-action)])))))
```

```
;this funny little fn does the work of making ANDs and ORs.  it works by
;repeated application of the following two rules about successive options
;in a cond list.
;  (... (P ! A) (Q ! B) ...) <=> (... ((AND (NOT P) Q) ! B) (P ! A) ...)
;  (... (P ! A) (Q ! A) ...) <=> (... ((OR P Q) ! A) ...)
;  Note, it has some smarts about nesting of ORs and ANDs,
;     and about (OR P T) <=> T
; it uses the first rule to move two branches which came from the same join
;together, and then uses the last two rules to combine them together.  In
;the simple case of two inner preds, the ordering of the last two branchs
;controls whether an AND or OR is used.

;note that copying is done so that there will be no
;sharing in the resulting code.

(defun 5consolodate-cond (code)
  (let (cond-list)
    (setq cond-list (over (cdar code) list (cons (car _) (cdr _))))
    (repeat ((cl cond-list) head-pred head-action not-head-pred)
      (desetq (head-pred . head-action) (car cl))
      (when (null (cdr cl))
        (when (and (eq head-pred T)
                   (caar head-action)
                   (eq (caar head-action) 'cond))
          (setf (car cl) [(caadar head-action) ! (cdadar head-action)])
          (setf (cdr cl) (over (cddar head-action)
                               list (cons (car _) (cdr _)))))
        (return))
      (cond ((or (over (cdr cl) fastor (equal head-action (cdr _)))
                 (and (null (cddr cl)) (5null-computation head-action)))
             (dolist (item (cdr cl))
               (let (((pred . action) item))
                 (cond ((equal action head-action)
                        (cond ((eq pred T)) ;don't do anything
                              ((and (not (atom head-pred))
                                    (eq (car head-pred) 'or))
                               (rplaca item ['or !!(cdr head-pred) pred]))
                              (t
                               (rplaca item ['or head-pred pred])))
                        (return))
                       (t (setq not-head-pred
                                (cond ((and (not (atom head-pred))
                                            (eq (car head-pred) 'not))
                                       (cadr head-pred))
                                      (t ['not head-pred])))
                          (setq head-pred (copytree head-pred))
                          (cond ((and (not (atom pred)) (eq (car pred) 'and))
                                 (rplaca item
                                         ['and not-head-pred ! (cdr pred)]))
                                (t
                                 (rplaca item
                                         ['and not-head-pred pred])))))))))
             (rplaca cl (cadr cl))
             (rplacd cl (cddr cl)))
            (t (setq cl (cdr cl)))))
    [['cond ! cond-list]]))
```

```
;at the moment this only codes up ssrs which can be implemented as
;simple loops.  case-action is an alist showing the exiting action code
;(if any) corresponding to each exit case of the ssr.
;this assumes that the output form of the body, and the ssr are IDENTICAL
;ie things are just coded up for the body, and assumed to work for the ssr

;there is code here to deal with zero values being returned,
;but this is the only place.  We are really only trying to make
;a few simple cliches come out right.

;The following controls how loops are to be created.  It is a function
;which should take in a list of init code and a list of code to
;be executed in the body of the loop and should return a list of
;code corresponding to the loop.  Note that the body code will probably
;contains instances of RETURN.  Two example functions are shown first.

(defun 5codessr-prog (init body)
  '((prog nil ,@ init LP ,@body ,(copylist '(go LP)))))

;as soon as we can we will get rid of the "DO".

(defun 5codessr-loop (init body)
  '(,@ init (loop do ,@ body)))

(define-KBE-option *loop-code-template*
                   "Iteration style"
  '5CODESSR-LOOP :assoc '(("DO-Loop" . 5CODESSR-LOOP)
                          ("GO-Tags" . 5CODESSR-PROG)))
```

```
(defun 5codessr (precomputation ssr case-action)
  (let* ((roles (??roles ssr))
         (dfjoin (car (roles 'dfjoin)))
         (body (car (roles 'body)))
         (rec (car (roles 'rec)))
         (df-retval@io
           (car (??& (&* (& dfjoin 'body) input) @io 'returnval)))
         (icode (append precomputation
                        (over (roles 'init) -> i
                              (5codeup i) -> c
                              select (5put-here i)
                              append c)
                        (5assign (& body) 'outside-input)))
         code found-exit)
    (when (and df-retval@io case-action)
      (barf "5CODESSR: returnval and actions.~% ssr: ~S" ssr))
    (repeat ((subseg (~seg (~to (?& (&* body) cflow (&* body)))))
             incase outcase xcase action-code ret-code newsubseg)
      (cond ((eq subseg rec)
             (setq code (append code (5assign (& rec) 'outside-input)))
             (return))
            ((not (eq (?segtype subseg) 'split))
             (let* ((c (5codeup subseg)))
               (if (5put-here subseg)
                   (setq code (append code c)))
               (setq newsubseg
                     (~seg (~to (?& (&* body) cflow (&* subseg)))))))
            (t
             (dolist (cf (??& (&* body) cflow (&* subseg)))
               (cond ((eq (?segtype (~seg (~to cf))) 'join)
                      (setq outcase (~case (~from cf))
                            xcase
                             (~case
                               (~to (?& (&* ssr) cflow
                                        (prog (next)
                                          (setq next (~to cf))
                                         L(setq next
                                            (~to (?& (&* body) cflow
                                                     (& (~seg next)))))
                                          (if (eq (~seg next) body)
                                              (return next))
                                          (go L)))))))
                     (t (setq incase (~case (~from cf))
                              newsubseg (~seg (~to cf))))))
             (setq found-exit T
                   ret-code
                   (reverse
                     (or (cdr (case-action xcase))
                         (and df-retval@io (5rep (5source df-retval@io)))))
                   action-code
                   [!!(5assign (& dfjoin 'body) 'outside-input)
                    !!(car (case-action xcase))
                    !!(reverse (cdr ret-code))]
                   ret-code
                   [['return !(if ret-code [(car ret-code)])]])
             (setq code
                   (append code
                           (5codepred subseg
                                      [[incase]
                                       [outcase action-code
                                                ! ret-code]])))))
      (setq subseg newsubseg))
    (setq code (funcall *loop-code-template* icode code))
    (if (and (null found-exit) df-retval@io) ;kludge for infinite loops
        (setq code (append code (5rep (5source df-retval@io)))))
    (setf (CODE-FOR body) code)                          ;this MUST be
    (5simple-stuff-at-end ssr (if (5put-here body) code))))
```

```
;this fn follows back on dflow in order to find the @port which
;produces something.  NOTE THAT IT MAY RETURN AN @IO, OR AN
;@CREATEVAR, WHICH DO NOT USE THE SAME SELECTOR FOR NAME!!! {THIS IS A
;CLUDGE}  Its input is always an @io!

(defun 5source (@port)
  (let ((sourceio (~01 ~from (??& * dflow * (~deep @port)))))
    (cond ((null sourceio) ;make dummy @io
           ;kludge should we yell instead?
           (& (~deep @port) @io 'missing nil nil))
          ((~01 (??& sourceio @createvar)))
          (T (?& sourceio @io)))))

;this determines what the code corresponding to an @port (@io, or
;@createvar) is.  It does copying if it is not a returnval so that the
;code for the program stays a tree.  (Note we are assuming that a
;returnval will only be used one place).  (5rep returns a list, 5rep1
;returns a single cons.)  Note that this bashes CODE-FOR so that
;the return value will only be used in one place.

(defun 5rep (@port)
  (cond ((@createvar-fact? @port) [(~varname @port)])
        ((eq (~kind @port) 'returnval)
         (let ((code (CODE-FOR (~seg~segcase (~deep @port)))))
           (when (null code)
             (barf "5REP: returnval has no code.~% Data: ~:=" @port))
           (setf (CODE-FOR (~seg~segcase (~deep @port))) nil)
           code))
        (t (if (~name @port) [(~name @port)]))))

(defun 5rep1 (@port) (5make1 (or (5rep @port) (list '.V.))))
```

```
;this takes care of impedence matching at segment boundries.
;it work hard to do minimal and aesthetic setqs.  (It uses a PSETQ
;if more than one setq has to be done in order to avoid figuring out
;what order to use in a setq.)

(defun 5assign (segcase type)
  (caseq type
    (outside-input
     (5assign1 (over (??& (&* segcase input) @io 'free-var)
                     nconc (5assign-pair (~name _) (5source _)))))
    (inside-input
     (5assign1 (over (??& (&* segcase input) @createvar)
                     nconc (5assign-pair (~varname _) (?& (~deep _) @io)))))
    (inside-output
     (5assign1 (over (??& (&* segcase output) @io 'free-var)
                     nconc (5assign-pair (~name _) (5source _)))))
    (outside-output
     (let ((pairs
            (over (??& segcase output) -> out
                  (~01 (??& out @createvar)) -> @createvar
              nconc (cond (@createvar
                           (5assign-pair (~varname @createvar)
                                         (?& (~deep @createvar) @io)))))))
       (5assign1 pairs)))))

(defun 5assign-pair (p-name s)
  (cond ((fact-key-dispatch s
           (@io (or (eq (~kind s) 'returnval)
                    (and (~name s) (not (eq p-name (~name s))))))
           (@createvar (not (eq p-name (~varname s)))))
         [[p-name ! s]])))

;this takes 0 or more setq pairs and makes code.  the multi-value
;thing is a cludge at present.  It just works in one special case that
;comes up when a cliche has a whole in it for an ssr

(defun 5assign1 (pairs)
  (setq pairs
        (over pairs -> [n ! s]
              (5rep1 s) -> rep
          select (and (not (eq rep n)) (not (equal rep '(5noop ,n))))
          list [n rep]))
  (cond ((null pairs) nil)
        ((null (cdr pairs)) [['setq ! (car pairs)]])
;we should be doing a psetq here to be really safe.
        (t (over (partial-order-sort (sortcar pairs #'alphalessp)
                                     #'5psetq-order)
             list (cons 'setq _)))))

;note you must use partial-order-sort with this

(defun 5psetq-order (pair1 pair2)
  (5po1 (cadr pair1) (car pair2)))

;this is used elsewhere

(defun 5po1 (list var)
  (do ((from list (cdr from)))
      ((null from) nil)
    (cond ((eq var from) (return T))
          ((not (listp from)) (return nil))
          ((5po1 (car from) var) (return T)))))
```

```
;Note that the ordinary sort assumes that its predicate is
;transitive.  This does not assume this.  It only assumes that
;the prediate correctly orders individual pairs.  we do assume that
;the predicate is non-cyclic in its ordering.  The sort is stable
;in that it never changes the order of anything unless it has to.

;the algorithm is terribly slow, but works ok under the assumption that
;it will cause VERY little change in order.  Note that you have
;to do N squared comparisons at the very least any way.

;what the algorithm does is to look and see if it finds any item
;that is supposed to be before something that it is after.
;if it doesn't find any then it is done.  Otherwise, it moves the thing
;to the new place and just starts again.

(defun partial-order-sort (list predicate)
  (let* ((work-list (cons nil list))
         (len (length work-list)))
    (prog ()
      L (do ((i 2 (1+ i))) ((>= i len) (return))
          (let ((item (nth i work-list)))
            (do ((j 1 (1+ j))) ((>= j i) (return))
              (when (funcall predicate item (nth j work-list))
                (pop (nthcdr i work-list)) ;not must be first since i>J
                (push item (nthcdr j work-list))
                (go L))))))
    (cdr work-list)))
```

```
;little utilities of one kind or another.

;prevent progns from nesting inside each other
;Note arg is a CONS.  Result is s list of CONSs.

(defun 5progn-trans (arg)
  (cond ((and (not (atom arg)) (eq (car arg) 'progn)) (cdr arg))
        (t [arg])))

;make a progn if need be.  note takes in a list and returns a cons

(defun 5make1 (code)
  (cond ((cdr code) ['progn ! code])
        (t (car code))))

;use a prog2 or prog1 if you must to get the right value back
;return LIST of CONSs

(defun 5make-prog2 (return-code final-code)
  (cond ((null final-code) return-code)
        ((null return-code) final-code)
        ((5null-computation return-code) [!! final-code nil])
        (t
         (if (cddr return-code) (setq return-code [(5make1 return-code)]))
         [[(cond ((cdr return-code) 'prog2)
                 (t 'prog1))
           !! return-code ! final-code]])))

;determine whether a seg must be coded inline, or whether it will
;get nested in some argument list.  Note that we are just assuming
;that side-effects are going to work out ok if there is retvar dflow.

(defun 5put-here (seg)
  (let* ((ret (~01 ~deep (??& (&* (&* seg) output) @io 'returnval))))
    (or (null ret)
        (??& ret @createvar)
        (caseq (5region-classify ret)        ;to hack incompleteness
          (ignore T)
          (head (not (assoc ret regions)))))))
```

```
;oldname may be a lisp function or a fortran one convert it to a lisp one.
;if it is a user name just leave it alone.

(defun 5get-lisp-name (oldname)
  (let* ((new
          (assq oldname
           '((_lt . minusp)(_eq . zerop)(_gt . plusp)
             (_+i . +)(_-i . -)(_*i . *)(_//i . //)(_**i . ^)(_u-i . -)
             (_+r . +/$)(_-r . -/$)(_*r . */$)(_//r . ///$)
             (_**r . expt)(_**ri . ^/$)(_u-r . -/$)
             (_+d . +/$)(_-d . -/$)(_*d . */$)(_//d . ///$)
             (_**d . expt)(_**di . ^/$)(_u-d . -/$)
             (_itor . float)(_itod . float)(_rtoi . fix)(_dtoi . fix)
             (_rtod . 5noop)(_dtor . 5noop)
             (_exp . exp)(_dexp . exp)(_alog . log)(_dlog . log)
             (_sin . sin)(_dsin . sin)(_cos . cos)(_dcos . cos)
             (_sqrt . sqrt)(_dsqrt . sqrt)
             (_iabs . abs)(_abs . abs)(_dabs . abs)
             (_float . float)(_dfloat . float)(_sngl . 5noop)
             (_dble . 5noop)(non-null . 5noop)(5variable . 5noop)
             (and-lexpr . and)(or-lexpr . or)))))
    (cond (new (cdr new))
          (t oldname))))

;oldconstant may be a lisp one or a fortran one.  make it a lisp one.

(defun 5get-lisp-constant (oldconstant)
  (cond ((atom oldconstant)
         (barf "5GET-LISP-CONSTANT: doubtful constant.~% Data: ~S"
                 oldconstant))
        ((eq (car oldconstant) 'quote) oldconstant)
        ((memq (car oldconstant) '(_i _r _d)) ''.(cadr oldconstant))
        (t
         (barf "5GET-LISP-CONSTANT: doubtful constant.~% Data: ~S"
                 oldconstant))))
```

```
;this fn addresses two problems that come up because the non-temporal
;plan is coded, when we want correct annotation
;of the temporal plan in the original context.
;NOTE THAT AS A SIDE EFFECT, THIS FN PERMANENTLY RESTORES THE CONTEXT
;TO THE ORGINAL CONTEXT.
;1. all of the @seg and @io facts have to be moved to the correct context.
;   note that it assumes that if a seg is in both contexts, then all of
;   its ports are the same inteh two places.
;2. There are quite a few segments in the temporal view which are not in the
;   non-temporal view.  Annotation is provided for them by copying from
;   segments which are in that view.  Most notably annotation must be
;   added to the temporal fragments.
;the changes happen on a loop by loop basis since that is where all the
;action is.
;  The argument ioc (created by temporal-compose) tells how all of the
;ports in the temporal view correspond to ports in the loop view.
;it is used to know which @ios to copy over.  The correspondance of segments
;isn't so obvious, and the actions taken are a bit add hoc.

;alist by segs of cons of @seg and list of @ios and @createvars.
(def-alistq loop-info)

(defun 5fix-@assertions (top-seg ioc original-context)
  (let* ((loop-info (over (all-subsegs top-seg) -> seg
                      bag [seg (?& (&* seg) @seg)
                               !!(??& (&* (&* seg) port) @io)
                               !!(??& (&* (&* seg) port) @createvar)])))
    ;;note only set here to keep loop facts in index.
    (set-db-context original-context)
    (over (reverse (all-subsegs top-seg)) -> seg  ;bottom up
          (cdr (assq seg loop-info)) -> [@seg ! @port-facts]
      (--& (&* seg) @seg)
      (--& (&* (&* seg) port) @io)
      (--& (&* (&* seg) port) @createvar)
      (cond (@seg
             (make-fact-be-in @seg)
             (over @port-facts
               select (fact-is-in? (~deep _))
                      (make-fact-be-in _))
             (if (null (?@seg-code seg))
                 (>@seg-code seg (5estimate-code seg)))
             (over [!(??& (&* seg) input) !(??& (&* seg) output)]
                   (if (null (??& _ @io))
                       (if (null (??& _ @io))
                           (>& _ @io 'free-var nil nil))))
             (over [!(??& (&* seg) tinput) !(??& (&* seg) toutput)]
                   (if (null (??& _ @io)) (>& _ @io 'free-var nil nil))))
            (T (5estimate-@info seg loop-info ioc))))
    (restore-db-context original-context)))

(defun 5estimate-code (seg)
  (let ((inner-code (over (??subseg seg) append (?@seg-code _))))
    (over inner-code -> sexpr
      select (and (not (atom sexpr))
                  (over inner-code
                    fastand (not (5sexpr-in (5root-sexpr sexpr)
                                            (5root-sexpr _)))))
      bag sexpr)))

(defun 5root-sexpr (list)
  (cond ((and (listp list) (memq (car list) '(*code-is-car* *code-is-cdr*)))
         (5root-sexpr (cdr list)))
        (t list)))

(defun 5sexpr-in (sexpr list)
  (and list (listp list)
       (or (eq sexpr (car list))
           (5sexpr-in sexpr (car list))
```

```
        (eq sexpr (cdr list))
        (5sexpr-in sexpr (cdr list)))))
```

```
(defun 5estimate-/@info (seg loop-info ioc)
  (let* ((role-facts (??& * role (&* seg))))
    (caseq (~01 ~rolename role-facts)
      ((bssr gen acc trn term filter)
       (>& (& seg) @seg 'opencode nil
           (cond ((?role seg 'op) (?@seg-code (?role seg 'op)))
                 ((??& (&* seg) plantype 'tempcomp)
                  (5estimate-code seg)))))
      (body
       (>& (& seg) @seg 'opencode nil
           (let ((op (?role (~seg~segcase (car role-facts)) 'op)))
             (if op (?@seg-code op)))))
      (dfjoin
       (let* ((ssr (~seg~segcase (car role-facts)))
              (supseg (?supseg ssr))
              (code (or (?@seg-code ssr)
                        (and supseg (eq (?plantype supseg) 'tempcomp)
                             (?@seg-code supseg)))))
         (>& (& seg) @seg 'join nil
             '((*CODE-IS-CAR* . ,(memq 'LP (car code)))))))
      (rec
       (let* ((ssr (~seg~segcase (car role-facts)))
              (ssr-code (car (?@seg-code ssr)))
              (supseg (?supseg ssr))
              (tc-code (and supseg (eq (?plantype supseg) 'tempcomp)
                            (car (?@seg-code supseg))))
              (loop (cond ((and (listp tc-code)
                                (eq (car tc-code) 'PROG))
                           tc-code)
                          ((and (listp ssr-code)
                                (eq (car ssr-code) 'PROG))
                           ssr-code))))
         (>& (& seg) @seg 'recursion nil [(car (last loop))])))
      (nil ;can be other random segs in bssr's ignore them
       (when (and (eq (?segtype seg) 'join)
                  (eq (?plantype (?supseg seg)) 'ssr-body))
         (>& (& seg) @seg 'join nil nil)))
      (T (>& (& seg) @seg 'opencode nil (5estimate-code seg))))
    (over [!(??& (&* seg) input) !(??& (&* seg) output)
           !(??& (&* seg) tinput) !(??& (&* seg) toutput)] -> port
          (cdr (assq port ioc)) -> loop-port
          (if loop-port
              (cddr (assq (~seg~segcase loop-port) loop-info))) -> loop@s
          (over loop@s
            (if (and (eq (~deep _) loop-port) (@io-fact? _))
                (return _))) -> loop@io
          (over loop@s
            (if (and (eq (~deep _) loop-port)
                     (@createvar-fact? _))
                (return _))) -> loop@createvar
      (fact-key-dispatch port
        ((input output)
         (cond ((null loop@io) (>& port @io 'free-var nil nil))
               (T (assert-fact (&~deep loop@io port))))
         (if loop@createvar (assert-fact (&~deep loop@createvar port))))
        ((tinput toutput)
         (>& port @io 'free-var ;VIEW cares that this isn't an actual-arg
             (or (and loop@createvar (~varname loop@createvar))
                 (and loop@io (~name loop@io))
                 (and (~lower-io port)
                      (~01 ~name (??& (~lower-io port) @io)))
                 (and (null (~lower-io port))
                      (null (??& (&* seg) toutput * * port))
                      'dummy)) nil))))))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp REPEAT Indent:1;
;;; Lisp REPEAT* Indent:1;
;;; End:;
```